RWTH Aachen University
Department of Computer Science

Closing the Bandwidth Gap: Custom CUDA Kernels for Tensor Network Contractions on A100 GPUs

Master Thesis

submitted by

Daniel Sinkin
Matriculation Number: 367316

First Examiner: Prof. Dr. TODO
Second Examiner: Prof. Dr. TODO
Advisors: Dr. Edoardo Di Napoli
External Institution: Jülich Supercomputing Centre (JSC)
Forschungszentrum Jülich

Aachen, March 6, 2026

Abstract

TODO

Acknowledgements

TODO

Contents

List of Figures

List of Tables

Listings

List of Symbols

Symbol	Meaning
$\mathcal{T}$	Generic tensor
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Matrices
$\vec{v}$	Vector
$\Psi(i_1, \ldots, i_L)$	Many-body wavefunction amplitude
L	Number of lattice sites / modes
d	Local physical dimension per site
χ	Bond dimension (MPS rank limit)
$A^{[\ell]}$	Local MPS tensor at site ℓ
α_ℓ	Virtual bond index between neighboring tensors
$N_{\text{state}} = d^L$	Number of coefficients in full-state representation
$N_{\text{MPS}} \sim L\, d\, \chi^2$	Parameter count of a uniform-bond MPS
$S = -\sum_a s_a^2 \log s_a^2$	Entanglement entropy from Schmidt coefficients
ϵ_{trunc}	Truncation error after rank reduction
M, N, K	GEMM dimensions ($\mathbf{A} \in \mathbb{R}^{M \times K}$, $\mathbf{B} \in \mathbb{R}^{K \times N}$)
$F = 2MNK$	Floating-point operation count for dense GEMM
Q	Data movement volume (bytes)
$I = F/Q$	Arithmetic intensity (FLOPs per byte)
P	Achieved performance (FLOPs/s)
$P_{\max}$	Theoretical peak compute throughput
$B_{\max}$	Peak memory bandwidth
$I^\star = P_{\max}/B_{\max}$	Roofline ridge point
$\eta = P/P_{\max}$	Compute efficiency relative to peak
$T_{\text{H2D}}(\pi, V)$	Host-to-device transfer time for path class π and volume V
$T_0(\pi)$	Fixed path-dependent transfer latency term
$B_{\text{eff}}(\pi)$	Effective end-to-end transfer bandwidth
V	Transfer volume in bytes
π	Path class (local, same-socket remote, cross-socket)
n_{IF}	Number of additional Infinity Fabric segments in the path
τ_{IF}	Average latency penalty per Infinity Fabric segment
$\mathcal{O}(\cdot)$	Asymptotic upper bound

List of Abbreviations

BF16	Brain Floating Point (16-bit)
BLAS	Basic Linear Algebra Subprograms
CCX	Core Complex (AMD)
CCD	Core Complex Die (AMD)
CUDA	Compute Unified Device Architecture
DFT	Density Functional Theory
DMRG	Density Matrix Renormalization Group
FMA	Fused Multiply-Add
FP8/16/32/64	IEEE 754 Quarter/Half/Single/Double Precision
GEMM	General Matrix Multiply
GPU	Graphics Processing Unit
HBM	High Bandwidth Memory
HPC	High Performance Computing
MPI	Message Passing Interface
MPS	Matrix Product State (tensor networks)
NUMA	Non-Uniform Memory Access
NVLink	NVIDIA GPU Interconnect
PCIe	Peripheral Component Interconnect Express
SFU	Special Function Unit
SIMD	Single Instruction, Multiple Data
SIMT	Single Instruction, Multiple Threads
SM	Streaming Multiprocessor
SVD	Singular Value Decomposition
TF32	TensorFloat-32
TN	Tensor Network
ULP	Unit in the Last Place

Glossary

Arithmetic intensity
The ratio of floating-point operations to bytes transferred from memory, typically measured in FLOPs/byte. Determines whether a kernel is compute-bound or memory-bound (see *roofline model*).

Bank conflict A shared memory access pattern in which multiple threads in a warp address different words in the same memory bank, forcing the accesses to be serialised.

Bond dimension In tensor networks, the size of an index shared between two tensors. Larger bond dimensions permit more accurate representations but increase computational and memory cost.

Coalescing The hardware mechanism by which individual memory requests from threads in a warp are combined into a minimal number of cache-line transactions. Requires consecutive threads to access consecutive addresses.

Contraction The generalisation of matrix multiplication to tensors: a pairwise summation over one or more shared indices between two tensors, producing a new tensor.

Kernel A function written in CUDA that executes in parallel across many GPU threads. Launched from the host (CPU) and runs on the device (GPU).

Machine epsilon The smallest floating-point number ε such that $1 + \varepsilon \neq 1$ in a given format. Characterises the relative spacing of representable values near 1.

Mixed precision A computational strategy that uses lower-precision arithmetic (e.g. FP16, BF16, TF32) for the bulk of computation while maintaining higher-precision accumulators (e.g. FP32) to preserve numerical accuracy.

Occupancy The ratio of active warps on an SM to the maximum number of warps the SM supports. Higher occupancy generally improves latency hiding but is not always necessary for peak performance.

Roofline model A performance model that bounds achievable throughput by the minimum of peak compute throughput and peak memory bandwidth multiplied by arithmetic intensity. Used to classify kernels as compute-bound or memory-bound.

Shared memory Fast, on-chip memory visible to all threads within a CUDA thread block. Used as a programmer-managed cache and for inter-thread communication.

SIMT Single Instruction, Multiple Threads. NVIDIA's execution model in which a warp of 32 threads executes the same instruction simultaneously, similar to SIMD in Flynn's taxonomy but with the ability for individual threads to follow divergent control-flow paths (at the cost of serialisation).

Tensor core A specialised hardware unit on NVIDIA GPUs that performs small matrix multiply-accumulate operations (e.g. 4×4 or 8×8) in a single cycle, providing significantly higher throughput than standard CUDA cores for supported precisions.

Tiling A loop transformation that partitions a computation into smaller blocks (tiles) to improve data locality and reuse in caches or shared memory.

Warp A group of 32 threads that execute in lock-step on the same SM processing block. The warp is the fundamental scheduling unit on NVIDIA GPUs.

Warp divergence
A performance penalty that occurs when threads within a warp take different branches of a conditional. The hardware serialises the divergent paths, reducing effective parallelism.

1. Introduction

1.1. Motivation

Tensor-network simulations increasingly rely on GPU-accelerated dense kernels, yet practical workloads often run in a regime where library defaults are not fully efficient: matrix blocks are small or irregular, kernel sequences are short, and memory-layout transforms are frequent. In this regime, launch overhead and data movement can dominate the total runtime, with occupancy limits amplifying the effect.

This work is conducted in collaboration with the Jülich Supercomputing Centre (JSC), where partner teams in computational physics require reliable performance gains on production HPC systems. The immediate hardware target is Ampere-class GPUs, in particular A100 nodes, with a conservative baseline of one node (four A100-SXM4-40GB GPUs connected via NVLink).

The goal is practical: build a profiling-driven optimisation process that produces measurable speedups for tensor-network contraction workloads on current JSC hardware and remains transferable to next-generation accelerators.

1.2. Problem Statement

The central problem is to identify and optimise the CUDA kernels and execution patterns that dominate runtime in tensor-network contraction workflows, while keeping the scope on HPC implementation aspects rather than physics modelling.

The work is organised around the following technical questions:

1. Which kernel patterns are the primary bottlenecks on A100 for the target workload distribution (small/medium contractions, layout transforms, short kernel sequences)?
2. How much performance can be recovered by reducing launches and fusing short kernels? How much additional gain comes from layout and occupancy-aware tuning?
3. How closely do optimised kernels approach hardware limits predicted by bandwidth and throughput models, and where do residual stalls remain?

These questions are addressed with reproducible benchmarks and Nsight profiling. Custom kernels are compared against vendor baselines (cuBLAS/cuTENSOR).

1.3. Contributions

Main contributions are:

1. A hardware-aware characterisation of tensor-network-relevant kernel patterns on Ampere GPUs, with emphasis on launch-dominated and bandwidth-limited regimes.
2. An implementation strategy for fused and layout-aware CUDA kernels, including explicit occupancy and memory-hierarchy trade-off analysis.
3. A profiling workflow that links observed performance counters to concrete optimisation decisions and documents where bottlenecks shift after each optimisation step.
4. A benchmark and comparison framework against cuBLAS/cuTENSOR baselines on a realistic single-node A100 environment.

1.4. Outline

Chapter 2 introduces the CUDA and hardware concepts required for the optimisation work, and frames tensor networks only as computational motivation. Chapter 3 defines the target kernels and baseline choices, then introduces the data-layout strategy. Chapter 4 details the kernel design and integration choices used to realise the optimisation approach. Chapter 5 reports benchmark and profiling results, including launch overhead experiments and comparison against library baselines. Chapter 6 summarises the findings, states the main limitations, and defines the next optimisation steps.

2. Background

This chapter summarizes the HPC background needed for the optimisation work: GPU architecture, CUDA execution, and prior kernel-engineering results. Tensor networks appear only as computational context.

2.1. Tensor Networks

2.1.1. Historical Background

Tensor networks entered computational physics through the density-matrix renormalization group (DMRG) in the early 1990s, where the key practical idea was to keep only the most relevant subspace during iterative updates [Whi92, Whi93]. During the 2000s, this was reformulated in explicit tensor-network language, especially as matrix product states (MPS), which clarified the relation between approximation quality and bond dimension [Sch11, Oru14].

For high-performance computing, the first generation of tensor-network codes was CPU-focused and built around BLAS/LAPACK kernels plus MPI-based distribution. Over time, the contraction-heavy parts moved to GPUs, where cuBLAS/cuTENSOR-style kernels and communication-aware scheduling became central for strong scaling across devices [SSK17, ML+23b, ML+24b]. The algorithmic core stayed similar, but the main performance bottlenecks shifted toward memory movement and launch overhead, then to multi-GPU orchestration at larger scale.

2.1.2. Why Tensor Networks Are of Interest

Tensor networks are useful because they replace exponential state representations with structured low-rank factorizations when entanglement is limited. For a chain of L sites with local dimension d, the full state vector requires

$$N_{\text{state}} = d^L$$

complex amplitudes. In contrast, a uniform-bond MPS uses

$$N_{\text{MPS}} \sim L\, d\, \chi^2,$$

which is polynomial in system size for fixed bond dimension χ.

This reduction is directly tied to entanglement structure. Across one bipartite cut, an MPS corresponds to a truncated Schmidt decomposition

$$|\Psi\rangle = \sum_{a=1}^{\chi} s_a \, |a_L\rangle \, |a_R\rangle \,,$$

and the von Neumann entropy

$$S = -\sum_a s_a^2 \log s_a^2$$

is effectively controlled by the retained rank χ.

From an HPC perspective, tensor networks are also interesting because they stress modern GPUs through dense contractions with repeated reshapes inside iterative workflows dominated by short kernels. This makes them a good case for studying kernel efficiency and orchestration costs together [ZSW20, A+23].

2.1.3. Computational Context

This section treats tensor networks as an *application source of computational patterns*, not as a physics topic. Detailed notation and model-specific physics choices are outside scope here and are introduced separately with domain experts.

For high-performance computing, the relevant point is that tensor-network codes repeatedly execute dense linear-algebra-like kernels on moderate to small tensor blocks inside long iterative loops. Both per-kernel efficiency and orchestration overhead therefore matter.

2.1.4. Matrix Product States (MPS) from a Computational View

Matrix Product States (MPS) are a common tensor-network representation where a high-order object is factorised into a chain of local tensors. The relevant point here is not the physics interpretation but the resulting compute structure:

- repeated contraction of small and medium dense tensors,
- frequent reshape/permutation steps between contraction phases,
- recurring low-rank truncation steps (typically SVD/QR-based),
- many short kernel sequences within iterative sweeps.

Using a minimal notation, an order-L MPS can be written as

$$\Psi(i_1, \ldots, i_L) = \sum_{\alpha_1,\ldots,\alpha_{L-1}} A^{[1]}_{i_1,\alpha_1} A^{[2]}_{\alpha_1,i_2,\alpha_2} \cdots A^{[L]}_{\alpha_{L-1},i_L},$$

where i_ℓ is a physical index and α_ℓ is a bond index. The bond dimension χ controls both approximation quality and computational cost.

From an HPC perspective, two scaling facts are useful planning rules:

1. storage for a uniform MPS scales as $\mathcal{O}(L\,d\,\chi^2)$ rather than exponentially in L,
2. dominant kernels in common MPS updates scale roughly with $\mathcal{O}(d\,\chi^3)$ per local update step, so increasing χ quickly shifts pressure to memory traffic and tensor-factorisation kernels.

A typical two-site update can be sketched with a simple cost model:

$$\mathrm{FLOPs}_{\mathrm{update}} \approx c_1\,d\,\chi^3 + c_2\,d^2\,\chi^2,$$

where the first term represents contraction work and the second represents truncation/-factorization work. Constants c_1, c_2 depend on layout and update scheme, but the scaling makes clear why χ is the dominant performance driver.

In practical implementations, local updates are often executed as:

1. fuse neighboring local tensors into an effective two-site tensor,
2. contract with environment/operator terms,
3. reshape to a matrix and run SVD/QR-style truncation,
4. redistribute factors back into MPS form.

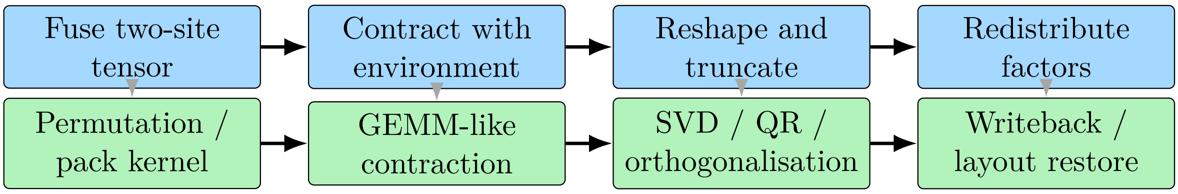


Figure 2.1.: Two-site MPS update pipeline and its kernel-level mapping. This is a direct bridge between tensor-network algorithm steps (top) and GPU optimisation targets (bottom).

The same structure appears in the optimisation targets of later chapters: GEMM-like contractions and layout transforms, together with launch-overhead control and profiling-guided tuning of memory/occupancy trade-offs.

2.1.5. Kernel Patterns Relevant to This Work

The optimisation work is driven by three recurring patterns:

1. **Small and medium dense contractions.** After index reshaping, many contractions reduce to GEMM-like operations with dimensions that are too small to saturate the GPU when launched individually.
2. **Layout transforms.** Permutations and reshapes are required to match library expectations (e.g. column-major interfaces), and these transforms can dominate runtime if memory access is not coalesced.
3. **Fine-grained kernel sequences.** Practical workflows execute large numbers of short kernels; launch overhead and synchronisation overhead become first-order effects in this regime (see Sections 2.4.5 and 5.2).

The profile aligns directly with CUDA topics such as occupancy control, shared-memory tiling and launch fusion, plus host-device scheduling.

2.1.6. Scope Boundary

To keep the thesis focused and technically coherent, we explicitly exclude:

- derivations of tensor-network physics models,
- detailed notation systems used in specific communities,
- algorithmic comparisons driven primarily by physics accuracy criteria.

Instead, kernels are evaluated by HPC criteria: runtime, achieved throughput, memory efficiency, and scaling behaviour on Ampere-class GPUs.

2.2. Mathematical Foundations for Performance Evaluation

This section collects the mathematical tools used for performance interpretation and numerical reliability. It is not a full theory treatment; the goal is a compact framework that links measurements to well-defined models.

2.2.1. Work, Data Movement, and Runtime Bounds

For a kernel with floating-point work F (FLOPs) and main-memory traffic Q (bytes), a first-order runtime model is

$$T \approx \max\left(\frac{F}{P_{\text{eff}}},\, \frac{Q}{B_{\text{eff}}}\right) + T_{\text{launch}},$$

where P_{eff} and B_{eff} are achieved compute and bandwidth rates, and T_{launch} is host/device dispatch overhead. The first term is a roofline-style throughput bound, while the additive launch term follows the latency-plus-throughput structure used in communication models such as Hockney's [WWP09, HJ88].

The corresponding arithmetic intensity

$$I = \frac{F}{Q}$$

is a primary explanatory variable because it determines whether a kernel is expected to be memory-bound or compute-bound [WWP09]. Under ideal sustained ceilings $(P_{\max}, B_{\max})$, throughput is bounded by

$$P \leq \min(P_{\max},\, B_{\max} I)\,.$$

This gives the standard ridge point

$$I^{\star} = \frac{P_{\max}}{B_{\max}},$$

which separates memory-bound and compute-bound regimes.

2.2.2. Parallel Scaling Metrics

Speedup $S_p = T_1/T_p$ and parallel efficiency $E_p = S_p/p$ quantify how well a workload exploits p processing units. Amdahl's law [Amd67] bounds strong-scaling speedup by the serial fraction, while Gustafson's relation [Gus88] gives the corresponding weak-scaling perspective. Both bounds are stated in full in Section C.1 and applied to node-level results in Section 5.4.

2.2.3. Floating-Point Error Model

For one floating-point operation, we use the standard model

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \leq u,$$

with unit roundoff u and $\circ \in \{+, -, \times, /\}$.

Conditioning explains why addition/subtraction can be much more sensitive than multiplication. A local relative condition number for addition is

$$\kappa_{\mathrm{add}}(x, y) = \frac{|x| + |y|}{|x + y|},$$

which becomes large under cancellation ($x \approx -y$). By contrast, multiplication has $\kappa_{\mathrm{mul}}(x, y) = 1$ under standard relative perturbation analysis [Hig02]. A concise engineering-focused discussion of conditioning and cancellation is also given in [DR22]. For dot products, a useful problem-condition measure is

$$\kappa_{\mathrm{dot}}(x, y) = \frac{|x|^\top |y|}{|x^\top y|},$$

which again grows when positive and negative contributions cancel.

Two concrete values make the conditioning effect explicit:

$$\kappa_{\mathrm{add}}(1, 0.8) = \frac{|1| + |0.8|}{|1 + 0.8|} = \frac{1.8}{1.8} = 1,$$

while under near-cancellation

$$\kappa_{\mathrm{add}}(1, -0.999999) = \frac{|1| + |-0.999999|}{|1 - 0.999999|} = \frac{1.999999}{10^{-6}} \approx 2 \times 10^6.$$

The corresponding multiplication stays locally well-conditioned:

$$\kappa_{\mathrm{mul}}(1, -0.999999) = 1.$$

For a length-k dot product, a common bound is

$$\left|\mathrm{fl}(x^\top y) - x^\top y\right| \le \gamma_k \, |x|^\top |y|, \qquad \gamma_k = \frac{ku}{1 - ku},$$

which makes explicit why long reduction chains are sensitive to precision choice. This is one reason why FP32 accuracy checks are kept in the workflow even when throughput is the primary objective [HM22].

This also motivates fused multiply-add and FP32 accumulation in tensor-core paths: IEEE-754 FMA semantics avoid intermediate rounding between multiply and add [IEE19], and Ampere TF32 tensor-core execution keeps accumulation in FP32 [NVI20a], reducing the impact of repeated low-precision reduction steps.

How far can FP32 be pushed? Two failure modes bound the useful range of single precision.

Complete absorption. Absorption occurs when a small addend falls below the unit in the last place (ULP) of the larger operand and is rounded away entirely. In FP16 this happens easily: Section C.3.2 shows that $\mathrm{fl}_{16}(1024 + 0.5) = 1024$ because the exponent gap of 11 exceeds the 10 fraction bits. In FP32 the 23 fraction bits provide far more headroom: absorption requires an exponent difference of at least 24, so $\mathrm{fl}_{32}(2^{24} + 1) = 2^{24}$ (the addend 1 is exactly one ULP below the representable granularity at magnitude $2^{24} = 16\,777\,216$). In practice, absorption in FP32 only becomes a concern when accumulating values whose magnitudes span roughly seven orders of magnitude or more.

Accumulated rounding in long reductions. For a length-k summation of positive values, the γ_k bound above gives a worst-case relative error of $ku/(1 - ku)$ with $u = 2^{-24} \approx 5.96 \times 10^{-8}$ for FP32. At $k = 1000$ this evaluates to $\gamma_{1000} \approx 6 \times 10^{-5}$, meaning roughly 4 out of 7–8 decimal digits remain reliable. At $k = 10\,000$ the bound grows to $\gamma_{10000} \approx 6 \times 10^{-4}$, leaving only 3 reliable digits. For comparison, FP64 ($u = 2^{-53}$) at $k = 1000$ gives $\gamma_{1000} \approx 10^{-13}$, keeping 13 digits intact. In the tensor contractions studied here, reduction lengths are typically in the low hundreds (set by the contraction dimension K), so FP32 retains 5–6 reliable digits per contraction—well above the 10^{-4} to 10^{-6} truncation error inherent in finite bond-dimension approximations (Section 2.3.3).

2.2.4. Timing Statistics

All runtime measurements in this work use the sample median as point estimate and the median absolute deviation (MAD) for dispersion, both of which are robust to the heavy-tailed noise typical of GPU benchmarks [Jai91, KJ13]. Confidence intervals are obtained via nonparametric bootstrap resampling, avoiding normality assumptions [ET94]. Full definitions of the estimators, the robust relative spread (RRS) criterion, and the bootstrap procedure are given in Section C.2.

2.3. GPU Architecture

2.3.1. Streaming Multiprocessor and Warp Execution

The Streaming Multiprocessor (SM) is the fundamental compute unit of the GPU. Each A100-SXM4 GPU exposes 108 active SMs. The GA100 die itself has 128 physical SM partitions, with 20 disabled through harvesting/binning (see Section D.2), and all CUDA threads ultimately execute on one of the active SMs. Understanding the internal structure of an SM is essential for reasoning about occupancy, register pressure, and warp scheduling.

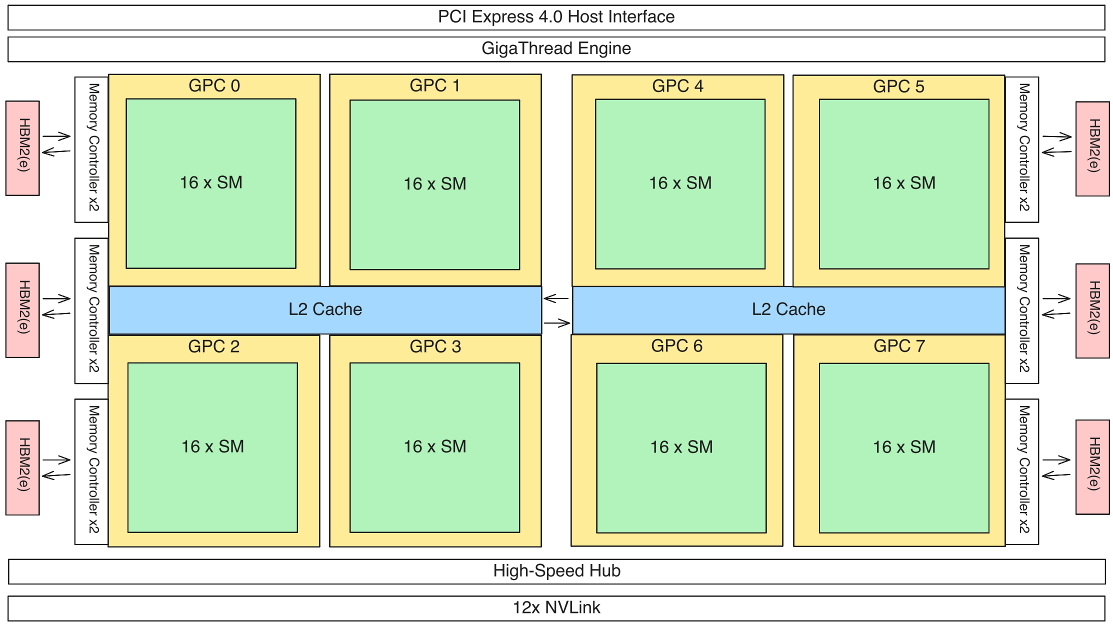


Figure 2.2.: Full GA100 floorplan schematic with graphics processing clusters (GPCs), L2 cache fabric, memory controllers, and NVLink/PCIe interfaces. This chip-level view provides the context for the SM-level detail in Figure 2.3.

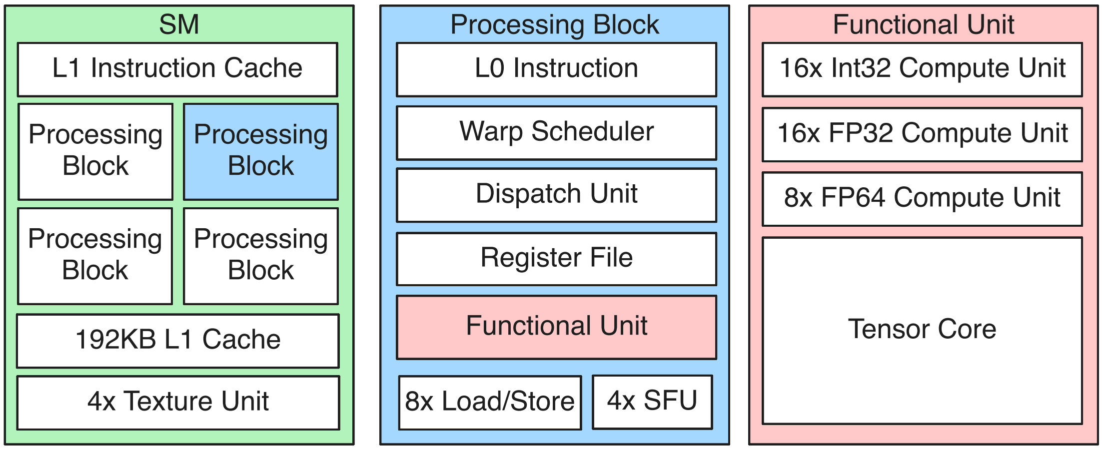


Figure 2.3.: Internal structure of an A100 Streaming Multiprocessor at three levels of detail. **Left:** the SM contains four processing blocks sharing an L1 data cache and shared memory. **Centre:** each processing block includes a warp scheduler, register file slice, execution units, LD/ST units, SFUs, and a texture unit. **Right:** the execution units comprise 16 INT32 cores, 16 FP32 cores, 8 FP64 cores, and one tensor core.

Each SM is partitioned into four *processing blocks* (also called sub-partitions). As shown in Figures 2.2 and 2.3, GA100 is organized hierarchically from chip-level GPC groupings down to the internals of each SM. At the SM level, each processing block contains:

- One **warp scheduler** with a single dispatch unit, capable of issuing one instruction per cycle from the warp it selects.

- A slice of the **register file** (16 384 32-bit registers per block, 65 536 per SM).
- **Execution units:** 16 INT32 cores, 16 FP32 cores, 8 FP64 cores, and one third-generation tensor core.
- **8 LD/ST units** for issuing memory load and store operations.
- **4 SFUs** (Special Function Units) that execute reciprocal ($1/x$), reciprocal square root ($1/\sqrt{x}$), and transcendental instructions (`sin`, `cos`, `exp`, `log`). Each SFU produces one result per cycle per thread, so a full warp of 32 threads requires 8 cycles on 4 SFUs. Because SFU throughput is much lower than the main FP32 pipeline, heavy use of transcendentals can become a bottleneck in kernels that otherwise appear compute-light.
- A **read-only/texture path** for cached read-mostly memory access.
- An **L0 instruction cache** private to the processing block.

The four processing blocks share the SM's L1 data cache and its configurable shared memory (up to 164 kB on the A100). Threads are grouped into *warps* of 32 and assigned to processing blocks at launch time. Each warp scheduler selects one of its resident warps per cycle and issues a single instruction to the appropriate execution unit. Because the four schedulers operate independently, an SM can have four instructions from four different warps in flight simultaneously, which is the primary mechanism for hiding memory and instruction latency.

For capacity accounting, the A100 limit of 64 resident warps (2048 threads) per SM implies up to 16 resident warps (512 threads) per processing block under an even split across the four sub-partitions. Across 108 active SMs, this gives a device-level maximum of 6912 resident warps and 221 184 resident threads.

The memory wall on GPUs. Despite the abundance of arithmetic units listed above, memory access patterns—not compute efficiency—are the dominant determinant of GPU kernel performance. This asymmetry has been understood since at least Wulf and McKee's observation that processor speed outpaces memory speed by a widening margin [WM95], and it has only intensified on GPUs: on the A100, peak FP32 throughput is 19.5 TFLOPS while peak HBM bandwidth is 1555 GB/s, giving a ratio of roughly 12.5 FP32 FLOPs per byte—any kernel whose arithmetic intensity falls below this ridge point is entirely bandwidth-limited and gains nothing from faster arithmetic. In practice, the majority of tensor-network contractions encountered in this work fall into or near the memory-bound regime, which means that optimising data layout, access coalescing, and on-chip reuse through shared-memory tiling yields far larger speedups than micro-optimising the compute pipeline. This observation guides the entire optimisation strategy developed in Chapters 3 and 4.

2.3.2. What Matters in A100 for This Work

Relative to the previous Volta generation, the A100 changes several hardware limits that matter directly for tensor-network kernels [NVI20a, NVI25d]:

- **Third-generation tensor cores and TF32 mode.** FP32 inputs can be accelerated on tensor cores through TF32 execution without changing storage format, which improves throughput for GEMM-like contractions.

- **Larger on-chip memory budget.** The A100 provides up to 164 kB shared memory per SM and a 40 MB L2 cache, increasing tile reuse opportunities and reducing pressure on HBM.

- **Asynchronous global-to-shared copy.** Ampere introduces hardware support for overlap-friendly copy pipelines (`cp.async`), which is important for latency hiding in custom kernels.

- **Stronger inter-GPU fabric.** NVLink 3.0 bandwidth makes multi-GPU contraction decomposition practical with lower communication penalty.

For performance modeling, we use the canonical roofline framework defined in Section 2.2. For A100-SXM4-40GB, the FP32 ridge point is approximately $I^\star \approx 12.5$ FLOPs/byte, used as the practical memory-vs-compute regime threshold in later optimisation chapters.

Compute capability and cross-generation compatibility. NVIDIA identifies each GPU architecture by a *compute capability* (CC) version that determines the available instruction set, hardware limits, and features. The A100 is CC 8.0 (Ampere); its predecessor V100 is CC 7.0 (Volta), and the successor H100 is CC 9.0 (Hopper).

Each CC increment can introduce instructions that are unavailable on older hardware. For example, CC 8.0 added hardware-accelerated asynchronous global-to-shared memory copies (`cp.async`) that do not exist on CC 7.0, while CC 9.0 introduced the Thread Block Cluster abstraction and the Tensor Memory Accelerator (TMA) for bulk asynchronous data movement that has no CC 8.0 equivalent. Conversely, higher-CC devices can usually run code compiled for a lower CC through forward compatibility, but without benefiting from new features.

Table 2.1 summarises the per-SM functional-unit counts across the three datacenter generations relevant to this work. The FP64:FP32 ratio is 1:2 on all three (full-rate FP64 is a datacenter feature; consumer parts such as CC 8.6 reduce it dramatically). Hopper doubles every major unit count relative to Ampere, but also doubles SM area, so per-SM efficiency remains similar; the aggregate gain comes from the higher SM count and faster clock.

Table 2.1.: Per-SM functional-unit counts for the three datacenter compute capabilities discussed in this work. Data compiled from NVIDIA architecture whitepapers [NVI17, NVI20a, NVI22].

Unit type (per SM)	V100 (CC 7.0)	A100 (CC 8.0)	H100 (CC 9.0)
FP32 CUDA cores	64	64	128
FP64 CUDA cores	32	32	64
INT32 cores	64	64	64
Tensor cores	8	4	4
SFU	16	16	16
LD/ST units	16	16	32
Active SMs (product)	80	108	132

In practice, this means kernels compiled for `sm_80` run on both A100 and H100, but kernels using H100-specific intrinsics (e.g. `cluster_sync`, TMA descriptors) will not run on A100 [NVI24b]. All kernels in this work target `sm_80` to maximise portability across CC $\geq$ 8.0 devices.

2.3.3. Floating-Point Formats and Precision Trade-offs

Scientific GPU workloads have often defaulted to FP64, following long-standing CPU-side numerical practice. A100 supports multiple floating-point formats with very different throughput, and for many contraction workloads FP32 or reduced precision is accurate enough at much lower cost. The following subsections summarize the relevant formats and motivate the precision choices used here.

IEEE 754 Binary Representation

The IEEE 754 standard [IEE19] defines the binary floating-point formats used across virtually all modern hardware. Modern NVIDIA GPUs follow IEEE 754 for the standard FP64/FP32/FP16 formats considered here [IEE19]. Each format encodes a real number as

$$(-1)^s \times 2^{e-\text{bias}} \times (1+f), \tag{2.1}$$

where s is a sign bit, e is an unsigned integer stored in the exponent field, the *bias* centres the exponent range around zero, and f is the fractional part of the significand (with an implicit leading 1 for normal numbers). The three fields are packed into a fixed-width bit string as shown in Table 2.2. The bias can be computed as

$$2^{k-1} - 1$$

where k is the number of exponent bits.

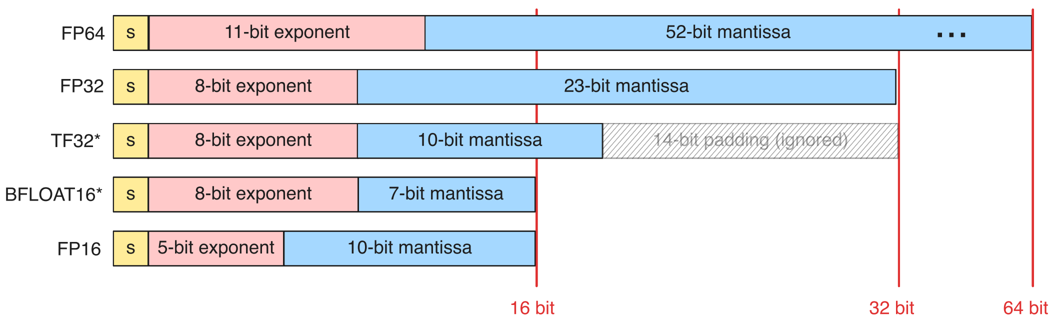


Figure 2.4.: Bit layout comparison of FP64, FP32, TF32, BF16, and FP16, with exponent and significand fields drawn to scale.

Table 2.2.: Bit layout of floating-point formats relevant to GPU computing. FP64, FP32, and FP16 are defined by IEEE 754 [IEE19]; BF16 and TF32 are industry-defined formats (see text). The mantissa column lists only the explicitly stored fraction bits; all formats carry an additional implicit leading bit for normal numbers.

Format	Total bits	Sign	Exponent	Bias	Mantissa[1]
FP64 (double)	64	1	11	1023	52
FP32 (single)	32	1	8	127	23
TF32	19	1	8	127	10
BF16 (bfloat16)	16	1	8	127	7
FP16 (half)	16	1	5	15	10
FP8 (E5M2)[2]	8	1	5	15	2
FP8 (E4M3)[2]	8	1	4	7	3

[1] IEEE 754 uses the term *significand*; *mantissa* is used here as it is more widely recognised.
[2] FP8 formats are not supported on the A100 (Ampere); they were introduced with the H100 (Hopper) architecture.

The number of exponent bits determines the *dynamic range* (the ratio of the largest to smallest representable magnitudes), while the number of significand bits determines the *precision* (the spacing between adjacent representable values). These properties are summarised in Table 2.3.

Table 2.3.: Numerical properties of floating-point formats on the A100. Machine epsilon (ε) is the smallest increment to 1.0 that produces a distinct value, i.e. $\varepsilon = 2^{-p}$ where p is the number of significand bits (including the implicit bit).

Format	Largest value	Smallest normal > 0	Machine epsilon
FP64	$\approx 1.8 \times 10^{308}$	$\approx 2.2 \times 10^{-308}$	$\approx 2.2 \times 10^{-16}$
FP32	$\approx 3.4 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	$\approx 1.2 \times 10^{-7}$
TF32	$\approx 3.4 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	$\approx 9.8 \times 10^{-4}$
BF16	$\approx 3.3 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	$\approx 7.8 \times 10^{-3}$
FP16	65504	$\approx 6.1 \times 10^{-5}$	$\approx 9.8 \times 10^{-4}$

Two observations are relevant here. First, TF32 and BF16 share the same 8-bit exponent as FP32, so they retain the FP32 dynamic range despite having fewer significand bits. TF32 was introduced for Ampere tensor cores: inputs are stored as FP32 values, while the tensor core internally truncates to 10 significand bits before multiply-accumulate, with accumulation in FP32 [NVI20a]. No explicit data conversion is required in user code. Precision/performance trade-offs for tensor cores have been measured in detail by Markidis et al. [MCL+18] and by Haidar et al. [HTDH18], who show that mixed-precision tensor-core paths can remain numerically robust when accumulation and solver structure are chosen carefully.

Second, FP16 has a severely limited dynamic range (maximum value 65504), which makes it unsuitable for many scientific workloads without careful scaling. BF16 ("brain floating point"), originally developed for deep learning training on Google's TPUs [KMM+19], avoids this limitation by retaining the FP32 exponent range at the cost of reduced precision (7 fraction bits versus 10 for FP16).

Practical Implications

Three effects of the bit-level representation are worth highlighting; step-by-step worked examples are given in Section C.3.

Absorption. When two operands differ by more exponent bits than the format has fraction bits, the smaller operand is rounded away entirely. In FP16 this happens easily (e.g. $\mathrm{fl}_{16}(1024 + 0.5) = 1024$), while FP32's 23 fraction bits make absorption a concern only when magnitudes span roughly seven orders of magnitude or more.

TF32 truncation error. Because TF32 tensor-core execution truncates inputs to 10 fraction bits before multiply-accumulate, individual entry errors are on the order of $\varepsilon_{\mathrm{TF32}} \approx 10^{-3}$, but FP32 accumulation and the statistical averaging over many products keep aggregate errors well below the truncation error of finite bond-dimension approximations [MCL+18, HTDH18, FHL+23].

Fused multiply-add (FMA). GPU tensor cores implement multiply-accumulate as a sequence of fused multiply-adds, applying only a single rounding step to round$(a \times b + c)$ rather than the two roundings of separate multiply and add. This yields tighter forward error, especially under cancellation [IEE19, Hig02, JLM17]. Recent tensor-core error analyses confirm the benefit for matrix products and blocked accumulation [FHL+23].

A100 Throughput by Precision

The performance gap between precisions is substantial. Table 2.4 reproduces the A100 throughput figures from NVIDIA's Ampere optimisation guide [RH20], broken down by execution unit.

Table 2.4.: Peak floating-point throughput (TFLOPS) of the A100 by format and execution unit. Tensor core figures in parentheses include sparsity acceleration (2:4 structured sparsity). Data from [RH20].

Format	Scalar (CUDA cores)	Vector (CUDA cores)	Tensor cores
FP64	9.7	9.7	19.5
FP32	19.5	19.5	156 (TF32)
FP16	19.5	78	312 (624)
BF16	19.5	39	312 (624)

The throughput differences are large. Moving from FP64 CUDA core arithmetic (9.7 TFLOPS) to FP32 (19.5 TFLOPS) doubles throughput at no algorithmic cost. Using tensor cores in TF32 mode raises peak throughput to 156 TFLOPS, and FP16/BF16 tensor-core throughput reaches 312 TFLOPS (a 32× factor over FP64). Many kernels remain memory-bandwidth-bound, but switching from 64-bit to 32-bit values still halves memory traffic and directly benefits bandwidth-limited sections.

The Case for Single Precision in Tensor Network Computations

The choice of FP64 in scientific software is often driven by convention rather than necessity [HM22]. Double precision provides roughly 15–16 decimal digits of accuracy, while single precision provides 7–8. Whether the additional digits matter depends on the conditioning of the computation and the accuracy actually required by the application. Higham and Mary [HM22] survey a broad class of numerical linear algebra algorithms where mixed- or reduced-precision arithmetic achieves results of comparable quality to FP64 at substantially lower cost—an observation that applies directly to the tensor contractions considered here.

For tensor network algorithms, several factors favour FP32:

1. **Physical observables are approximate.** Tensor network methods are inherently approximate—the bond dimension truncation introduces controlled errors that typically far exceed single-precision rounding. When the truncation error is $\mathcal{O}(10^{-4})$ to $\mathcal{O}(10^{-6})$, carrying 10^{-16} precision in every floating-point operation is wasteful.

2. **Iterative refinement.** Many tensor network algorithms are iterative (e.g. DMRG sweeps, variational optimisation). Rounding errors from FP32 arithmetic are corrected at each iteration, so they do not accumulate in the same way as in a single long computation.

3. **Memory pressure.** Tensors with large bond dimensions consume substantial memory. Halving the per-element storage from 8 bytes to 4 bytes doubles the tensor sizes that fit in GPU memory, or equivalently allows larger bond dimensions for the same memory budget.

4. **Bandwidth amplification.** Since tensor contractions are frequently memory-bandwidth-bound (Section 2.3.5), the $2\times$ reduction in data movement from FP32 translates almost directly into a $2\times$ speedup for bandwidth-limited kernels, before accounting for any compute throughput gains.

Combined, these factors often yield speedups larger than the raw $2\times$ compute ratio, as shown in Chapter 5. Where needed, mixed-precision strategies—accumulating in FP32 while storing in FP16 or BF16—can push performance further (see [HM22] for a rigorous treatment), while FP32 remains the primary working precision in this work.

2.3.4. DRAM Fundamentals and High Bandwidth Memory

Understanding the physical memory technology that backs GPU global memory is important for reasoning about bandwidth limits, access latency, and power behavior. This subsection summarizes the relevant DRAM fundamentals and then describes the High Bandwidth Memory (HBM) family used in modern GPU accelerators, with emphasis on the HBM2e variant deployed in the A100-SXM4-80GB.

The DRAM Cell

Dynamic Random Access Memory stores each bit as charge on a single capacitor, gated by a single access transistor. This *1T1C* (one-transistor, one-capacitor) design is the dominant commodity DRAM cell and has been since the early 1970s [LRJ18]. In steady state, a charged capacitor represents a logical 1 and a discharged capacitor a logical 0 (or vice versa, depending on convention). The access transistor connects the capacitor to a shared *bitline* (column wire) when the transistor's gate is driven high by the *wordline* (row wire).

Because the capacitor is small (on the order of femtofarads), two consequences follow. First, the stored charge leaks over time, so every cell must be periodically *refreshed*—read and rewritten—to preserve its content; this is the "dynamic" in DRAM. Typical refresh intervals are 32 ms to 64 ms for the entire array [LRJ18]. Second, reading a cell is *destructive*: the act of sharing charge between the cell capacitor and the bitline disturbs the stored voltage, so the sense amplifier must both detect the charge and immediately restore it.

Connection to energy and data-dependent power. Switching a bit between 0 and 1 requires charging or discharging the cell capacitor and toggling the I/O drivers on the data bus. The classical switching-energy model gives a per-transition energy proportional to $\frac{1}{2}CV^2$ (charging/discharging a capacitance C through a voltage swing V). When many bits toggle simultaneously—as happens when writing data with high Hamming distance from the previous content—aggregate switching energy increases. Conversely, data patterns with low bit-toggle rates (e.g. writing a matrix of zeros, or repeated values) cause fewer transitions and consume less I/O power.

This data-dependent power behavior is not merely theoretical. HBM2e devices implement *Data Bus Inversion* (DBI), a per-byte encoding that inverts the data on the bus whenever more than half the bits would toggle, thereby capping the worst-case toggle rate at 50% [Mic21]. The mechanism is a direct hardware response to switching-energy concerns. On the GPU side, the same physics underlies the data-pattern sensitivity explored experimentally in Section 5.7.6: if the memory subsystem throttles power under sustained high-toggle workloads, the effect can propagate into observable performance variation.

Row, Column, and Bank Organization

Internally, DRAM cells are organized in a two-dimensional array of rows and columns, as illustrated schematically in Figure 2 of Li et al. [LRJ18]. A read proceeds in three phases:

1. **Row activation** (*activate*, timing parameter t_{RCD}): the wordline for the target row is raised, connecting all capacitors in that row to their respective bitlines. The sense amplifiers detect and amplify the charge, effectively copying the entire row into the *row buffer* (sense-amplifier latch array). This is the most time-consuming step and also restores the cells that were destructively read.

2. **Column access** (timing parameter t_{CL}): a column address selects a subset of the row buffer and drives it onto the I/O pins. Because the full row is already latched, subsequent reads to different columns in the same row ("row-buffer hits") require only this shorter column-access latency.

3. **Precharge** (timing parameter t_{RP}): before a different row in the same bank can be activated, the bitlines must be precharged to a reference voltage. This closes the current row.

The distinction between a *row-buffer hit* (column access only, cost $\approx t_{\mathrm{CL}}$) and a *row-buffer miss* requiring a full row switch (cost $\approx t_{\mathrm{RP}} + t_{\mathrm{RCD}} + t_{\mathrm{CL}}$) is central to

DRAM performance; the existing discussion in Section 2.3.5 develops the quantitative implications for kernel design.

To increase parallelism, the array is partitioned into *banks* that can be activated independently. Multiple outstanding requests to different banks can overlap, which is how high-bandwidth memory technologies sustain throughput despite individual access latencies in the tens of nanoseconds [LRJ18].

Evolution of DRAM bandwidth. Historically, increasing DRAM bandwidth has been far easier than decreasing latency. The core timing parameters (t_{RCD}, t_{RP}, t_{CL}) are constrained by physical properties of the storage capacitors and bitlines; they have decreased only modestly over two decades. Bandwidth improvements have instead come from widening the data path (the *prefetch buffer* between core and I/O), increasing pin data rates, and adding more channels and banks. This observation—articulated clearly by Li et al. [LRJ18]—explains why the HBM approach (massively wider buses rather than faster pins) is so effective.

Where bandwidth is lost in practice. Even with wide buses and many banks, achieved DRAM bandwidth is always below the theoretical peak. Eyerman et al. [EHH22] introduce *bandwidth stacks* and *latency stacks* as a decomposition methodology: for each memory-controller cycle that does *not* transfer useful data, the lost bandwidth is attributed to a specific cause—row-buffer misses (precharge + activate overhead), DRAM refresh, bus turnaround between reads and writes, or queuing delays from timing constraints. Their analysis shows that row conflicts and refresh overhead are often the dominant bandwidth losses, while write-to-read bus turnaround becomes significant in mixed read/write traffic.

This decomposition connects directly to GPU kernel design. Coalesced warp accesses maximise row-buffer hit rates, reducing the precharge/activate fraction. Tiled algorithms that operate on contiguous blocks of memory reduce the frequency of row switches within a bank. And because HBM refresh is non-negotiable (the 1T1C cell requires it), the refresh overhead sets a hard floor on bandwidth loss that no access pattern can avoid. The practical implication is that an HBM channel delivering 90–95% of its peak bandwidth under a perfectly sequential stream is already close to optimal; the kernel optimiser's job is to keep real workloads as close to that ceiling as possible [EHH22, LRJ18].

High Bandwidth Memory

JEDEC's High Bandwidth Memory standard addresses the bandwidth gap by using 3D integration to package a stack of DRAM dies together with a base logic die [JED13, JED16, JED20]. The stack communicates internally via *through-silicon vias* (TSVs)—vertical conductors etched through the silicon substrate—which enable a very wide parallel bus within the stack (1024 data bits across 8 independent channels of 128 bits each). Jun et al. [JCL+17] describe the core engineering challenges of this stacking approach—TSV reliability, thermal dissipation through the stack, and high-throughput testing of individual dies before assembly—and provide a generational comparison of the HBM family from the manufacturer's perspective.

The entire HBM stack is mounted on a *silicon interposer* alongside the GPU die, forming a *system-in-package* (SiP). This 2.5D packaging approach avoids the signal-routing constraints of a conventional PCB, where trace count is limited to roughly 384 data lines per SoC [Mic21]. Because the interposer traces are much shorter and denser than PCB traces, HBM achieves its bandwidth at comparatively low per-pin data rates (around 1 Gbps for HBM1, 2 Gbps for HBM2, and 3.2 Gbps for HBM2e), which in turn reduces I/O power substantially compared with high-speed serial interfaces like GDDR6 that must drive pins at 16 Gbps or more [LRJ18, Mic21].

Memory dies and channels. The DRAM layers in an HBM stack are called *memory dies* (or *core dies*). Each memory die is a complete DRAM chip in its own right, containing billions of 1T1C storage cells organized into banks, rows, and columns. The dies are manufactured and tested individually before being stacked; Micron ships them as *Known Good Stacked Die* (KGSD) [Mic21].

A *channel* is a fully independent memory interface with its own clock, command/address bus, and 128-bit data path (204 signals in total) [Mic21]. Because each channel has its own controller state, eight outstanding memory transactions—one per channel—can proceed in parallel. Each of the 8 channels is further subdivided into two *pseudo channels* (PCs) that share clock and command inputs but have independent 64-bit data paths, effectively providing 16 semi-independent access points per device [Mic21]. This high degree of internal parallelism is what allows HBM to sustain throughput even when individual DRAM timing parameters (t_{RCD}, t_{RP}, t_{CL}) are comparable to conventional GDDR.

Banks. Within each pseudo channel the DRAM array is partitioned into 16 *banks* (B0–B15) that can be activated independently. A bank is the smallest unit that can hold an open row: each bank has its own row buffer (sense-amplifier latch), so accesses to different banks can overlap without incurring row-switch penalties. In the 4-Hi configuration each channel has 16 banks; in 8-Hi, two dies contribute to the same channel, yielding $2 \times 16 = 32$ banks per channel (distinguished by a *stack ID* bit that selects the upper or lower memory die) [Mic21].

Channel-to-die mapping. Channels are distributed across the memory dies in pairs. In a 4-Hi stack, each of the four core dies carries two channels (e.g. the bottom die carries Channels A and C, the next die Channels B and D, and so on). In an 8-Hi stack the same channel pair is replicated across two dies, doubling the capacity per channel while preserving the same 8-channel topology. Figure 2.5 illustrates this mapping for both configurations: blue layers are memory core dies, the yellow layer is the interface (base) die, and the red layer in the 8-Hi diagram marks a stack position that may be disabled for yield. Figure 2.6 shows the internal layout of a single memory die, where each pseudo channel's 16 banks are visible as individually addressable units.

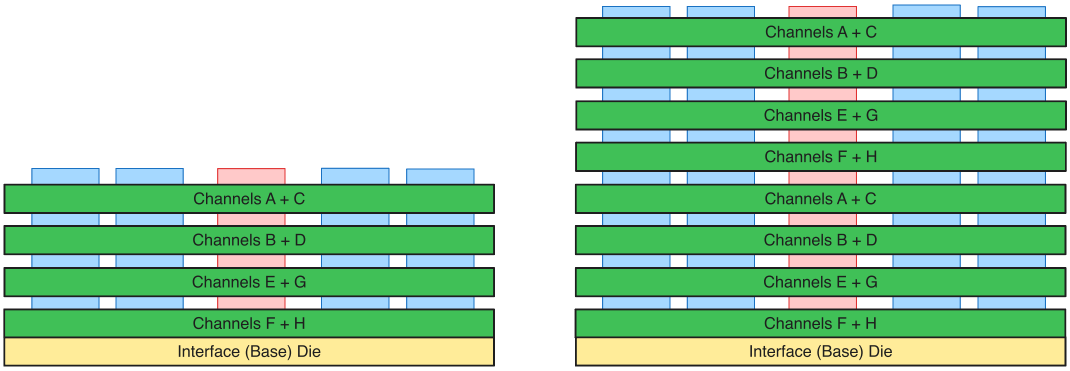


Figure 2.5.: HBM2e stack side view: 4-Hi (left) vs. 8-Hi (right). Channel pairs are labelled per die. After [Mic21], Fig. 4.

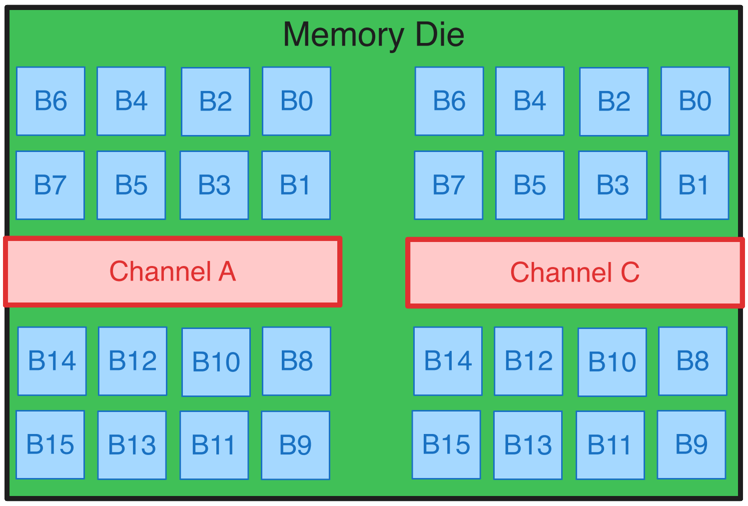


Figure 2.6.: Top-down view of a single HBM2e memory die with two pseudo channels (A, C) and their 16 banks each (B0–B15). After [Mic21], Fig. 4.

HBM generation roadmap. Table 2.5 summarises the key parameters across HBM generations relevant to datacenter GPUs. Through HBM2e the 1024-bit bus and 8-channel topology are preserved; bandwidth scaling comes almost entirely from increasing the per-pin data rate, while capacity scaling comes from stacking more dies (4-Hi → 8-Hi → 12-Hi) and from denser per-die DRAM arrays. HBM3/3e doubles the channel count to 16 (each now 64-bit) while retaining the 1024-bit aggregate width [JCL+17, JED20].

Table 2.5.: HBM generation comparison for datacenter-relevant configurations. Data compiled from JEDEC standards [JED13, JED16, JED20] and Jun et al. [JCL+17].

	HBM	HBM2	HBM2e	HBM3	HBM3e
JEDEC adoption	2013	2016	2018	2022	2023
Pin data rate	1 Gbps	2 Gbps	3.2 Gbps	6.4 Gbps	9.6 Gbps
Bus width (bits)	1024	1024	1024	1024	1024
Channels per stack	8	8	8	16	16
Max stack height	4-Hi	8-Hi	8-Hi	8/12-Hi	12-Hi
Max capacity / stack	4 GB	8 GB	16 GB	24 GB	36 GB
BW per stack	128 GB/s	256 GB/s	410 GB/s	819 GB/s	1229 GB/s

HBM2e vs. GDDR6(X) and the A100 Memory Subsystem

HBM2e is often positioned as an alternative to the well-established GDDR6 and GDDR6X SGRAM technologies, which also provide high memory bandwidth but through a fundamentally different trade-off [Mic21]. Where HBM2e achieves bandwidth through a 32× wider I/O bus at modest pin speeds (3.2 Gbps), GDDR6(X) compensates for its narrow 32-bit bus by driving pins at 16 —-24 Gbps, which demands power-hungry techniques such as on-die signal termination, delay-locked loops, and clock-data recovery [Mic21]. Table 2.6 contrasts the key parameters.

The V100 (Volta, 2017) was the first NVIDIA GPU accelerator to deploy HBM2 at scale, establishing the pattern of in-package high-bandwidth memory that the A100 continues.

Table 2.6.: HBM2e DRAM vs. GDDR6(X) SGRAM. Data from [Mic21].

Parameter	HBM2e	GDDR6 / GDDR6X
Capacity per device	8–16 GB	1–2 GB
Channels per device	8	2
Devices per system	4–6	up to 24
Max pin data rate	3.2 Gbps	16 / 24 Gbps
I/O width (bits)	1024	32
BW per device	410 GB/s	64 GB/s
Total system BW	up to 2450 GB/s	up to 768 / 1152 GB/s
Package	KGSD (stacked die)	BGA-180

Several observations from this comparison connect directly to the performance modeling in later chapters:

1. **Latency is comparable.** The core DRAM timing parameters (t_{RCD}, t_{RP}, t_{CL}) are in the same ∼14 ns range for both HBM2e and GDDR6. High bandwidth does not imply low latency; latency hiding requires sufficient parallelism (banks, channels, and outstanding requests) [LRJ18].

2. **HBM2 to HBM2e is a pin-speed upgrade.** The physical interface (interposer, TSV pitch, channel count) is unchanged between HBM2 and HBM2e. The

bandwidth increase from 256 GB/s to 410 GB/s per stack comes from raising the per-pin data rate from 2 Gbps to 3.2 Gbps [Mic21]. HBM2e also doubles the maximum stack capacity from 8 GB to 16 GB (eight-high configuration with denser dies), which is why the A100-SXM4-80GB can offer 80 GB from five stacks.

3. **GPU deployment history.** The V100 (Volta, 2017) used four stacks of HBM2 providing 900 GB/s aggregate bandwidth and 16 GB or 32 GB capacity. Both A100 variants use five active HBM stacks out of six physical stacks on the package (one disabled for yield), giving $5 \times 8 = 40$ independent channels in both cases. The A100-SXM4-40GB uses HBM2 in a *4-Hi* (four-high) stack configuration: 1 GB per die $\times$ 4 dies = 8 GB per stack, for 40 GB total at 1555 GB/s. The A100-SXM4-80GB upgrades to HBM2e in an *8-Hi* (eight-high) configuration: 2 GB per die $\times$ 8 dies = 16 GB per stack, for 80 GB total at 2039 GB/s [NVI20a, Mic21]. The stacks are physically twice as tall but share the same channel topology; the bandwidth increase comes from the higher per-pin data rate of HBM2e (3.2 Gbps vs. ~2.4 Gbps for the HBM2 configuration used in A100-40GB). The quantitative impact of this bandwidth difference on roofline modeling is developed in Section 2.3.6.

4. **Hopper generation.** The H100-SXM5-80GB (2022) moved to HBM3 with six stacks of 13.3 GB each, providing 80 GB at 3350 GB/s [NVI22]. The H200 (2024) shares the same GH100 die but upgrades to HBM3e: six stacks totalling 141 GB at 4800 GB/s. This pattern—same compute die, upgraded memory—mirrors the A100-40GB→80GB transition and illustrates that memory technology often advances faster than a GPU generation cycle.

2.3.5. Memory Hierarchy

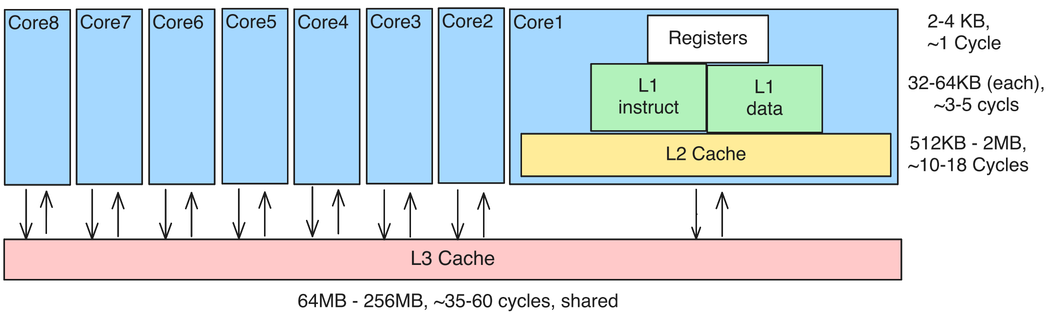


Figure 2.7.: Memory hierarchy of a modern CPU, showing the register file and cache levels (L1, L2, L3) before main memory (DRAM).

As shown in Figure 2.7, modern CPUs rely on a deep cache hierarchy to reduce effective memory latency.

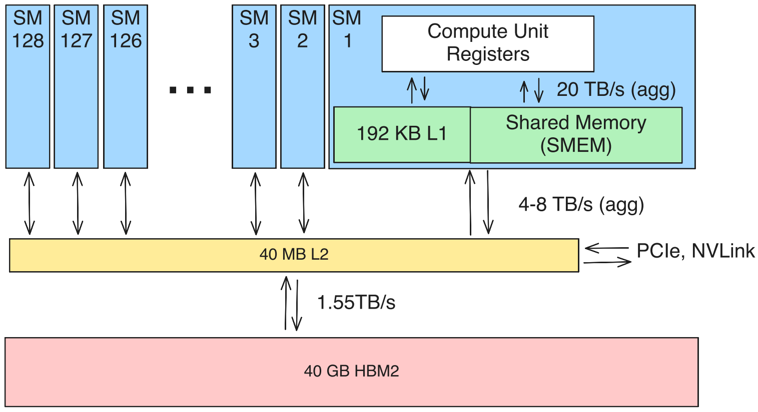


Figure 2.8.: Memory hierarchy of an A100 GPU, showing registers, shared memory/L1, L2 cache, and HBM global memory.

GPU Memory Hierarchy Where CPUs use deep cache hierarchies to hide latency, GPUs prioritise bandwidth and expose a shallower hierarchy tuned for throughput. Four levels are relevant:

- **Registers** reside in the SM and offer single-cycle access. They are private to each thread and are the scarcest on-chip resource: the number of registers a kernel uses directly limits occupancy.
- **Shared memory** is also on-chip but visible to all threads in a block, making it the primary mechanism for explicit data reuse and inter-thread communication. Most high-performance kernels—tiled matrix multiplications, tensor contractions—stage data through shared memory to avoid repeated global loads.
- **L2 cache** is shared across all SMs and transparently caches global memory traffic. On the A100 it is 40 MB, modest relative to the core count, which reflects the throughput-oriented design.
- **HBM (global memory)** provides the bulk storage. The A100-SXM4-40GB provides 40 GB HBM with a peak bandwidth of approximately 1555 GB/s (Table 2.9). Access latency, however, is roughly 500× that of a register access (Table 2.15).

On Ampere-class GPUs, memory behavior is often the dominant constraint before arithmetic throughput saturates. For HBM-backed global memory, row-buffer locality and coalesced warp accesses strongly influence achieved bandwidth; strided or irregular patterns can reduce effective throughput substantially. For this reason, the optimisation strategy prioritises three levers: *(i)* layout and access regularity (Sections 2.3.5 and 2.4.3), *(ii)* occupancy-aware block sizing, typically starting from 128 to 256 threads per block and then tuning against resource limits (Section 2.4.1), and *(iii)* profiler-guided iteration with Nsight Compute and Nsight Systems (Sections 2.4.4 and 5.5).

HBM Row/Column Access Path

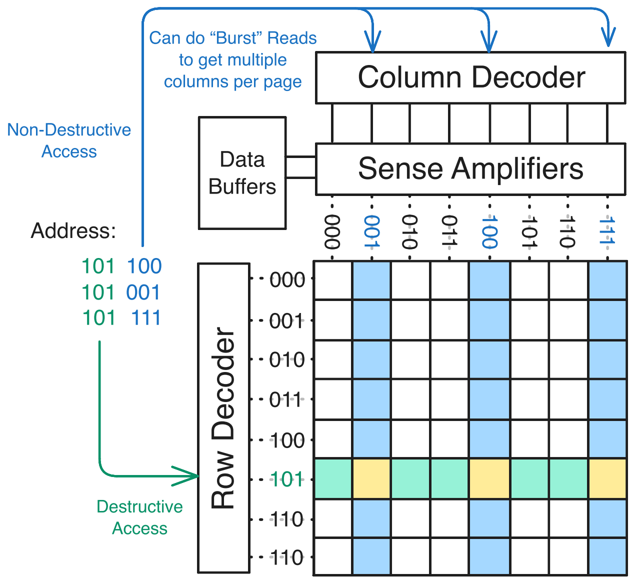


Figure 2.9.: Schematic for HBM/DRAM row-column access. A read activates one row, latches it into the sense amplifiers, and then uses the column decoder to select transferred data (including burst reads from the open row buffer).

At a conceptual level, the read path is: *row activate* → *sense-amplifier latch/restore* → *column select* → *I/O transfer*. The row-activation step moves a full row into the sense amplifiers (often called the row buffer or "page" in architecture talks). DRAM cell charge is disturbed by this operation and then restored through the sense amplifiers while the row stays open. The *burst size* is the fixed data amount returned by one column-read transfer once the row is open; in the HBM2-style model used here, this is 64 B per burst. Practically, this means several nearby values can be served from the same open row with burst transfers before a row switch is needed.

Using standard DRAM timing notation, a row-buffer hit read is approximated by the column-read latency t_{CL}. A row switch (page switch) requires precharge of the current row (t_{RP}), activation of the new row (t_{RCD}), and then the column read:

$$T_{\mathrm{hit}} = t_{\mathrm{CL}}, \tag{2.2}$$
$$T_{\mathrm{switch}} = t_{\mathrm{RP}} + t_{\mathrm{RCD}} + t_{\mathrm{CL}}. \tag{2.3}$$

Hence, the switch penalty relative to a hit is $T_{\mathrm{switch}}/T_{\mathrm{hit}}$. Representative HBM/HBM2 timing values reported in the memory-systems literature are near-symmetric ($t_{\mathrm{RP}} \approx t_{\mathrm{RCD}} \approx t_{\mathrm{CL}} \approx 14\,\mathrm{ns}$), which gives a $\approx 3\times$ row-switch cost (48 cycles for a full switch versus 16 for a column read) [LRJ18, Jon22].

For kernel design, this matters because row-buffer hits are cheaper than repeated row switches in the same bank. Coalesced accesses and shape-bucketed launches increase locality; highly strided or scattered accesses raise the rate of row misses/conflicts and reduce effective bandwidth. The coalescing trends in Table 5.5 and Figure 5.2 are consistent with this hardware-level mechanism.

Finally, the practical block-size implication is a heuristic, not a hard rule: for bandwidth-sensitive kernels on A100, 128 threads per block is a good starting point (four warps provide enough independent requests for latency hiding), but it is not mandatory. The optimal block size still depends on register pressure, shared-memory footprint, and scheduler efficiency.

2.3.6. NVIDIA A100 Ampere Architecture

Hardware Overview

Table 2.7.: A100 KPI card (SXM4-40GB baseline): per processing core (SM sub-partition).

Property	Value
Warp schedulers	1
Warp dispatch units	1
FP32 CUDA cores	16
FP64 CUDA cores	8
Tensor cores	1
Maximum warps (derived)	16
Maximum resident threads (derived)	512

Table 2.8.: A100 KPI card (SXM4-40GB baseline): per SM.

Property	Value
Processing cores (sub-partitions)	4
FP32 CUDA cores	64
FP64 CUDA cores	32
Tensor cores	4
Warp schedulers	4
Warp dispatch units	4
Maximum resident threads	2048
Maximum resident warps	64
Maximum resident blocks	32
Register file (32-bit registers)	65 536
Shared memory (KB)	164

Table 2.9.: A100 KPI card (SXM4-40GB baseline): per GPU.

Property	Value
Streaming multiprocessors (SMs)	108
Total processing cores (sub-partitions)	432
Maximum resident warps	6912
Maximum resident threads	221 184
L2 cache (MB)	40
HBM capacity (GB)	40
Peak HBM bandwidth (GB/s)	1555
Base clock frequency (GHz)	1.41

The A100 employs high-bandwidth memory as its main device memory, providing the throughput required to sustain dense linear algebra kernels.

Yield context. The detailed first-order yield sensitivity model and area-scaling calculation for GA100 are provided in Section D.2 to keep this chapter focused on kernel-relevant execution concepts.

GA100 Die vs A100 Module: What Is Around the Chip

It is useful to distinguish the *GA100* silicon die from the complete *A100 accelerator module*. GA100 is the GPU chip itself (compute cores, caches, memory controllers, and I/O logic). The A100 is the packaged and integrated product used in systems (SXM4 or PCIe form factor), which adds memory stacks, package/interposer, power delivery, and board/module interfaces.

For architectural reasoning, the main package-level components around GA100 are:

- **HBM stacks on the same package:** A100 places high-bandwidth memory adjacent to the GPU die in-package, enabling a very wide memory interface with short electrical paths.
- **Memory controllers and links:** the A100 implementation exposes five active HBM stacks and ten 512-bit memory controllers (effective 5120-bit memory interface) [NVI20b, NVI20a].
- **Interposer/package fabric:** GPU die and HBM stacks are integrated in one package so bandwidth is far higher than off-package GDDR designs at comparable power.
- **Module-level interfaces:** outside the package, the accelerator provides PCIe host connectivity and NVLink links for GPU-GPU communication, depending on form factor and server integration [NVI25c].

This distinction matters for performance interpretation: for memory-bound kernels, the limiting path is often the GA100↔HBM package subsystem; for multi-GPU scaling, the dominant path can shift to NVLink; and for host-staged workloads, PCIe plus CPU-NUMA placement becomes critical.

A100 40GB vs 80GB: HBM2 vs HBM2e

For the benchmark setup used here, the key architectural difference between A100-SXM4-40GB and A100-SXM4-80GB is the memory subsystem, not the SM compute front end. The 40GB card uses HBM2, while the 80GB card uses HBM2e with higher aggregate bandwidth and doubled capacity. A concise comparison is given in Table 2.10.

Table 2.10.: A100 SXM4 variant comparison relevant to performance modeling. Data compiled from NVIDIA's A100 architecture whitepaper and product specifications [NVI20a, NVI25c].

Property	A100-SXM4-40GB	A100-SXM4-80GB
Memory technology	HBM2	HBM2e
HBM capacity	40 GB	80 GB
Peak HBM bandwidth	1555 GB/s	2039 GB/s
FP32 peak throughput	19.5 TFLOPS	19.5 TFLOPS
L2 cache size	40 MB	40 MB
NVLink aggregate bandwidth per GPU (bidirectional)	600 GB/s	600 GB/s

Two immediate modeling consequences follow:

1. **Higher memory roofline.** For memory-bound kernels, the best-case throughput scaling from 40GB to 80GB is approximately the bandwidth ratio $2039/1555 \approx 1.31$ (about 31%).

2. **Lower ridge intensity.** With unchanged FP32 peak but higher bandwidth, the FP32 roofline ridge point shifts from

$$I^{\star}_{40} = \frac{19.5}{1555} \approx 12.5 \text{ FLOPs/byte}$$

to

$$I^{\star}_{80} = \frac{19.5}{2039} \approx 9.6 \text{ FLOPs/byte.}$$

This means more kernels fall into the compute-bound region on A100-80GB compared with A100-40GB at the same arithmetic intensity.

A100-80GB mainly increases headroom for memory-bound and capacity-limited workloads. Compute-pipeline ceilings stay almost unchanged, so compute-bound kernels should not see comparable speedups.

Theoretical Peak Performance

TPP definition. TPP denotes theoretical peak performance. For CUDA-core arithmetic on A100, using fused multiply-add (2 FLOPs/cycle):

$$P^{\text{TPP}}_{\text{FP32}} = N_{\text{SM}} \cdot N_{\text{FP32/SM}} \cdot 2f = 108 \cdot 64 \cdot 2 \cdot 1.41 \text{ GHz} \approx 19.5 \text{ TFLOP/s},$$

$$P^{\text{TPP}}_{\text{FP64}} = N_{\text{SM}} \cdot N_{\text{FP64/SM}} \cdot 2f = 108 \cdot 32 \cdot 2 \cdot 1.41 \text{ GHz} \approx 9.7 \text{ TFLOP/s.}$$

Using the sustained peak HBM bandwidth $B_{\max} = 1555\,\text{GB/s}$, a bandwidth-only streaming bound for one FLOP per fetched value is

$$P_{\text{BW,FP64}}^{(1\text{ FLOP/value})} = \frac{1555}{8} \approx 194.4\,\text{GFLOP/s}, \qquad P_{\text{BW,FP32}}^{(1\text{ FLOP/value})} = \frac{1555}{4} \approx 388.8\,\text{GFLOP/s}.$$

Table 2.11.: A100 throughput ceilings for the performance model.

Metric	Expression	Value
FP64 TPP (CUDA cores)	$108 \cdot 32 \cdot 2 \cdot 1.41$ GHz	9.7 TFLOP/s
FP64 TPP (tensor cores)	NVIDIA peak specification	19.5 TFLOP/s
FP32 TPP (CUDA cores)	$108 \cdot 64 \cdot 2 \cdot 1.41$ GHz	19.5 TFLOP/s
TF32 TPP (tensor cores)	NVIDIA peak specification	156.0 TFLOP/s
FP16 TPP (tensor cores)	NVIDIA peak specification	312.0 TFLOP/s
Bandwidth-based FP64 ceiling (1 FLOP/value)	1555/8	194.4 GFLOP/s
Bandwidth-based FP32 ceiling (1 FLOP/value)	1555/4	388.8 GFLOP/s

Derived Performance Limits

Table 2.12.: Derived A100 limits and capacity counts for performance analysis: per processing core (SM sub-partition).

Metric	Value
Peak FP32 performance (GFLOP/s)	45.12
Peak FP64 performance (GFLOP/s)	22.56
Maximum resident warps (derived)	16
Maximum resident threads (derived)	512

Table 2.13.: Derived A100 limits and capacity counts for performance analysis: per SM.

Metric	Value
Peak FP32 performance (GFLOP/s)	180.48
Peak FP64 performance (GFLOP/s)	90.24
Tensor-core FP16 performance (TFLOP/s)	2.89
Memory bandwidth share (GB/s)	14.40
Arithmetic-intensity threshold FP32 (FLOPs/byte)	12.5
Arithmetic-intensity threshold FP64 (FLOPs/byte)	6.2

Table 2.14.: Derived A100 limits and capacity counts for performance analysis: per GPU.

Metric	Value
Bandwidth-based FP64 ceiling (1 FLOP/value) (GFLOP/s)	194.4
Bandwidth-based FP32 ceiling (1 FLOP/value) (GFLOP/s)	388.8
Total FP32 CUDA cores	6912
Total FP64 CUDA cores	3456
Total processing cores (sub-partitions)	432
Maximum resident warps	6912
Maximum resident threads	221 184

Memory Latency

Table 2.15.: Approximate memory access latency at different hierarchy levels.

Memory level	Latency (cycles)
Registers	1
Shared memory	20
L2 cache	200
HBM global memory	500

2.3.7. Compute Node Topology

The performance of GPU-accelerated applications depends on more than the GPU itself. Node topology matters: CPU placement, GPU placement, and memory connectivity. This subsection describes the JURECA-DC node architecture, which is representative of modern multi-GPU HPC nodes.

CPU and NUMA Topology

Each compute node contains two AMD EPYC 7742 processors, each with 64 physical cores [Jue26]. The EPYC 7002 generation is based on Zen 2 and follows a chiplet organization with core chiplets and a central I/O die [Adv20]. Since the terms are easy to mix up, we fix the naming used here:

- **CCD** (*Core Complex Die*): a compute chiplet containing CPU cores.
- **CCX** (*Core Complex*): a core cluster inside a CCD. On EPYC 7742, each CCD has two CCXes.
- **cIOD** (*central I/O die*): the central die that provides memory controllers, PCIe root complexes, and fabric routing.

On this EPYC 7002 generation, each CCX contains four cores with a private 16 MB L3 cache [Adv20]. The hierarchy is shown in Figure 2.10.

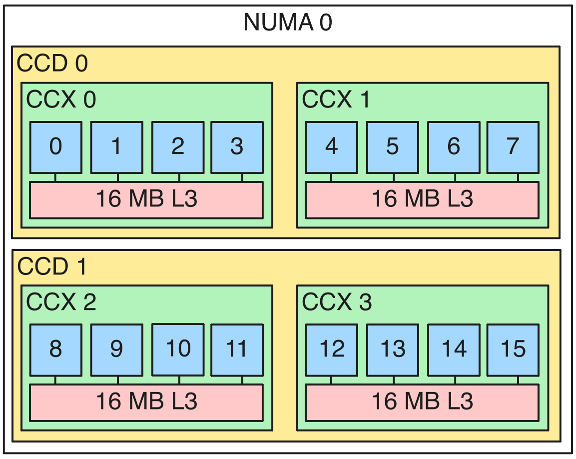


Figure 2.10.: NUMA and chiplet hierarchy of a single AMD EPYC 7742 socket. Each socket contains 8 CCDs grouped into 4 NUMA domains (NPS=4 configuration). Each CCD holds two CCXes, each with 4 cores and a private 16 MB L3 cache. All CCDs connect to a central I/O die that houses the memory controllers and PCIe root complexes.

The BIOS is configured with NPS=4, which exposes four NUMA domains per socket (eight in total across the node). Each NUMA domain encompasses two CCDs (16 cores, 32 threads with SMT) and a quarter of the socket's memory controllers. This configuration allows the operating system and runtime to make NUMA-aware allocation decisions.

Infinity Fabric and latency classes. AMD's Infinity Fabric connects CCX/CCD resources to the cIOD and also connects the two sockets. The practical latency hierarchy for software placement decisions is:

1. **Intra-CCX:** cores sharing the same 16 MB L3 cache communicate through it at low latency.
2. **Cross-CCX / cross-CCD, same socket:** communication no longer benefits from a shared L3 and is routed via Infinity Fabric through cIOD resources, with higher latency.
3. **Cross-socket:** traffic traverses inter-socket Infinity Fabric links (xGMI), which is the highest-latency path.

Why this matters for GPU work. The GPU is not attached directly to a random core; it is attached to a PCIe root complex on the cIOD, and each GPU therefore has a *local* NUMA region on the host side. A host-to-device transfer is fastest when both the launching thread and the source host memory are local to the GPU-affine NUMA domain.

If either thread placement or host-memory placement is remote, the transfer must take additional Infinity Fabric hops before reaching PCIe, which increases latency and can reduce effective bandwidth.

GPU Interconnect Topology

Each node has four NVIDIA A100-SXM4-40GB GPUs arranged in a fully connected mesh via NVLink 3.0. Every GPU pair is connected by four NVLink bridges (denoted NV4 in NVIDIA's topology notation), providing 100 GB/s of uni-directional bandwidth between any two devices. This symmetric, all-to-all connectivity means that multi-GPU tensor contractions do not suffer from topology-dependent bandwidth asymmetries.

PCIe and NVLink roles. Although both are high-speed interconnects, they serve different roles:

- **PCIe Gen4 $\times$16** is the primary host-device path. On A100 nodes, it carries control traffic and host staging transfers, with a nominal bidirectional bandwidth of about 64 GB/s.

- **NVLink 3.0** is the primary GPU-GPU path. It is used for peer-to-peer exchange and collective communication, with up to 600 GB/s bidirectional bandwidth per A100 SXM4 GPU.

Use PCIe-local NUMA placement to minimise host-device staging costs, and prefer NVLink for inter-GPU tensor redistribution whenever possible [NVI25c, NVI24b].

Each GPU is physically attached via PCIe Gen 4 $\times$16 to a specific NUMA domain on one of the two CPU sockets:

Table 2.16.: GPU-to-NUMA affinity on a JURECA-DC compute node.

GPU	Socket	NUMA domain	CPU cores (physical)
GPU 0	0	3	48–63
GPU 1	0	1	16–31
GPU 2	1	7	112–127
GPU 3	1	5	80–95

The full topology, including the three classes of CPU–GPU data paths, is illustrated in Figure 2.11.

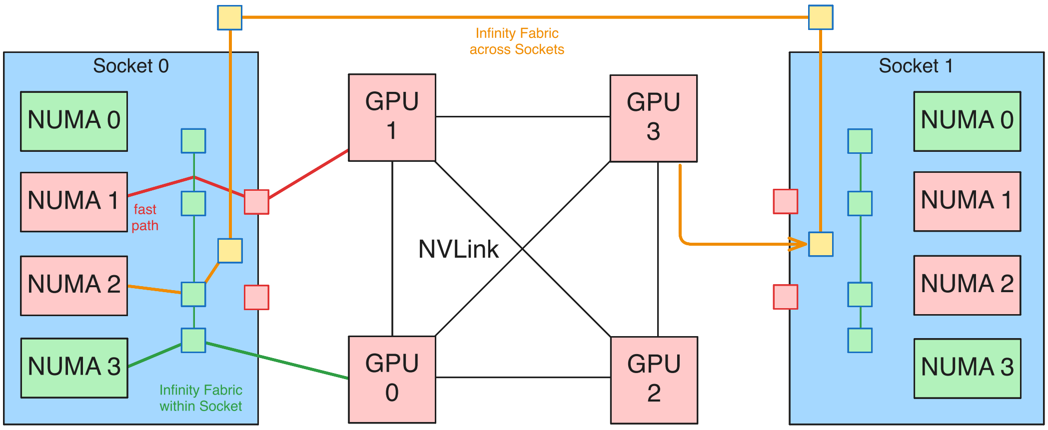


Figure 2.11.: Compute node topology showing the four A100 GPUs (centre) connected via NVLink 3.0, with the two CPU sockets on either side. Three representative host–device paths are highlighted: a fast local PCIe path (GPU 1 ↔ NUMA 1), an intra-socket Infinity Fabric path (GPU 0 ↔ NUMA 3), and a cross-socket Infinity Fabric path (GPU 3 ↔ NUMA 2).

The distinction between these paths matters for host–device data transfers. A `cudaMemcpy` initiated from a CPU thread pinned to a GPU's local NUMA domain takes the shortest PCIe path through the I/O die. If the source thread or memory allocation resides on the wrong NUMA domain—or worse, the wrong socket—the transfer must additionally traverse Infinity Fabric, increasing latency. For GPU-to-GPU communication, NVLink bypasses the CPU subsystem entirely, making the CPU topology irrelevant for peer-to-peer transfers.

Fast-path policy. For runs that include host-device staging, the following policy is applied:

1. pin the orchestration thread to cores in the GPU-affine NUMA domain,
2. allocate pinned host buffers on the same NUMA domain,
3. execute `cudaMemcpyAsync` from that thread/context to preserve the local PCIe path.

This is a standard NUMA-locality rule for PCIe-attached accelerators and is independent of tensor-network physics details. More tightly integrated CPU–GPU systems (e.g. Grace-Hopper style designs) use different host-device topology assumptions and are therefore outside the scope of this node-level model.

Host-device path cost model. The full parametric latency+bandwidth model, per-GPU path-class mapping, and closed-form penalty expressions are given in Chapter D.

In practice, this topology has two implications for the workload:

- **Single-GPU kernels:** the CPU topology is largely invisible, since kernel launch overhead is small and tensor data resides in GPU memory throughout the computation.
- **Multi-GPU contractions:** data distribution and inter-GPU communication use NVLink exclusively. Host-side orchestration threads are pinned to the appropriate NUMA domain to minimise any residual host–device transfer overhead.

2.4. CUDA Programming Model

CUDA (Compute Unified Device Architecture) is NVIDIA's parallel programming platform for general-purpose computation on GPUs. A CUDA program combines host code (CPU side) with *kernels* launched on the GPU. Parallelism is specified through a grid of thread blocks at launch time, and the hardware schedules those blocks onto the available SMs.

2.4.1. Thread Hierarchy and Kernel Launch

CUDA organises parallel execution into a three-level hierarchy: *grids*, *thread blocks* (or simply *blocks*), and *threads*. A kernel launch creates one grid, which is partitioned into blocks. Each block contains a fixed number of threads that execute concurrently on the same SM and can cooperate through shared memory and synchronisation barriers. Threads in different blocks cannot synchronise with each other during kernel execution.

Both the grid and each block can be specified with up to three dimensions, which provides a natural mapping for problems defined over multidimensional arrays. The dimensions are set via the `dim3` type, and each thread identifies its position using the built-in variables `threadIdx`, `blockIdx`, `blockDim`, and `gridDim`.

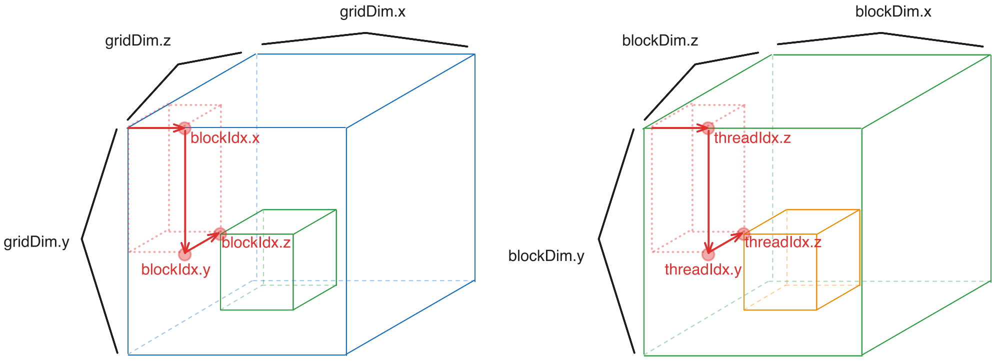


Figure 2.12.: CUDA execution hierarchy showing the mapping of grids to thread blocks and threads. Threads are organised in up to three dimensions and identified using the built-in variables `threadIdx` and `blockIdx`.

The hierarchy in Figure 2.12 also includes *warps*: groups of 32 threads that execute in lock-step on the SM's SIMT (Single Instruction, Multiple Thread) units.

Kernel Syntax and Launch Configuration

A kernel is declared with the `__global__` qualifier and launched using the triple-chevron syntax `<<<gridDim, blockDim>>>`. The following example illustrates a minimal kernel that adds two vectors element-wise:

```
__global__ void vecAdd(const float *a, const float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

// Host launch
```

```
9  int n = 1 << 20;                    // 1,048,576 elements
10 int threadsPerBlock = 256;
11 int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
12 vecAdd<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);
```

Listing 2.1: Element-wise vector addition kernel and its host-side launch.

The global thread index is computed on line 2 as `blockIdx.x * blockDim.x + threadIdx.x`. This is the standard one-dimensional addressing pattern: `blockIdx.x` selects which block the thread belongs to, `blockDim.x` gives the number of threads per block, and `threadIdx.x` is the thread's position within its block. The bounds check on line 3 is necessary because the total number of threads launched (blocks × threads per block) is rounded up to a multiple of the block size and may therefore exceed `n`.

Two-Dimensional Grid Addressing

For problems with a natural two-dimensional structure, such as matrix operations or image processing, both the grid and block dimensions can be specified in two (or three) dimensions. The following example demonstrates a kernel that transposes an $M \times N$ matrix:

```
__global__ void transpose(const float *in, float *out, int M, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < M && col < N) {
        out[col * M + row] = in[row * N + col];
    }
}

// Host launch
dim3 blockDim(16, 16);          // 256 threads per block
dim3 gridDim(
    (N + blockDim.x - 1) / blockDim.x,
    (M + blockDim.y - 1) / blockDim.y
);
transpose<<<gridDim, blockDim>>>(d_in, d_out, M, N);
```

Listing 2.2: Matrix transpose using two-dimensional grid and block addressing.

Here each thread is addressed by a `(row, col)` pair derived from the two-dimensional block and thread indices. The grid is sized so that at least $M \times N$ threads are launched, again with bounds checking to handle dimensions that are not exact multiples of the block size.

Block Size Selection and Hardware Constraints

The choice of block size affects both correctness and performance. The CUDA programming model imposes an upper limit of 1024 threads per block. Beyond this hard limit, several performance-relevant factors guide the choice:

- **Warp granularity.** Because threads are scheduled in warps of 32, the block size should be a multiple of 32 to avoid partially filled warps whose unused lanes still consume scheduling resources.

- **Occupancy.** Each SM has a finite number of registers, a fixed amount of shared memory, and a maximum number of resident threads (2048 on the A100). If a kernel uses many registers per thread, fewer threads can reside on the SM simultaneously, which may leave the hardware underutilised. The CUDA occupancy calculator relates block size and per-thread resource usage to the fraction of the SM's capacity that is occupied.
- **Shared memory per block.** Shared memory is partitioned among the blocks resident on an SM. A block that allocates a large amount of shared memory limits the number of blocks that can coexist, potentially reducing occupancy.

In practice, block sizes of 128 or 256 threads are common defaults that balance these constraints. More performance-critical kernels tune the block size empirically or use the `__launch_bounds__` qualifier to give the compiler additional information for register allocation.

Linearisation of Multidimensional Indices

GPU memory is addressed linearly, so multidimensional arrays must be mapped to one-dimensional offsets. For a tensor stored in row-major order, the linear index of element (i, j) in an $M \times N$ matrix is

$$\text{offset} = i \cdot N + j, \tag{2.4}$$

which generalises to higher-order tensors by successive multiplication with trailing dimensions. For a 3D tensor with logical indices (x, y, z) and extents (s_x, s_y, s_z), a common row-major linearisation is

$$\text{offset}(x, y, z) = x + s_x\, y + s_x s_y\, z. \tag{2.5}$$

Equivalently, using row-major index names (i, j, k) for a shape (D_0, D_1, D_2):

$$\text{offset}(i, j, k) = ((iD_1) + j)D_2 + k.$$

In compact form for rank-d tensors with extents $(D_0, \dots, D_{d-1})$,

$$\text{offset}(i_0, \dots, i_{d-1}) = \sum_{m=0}^{d-1} i_m \prod_{n=m+1}^{d-1} D_n.$$

In column-major (Fortran) order, the convention used by cuBLAS and most BLAS libraries, the 2D index is instead

$$\text{offset} = j \cdot M + i. \tag{2.6}$$

For the same 3D extents (s_x, s_y, s_z) and indices (x, y, z), the column-major form is

$$\text{offset}(x, y, z) = z + s_z\, y + s_z s_y\, x.$$

Listing 2.2 uses row-major layout for the input (`in[row * N + col]`) and column-major layout for the output (`out[col * M + row]`), which is precisely the transpose operation.

A concrete example: for a 4×3 matrix stored in row-major order, the element at row 2, column 1 maps to linear offset $2 \cdot 3 + 1 = 7$. Understanding this mapping is essential for implementing coalesced memory access patterns, discussed in Section 2.4.3.

2.4.2. Shared Memory and Synchronisation

Shared memory is an on-chip, programmer-managed memory space visible to all threads within a block. It serves two purposes: as a software-managed cache to stage data from global memory, and as a communication channel between threads in the same block. Because shared memory has much lower latency than global memory (approximately 20 cycles versus 500 cycles on the A100, cf. Table 2.15), its effective use is often the difference between a bandwidth-bound kernel and one that approaches peak compute throughput.

Static and Dynamic Allocation

Shared memory can be allocated statically at compile time or dynamically at launch time. Static allocation uses the `__shared__` qualifier with a fixed array size:

```
__shared__ float tile[TILE_SIZE][TILE_SIZE];
```

Listing 2.3: Static shared memory allocation for a tile.

Dynamic allocation is specified as a third argument in the launch configuration and accessed through an `extern __shared__` declaration:

```
extern __shared__ float smem[];

// Host launch with dynamic shared memory size in bytes
kernel<<<grid, block, sharedBytes>>>(args);
```

Listing 2.4: Dynamic shared memory allocation.

Dynamic allocation is useful when the tile size depends on runtime parameters, which is common in autotuned kernels.

Tiled Matrix Multiplication

The canonical use of shared memory is tiled (or blocked) matrix multiplication. A naïve matrix multiplication kernel that computes $C = AB$ with $A \in \mathbb{R}^{M\times K}$ and $B \in \mathbb{R}^{K\times N}$ has each thread compute one element of C by reading an entire row of A and column of B from global memory. This results in $\mathcal{O}(K)$ global loads per thread and an arithmetic intensity of roughly 2 FLOPs/byte—well below the A100's crossover point of ≈ 9.6 FLOPs/byte for FP32 (Table 2.13).

Tiling reduces global memory traffic by loading the input matrices into shared memory in small blocks (tiles), computing partial dot products from the tile, and then advancing to the next tile. Listing 2.5 shows the structure of this approach.

```
#define TILE 16

__global__ void matmul(const float *A, const float *B, float *C,
                       int M, int N, int K) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < (K + TILE - 1) / TILE; t++) {
```

```
13          // Cooperative load: each thread loads one element of each tile
14          int aCol = t * TILE + threadIdx.x;
15          int bRow = t * TILE + threadIdx.y;
16          As[threadIdx.y][threadIdx.x] = (row < M && aCol < K)
17                                        ? A[row * K + aCol] : 0.0f;
18          Bs[threadIdx.y][threadIdx.x] = (bRow < K && col < N)
19                                        ? B[bRow * N + col] : 0.0f;
20
21          __syncthreads();  // ensure the tile is fully loaded
22
23          for (int k = 0; k < TILE; k++) {
24              sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
25          }
26
27          __syncthreads();  // safe to overwrite tile in next iteration
28      }
29
30      if (row < M && col < N) {
31          C[row * N + col] = sum;
32      }
33  }
```

Listing 2.5: Tiled matrix multiplication using shared memory. Each block computes one tile of the output matrix C.

The kernel structure has three important properties:

1. **Cooperative loading (lines 14–19).** Every thread in the block loads exactly one element of the A-tile and one element of the B-tile. The 256 threads in a 16×16 block therefore load the entire 16×16 tile cooperatively. Boundary conditions are handled by loading zero for out-of-range indices.

2. **Barrier synchronisation (line 21).** The `__syncthreads()` call ensures that all threads have finished writing to shared memory before any thread begins reading from the tile. A second barrier after the computation (line 27) prevents any thread from overwriting the tile before all threads have finished using it.

3. **Data reuse.** Each element loaded into `As` is read by all 16 threads in its row; each element of `Bs` is read by all 16 threads in its column. This $16\times$ reuse from shared memory instead of global memory raises the effective arithmetic intensity by the tile size factor.

For a tile size of T, the number of global memory loads per output element drops from $2K$ (naïve) to $2K/T$, and the arithmetic intensity increases proportionally by T. Larger tiles improve reuse but require more shared memory per block, which can limit occupancy.

2.4.3. Memory Coalescing and Bank Conflicts

Global Memory Coalescing

When a warp executes a load or store instruction, the hardware combines the 32 individual thread addresses into as few memory transactions as possible. If consecutive threads access consecutive memory addresses (i.e. thread i accesses address base $+ i$), the accesses are *coalesced* into a minimal number of 128-byte cache line transactions. Non-coalesced access patterns—strided or scattered—require multiple transactions for the same warp, wasting bandwidth.

Consider accessing a row-major $M \times N$ matrix. Iterating over columns (adjacent elements in memory) with consecutive threads produces coalesced accesses:

```
// Coalesced: threads in a warp read adjacent elements
float val = matrix[row * N + threadIdx.x];
```

Listing 2.6: Coalesced access pattern: consecutive threads read consecutive columns.

Iterating over rows with consecutive threads instead produces strided accesses with stride N, which is poorly coalesced:

```
// Non-coalesced: stride-N access
float val = matrix[threadIdx.x * N + col];
```

Listing 2.7: Non-coalesced access pattern: consecutive threads read elements separated by stride N.

This distinction is particularly relevant for tensor contractions, where the choice of loop ordering and index layout determines whether the innermost memory accesses are contiguous or strided.

Shared Memory Bank Conflicts

Shared memory is divided into 32 *banks*, each 4 bytes wide, interleaved in a round-robin fashion: word k resides in bank $k \bmod 32$. When multiple threads in a warp access different words in the same bank simultaneously, the accesses are serialised into multiple rounds, creating a *bank conflict*. The worst case is a 32-way bank conflict, where all threads hit the same bank, serialising the access entirely.

A common source of bank conflicts arises when accessing columns of a shared memory array whose leading dimension is a multiple of 32:

```
// 32-way bank conflict: column access, stride = 32
__shared__ float tile[32][32];
float val = tile[threadIdx.x][col];  // threads 0..31 all hit same bank

// Fix: pad the leading dimension by one
__shared__ float tile[32][33];       // stride = 33, conflicts eliminated
float val = tile[threadIdx.x][col];
```

Listing 2.8: Bank conflict when accessing a column of a 32-wide shared array, and the padding fix.

Adding one element of padding to the inner dimension changes the stride to 33, which is coprime to 32, so consecutive rows map to distinct banks. This is a standard optimisation in tiled kernels.

2.4.4. Performance Profiling with Nsight Compute

NVIDIA Nsight Compute is a kernel-level profiling tool for CUDA applications. It collects hardware performance counters and presents them as high-level metrics and roofline-model analyses, making it the primary tool for identifying performance bottlenecks in GPU kernels.

A typical profiling workflow proceeds as follows. First, the application is run under `ncu` (the Nsight Compute command-line interface), which replays each kernel multiple times to collect a full set of counters. Then the resulting report is examined either in the Nsight Compute GUI or by querying specific metrics from the command line.

Key metrics reported by Nsight Compute that are relevant to the optimisations discussed in subsequent chapters include:

- **Achieved occupancy:** the ratio of active warps per cycle to the maximum the SM can support, indicating how effectively the kernel hides memory latency through parallelism.
- **Memory throughput:** the achieved bandwidth to each level of the memory hierarchy (HBM, L2, shared memory), compared against the theoretical peak. A kernel achieving close to peak HBM bandwidth is memory-bound.
- **Compute throughput:** the achieved FLOP/s relative to the peak, broken down by instruction type. A kernel well below peak compute with high memory throughput is bandwidth-limited.
- **Warp stall reasons:** a breakdown of why warps were stalled (e.g. waiting for memory, waiting at a barrier, instruction dependencies), which directly guides optimisation.
- **Shared memory bank conflicts:** the number of replayed shared memory accesses due to bank conflicts, indicating whether padding or access pattern changes are needed.
- **L2 cache hit rate:** the fraction of global memory requests served by the L2 cache, relevant for understanding data reuse across thread blocks.

In the implementation chapters, Nsight Compute profiles are used to validate performance models, confirm that kernels operate in the expected regime (compute-bound or memory-bound), and guide iterative optimisation of tensor contraction kernels.

2.4.5. Kernel Launch Mechanics

The triple-chevron syntax `<<<gridDim, blockDim>>>` conceals a multi-stage pipeline that spans both host and device [NVI24b]. Understanding this pipeline is essential for reasoning about the cost of launching many small kernels, which is the regime encountered in tensor network contractions.

When the host executes a kernel launch, the following sequence of operations takes place:

1. **Argument marshalling.** The kernel's parameters—device pointers, scalars, dimensions—are packed into a parameter buffer and validated against the current CUDA context.
2. **Command enqueueing.** The launch command, containing the kernel function pointer, grid and block dimensions, dynamic shared memory size, and the parameter buffer, is placed into the stream's command queue. The host-side API call returns immediately; the launch is *asynchronous* from the host's perspective.

3. **Driver serialisation.** The CUDA driver serialises the queued command into the GPU's hardware command buffer.

4. **Block distribution.** The GPU's global block scheduler (referred to as the *GigaThread engine* in NVIDIA's architecture whitepapers [NVI20a]) dequeues the launch and begins distributing thread blocks to SMs. For each block, the scheduler verifies that the target SM has sufficient free resources—registers, shared memory, warp slots, and block slots—before making the assignment. Once assigned, a block remains on its SM until all of its warps complete execution; blocks do not migrate between SMs [NVI24b].

5. **Warp execution.** Within the assigned SM, the block's threads are partitioned into warps and handed to the processing block's warp scheduler, which begins issuing instructions as described in Section 2.3.1.

Steps 1–4 incur a fixed cost that is largely independent of the kernel's computational workload. On the A100, this cost is typically in the range of 5–10 μs per launch. For large kernels that execute for milliseconds, the overhead is negligible. For small kernels, however, the launch cost can exceed the computation time by an order of magnitude or more. This has a direct implication for workloads that require many small operations, such as batches of low-dimensional GEMM calls arising from tensor contractions: launching each operation as a separate kernel wastes the majority of the wall-clock time on launch overhead rather than useful computation.

MemcpyAsync vs Memcpy in Host–Device Timelines

Figures 2.13 and 2.14 illustrate two common execution patterns. The asynchronous version enqueues transfers and kernels and lets the host continue with CPU work until an explicit synchronisation point. The synchronous version uses blocking `cudaMemcpy`, which stalls the host thread during each transfer.

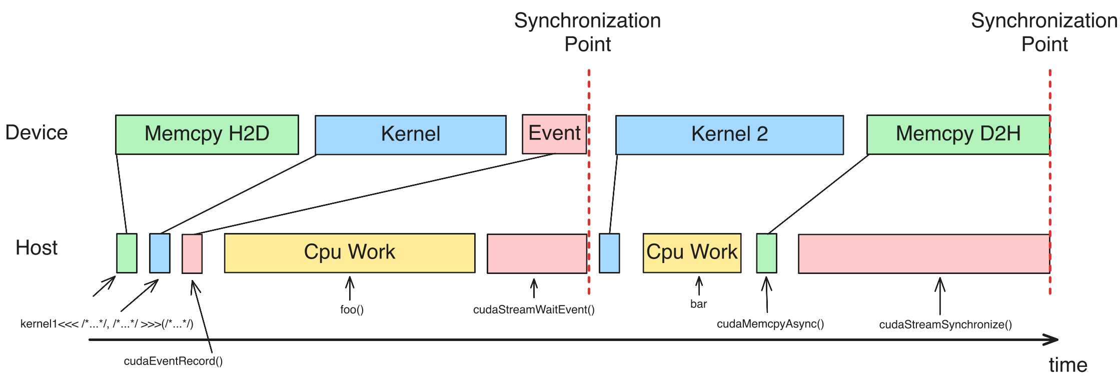


Figure 2.13.: Asynchronous host–device timeline using `cudaMemcpyAsync`, streams, and event-based ordering. Host work overlaps queued device work until explicit synchronisation.

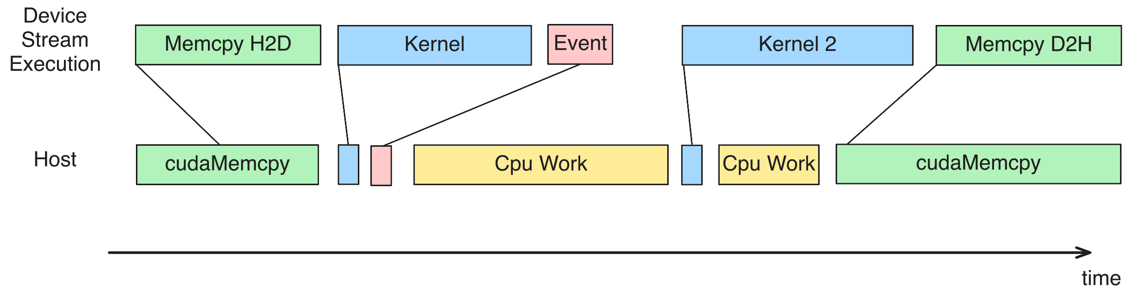


Figure 2.14.: Synchronous host–device timeline using blocking `cudaMemcpy`. The host is stalled during H2D and D2H transfers.

The corresponding asynchronous CUDA pattern is shown below. For true asynchronous host-device copies, host buffers should be page-locked (pinned), e.g. via `cudaMallocHost` [NVI24b, NVI25b].

```
cudaStream_t s1, s2;
cudaEvent_t done1;
cudaStreamCreate(&s1);
cudaStreamCreate(&s2);
cudaEventCreate(&done1);

// h_in / h_out should be pinned for true async copies.
cudaMemcpyAsync(d_in, h_in, bytes, cudaMemcpyHostToDevice, s1);
kernel1<<<grid1, block1, 0, s1>>>(d_in, d_tmp);
cudaEventRecord(done1, s1);

foo();  // host CPU work overlaps queued device work

cudaStreamWaitEvent(s2, done1, 0);
kernel2<<<grid2, block2, 0, s2>>>(d_tmp, d_out);

bar();  // more host CPU work

cudaMemcpyAsync(h_out, d_out, bytes, cudaMemcpyDeviceToHost, s2);
cudaStreamSynchronize(s2);  // explicit synchronization point
```

Listing 2.9: Asynchronous pattern with `cudaMemcpyAsync`, stream ordering, and explicit final synchronisation.

Practical interpretation. The asynchronous pattern is usually preferred when host-side orchestration can be overlapped with device execution. The synchronous pattern is simpler and often acceptable for single-shot transfers, but it introduces avoidable host idle time in pipelines with many transfer/compute stages.

Kernel Fusion

The standard mitigation for launch overhead is *kernel fusion*: combining multiple logically independent operations into a single kernel, so that the launch cost is paid once rather than once per operation. In the context of this thesis, fusion is achieved using cuBLASDx [NVI25a], which provides a device-side GEMM API that can be called from within a user-written kernel. This allows multiple GEMM operations to be executed sequentially (or cooperatively) inside a single kernel invocation, eliminating the per-operation launch cost entirely. The cuBLASDx library is introduced in Section 2.5.1, and the performance impact of fusion is quantified experimentally in Section 5.2.

2.5. Related Work

2.5.1. cuBLAS, cuBLASDx, and cuTENSOR

NVIDIA's vendor libraries define the practical baseline for dense kernels on Ampere GPUs. cuBLAS provides highly optimised BLAS routines and remains the reference point for GEMM-like workloads [NVI24a]. cuTENSOR generalises this to tensor contractions and permutation-heavy primitives [NVI24c].

For the small-kernel regime relevant to tensor-network workflows, cuBLASDx is especially important: it enables device-side GEMM composition and supports launch-amortisation strategies that are difficult to implement with host-only library calls [NVI25a]. The launch-overhead benchmark in this thesis (Section 5.2) uses this capability directly.

Custom kernels are evaluated against these libraries as performance baselines rather than replacements. The objective is to understand where workload-specific structure can beat general-purpose interfaces.

2.5.2. ChASE Eigensolver

The ChASE line of work provides relevant experience in GPU-oriented numerical software engineering, especially for hybrid CPU–GPU execution and scalable performance tuning [WSDN19, WDADN22, WDN23]. Although ChASE targets eigenvalue problems rather than tensor networks, it shares core HPC concerns with this work:

- balancing algorithmic structure with hardware-aware implementation,
- minimising communication and orchestration overhead,
- validating performance claims with rigorous benchmarking.

These principles influence the methodological choices in later chapters, particularly the emphasis on bottleneck-driven optimisation.

2.5.3. Existing GPU Tensor Network Implementations

Prior work on tensor contractions shows that performance is sensitive to index ordering and data layout, plus the mapping of contractions to BLAS kernels ([SSK17] provides an early and still useful reference point). Across implementations, two recurring observations are relevant here:

1. **Kernel granularity matters.** Many contractions are too small to fully utilise GPU throughput when issued individually.
2. **Data movement dominates.** Layout transforms and non-coalesced accesses can erase gains from nominally fast compute kernels.

This work builds on those observations with a stricter focus on Ampere-era execution details: launch overhead, occupancy limits, memory-hierarchy effects, plus profile-guided fusion on a realistic node topology.

2.5.4. Selected Work in Collaborating Research Directions

The following review focuses on work that is directly relevant to this project context: Wu, Di Napoli, Rizzi, Waintal, Legeza, and their close collaborators.

Wu and Di Napoli: GPU-Distributed Dense Kernels and Communication

Wu et al. report a distributed hybrid CPU–GPU eigensolver where the dominant kernel is a distributed Hermitian matrix–matrix multiplication step (`HEMM`), and show that replacing vendor `PZGEMM` with a custom GPU-distributed implementation significantly improves strong scaling on JUWELS-Booster [WDADN22]. The SC-W extension further improves distributed behaviour with NCCL-based communication optimisations [WDN23].

A practical architectural lesson is clear: once local kernels are efficient, communication orchestration and overlap become major bottlenecks.

Legeza Group: Massively Parallel DMRG/MPS on CPU–GPU Systems

Menczer and Legeza report peer-reviewed JCTC results on massively parallel tensor-network algorithms and accelerator-oriented implementations, with complementary preprints for symmetry-aware and newest H100-focused optimisations ([ML25, ML24a, ML$^+$23a, ML$^+$24b]).

These studies report good scaling across heterogeneous CPU–GPU systems and explicitly motivate mixed-precision/tensor-core execution for contraction-heavy workloads [ML25, ML24a, ML$^+$24b]. The direct transfer for this project is straightforward: reduce launch count for small contractions, bucket by shape, then tune stream-level scheduling.

Rizzi and Waintal: Operator-Sum Formulations and Practical TN Performance

Krinitsin, Rizzi, and Waintal present a TTN time-evolution implementation with two-level parallelisation (shared and distributed memory) and report order of magnitude speedups from GPU acceleration in the targeted workload regime [KRW25]. A practical point for this project is their emphasis on local operator-sum formulations, which can avoid expensive explicit intermediate constructions in contraction pipelines.

Waintal and collaborators provide broader algorithmic limits and scalable simulation perspectives for circuit-style tensor-network workloads ([ZSW20, A$^+$23]). Even where papers are not CUDA-kernel papers, the computational patterns they identify (many medium/small contractions, truncation-heavy iterative sweeps) match the exact regime where launch overhead and layout efficiency matter most, with batching as the main countermeasure on A100-class hardware.

A100 CUDA Optimisation Practice (Documentation SOTA)

For low-level CUDA engineering on Ampere, the most useful guidance comes from NVIDIA's architecture and profiling documentation rather than from application papers. The recurring recommendations are:

1. treat memory traffic and reuse as first-order constraints (coalescing, shared-memory tiling, and cache-aware layouts),

2. use occupancy as a constraint, not a sole objective; register and shared-memory pressure must be balanced against instruction-level efficiency,
3. reduce launch overhead for short kernels through batching, fusion, and stream-graph style execution where appropriate,
4. validate each optimisation with metric-level profiling instead of relying on throughput numbers alone.

These points are consistent across the Ampere tuning guide, the CUDA best-practices guide, and the Nsight Compute profiling guide ([NVI25d, NVI25b, NVI25e]).

Integration priorities. Given the current project scope (single A100 node, contraction-heavy short kernel sequences), the most relevant near-term integration steps are:

1. build a shape-bucketed benchmark suite for representative contraction sizes,
2. maintain a per-kernel profiler checklist (occupancy, memory throughput, stall breakdown, launch cost),
3. prioritise launch amortisation before aggressive micro-optimisation for kernels that run in the sub-millisecond regime,
4. keep FP32 as baseline and treat TF32 as an opt-in acceleration path with explicit numerical checks.

Recurring Implementation Patterns

Across the above works, four recurring implementation patterns emerge:

1. **Task bucketing by contraction shape.** Grouping same-shape small contractions enables batched GEMM and reduces launch count.
2. **Operator-sum execution paths.** Avoiding explicit large operator objects can reduce memory traffic and intermediate-kernel overhead.
3. **Communication–computation overlap.** Multi-GPU execution benefits from explicit stream partitioning and asynchronous collectives.
4. **Mixed-precision acceleration with guarded validation.** FP32/TF32 tensor-core execution is used for throughput, with strict numerical checks.

Paper Notes for Implementation Planning (Working Draft)

The table below is kept as implementation-oriented notes. It maps paper-level results and methodology to concrete optimisation actions for this project.

Table 2.17.: Working notes: transfer from selected literature to CUDA optimisation actions in the project.

Paper (focus)	Reported methodology/result signal	Direct project action
[ML25] (massively parallel TN state algorithms on CPU–GPU systems)	explicit multi-level parallelisation and mixed CPU/GPU execution for contraction-heavy workloads	keep kernels bucketed by shape and preserve stream-level concurrency; avoid one-kernel-per-contraction execution for small blocks
[ML24a] (TN algorithms on AI accelerators)	hardware-specific kernel organisation and execution strategy strongly influence effective performance, not just raw peak throughput	separate profiling by contraction class and layout class; avoid aggregating all kernels into one average metric
[ML+24b] (DMRG implementation/tuning with tensor-core usage)	strong emphasis on mixed precision and throughput-oriented kernel organisation	keep FP32 baseline, treat TF32 as opt-in path with fixed numerical validation; prioritise high-call-count kernels first
[KRW25] (TTN time evolution, Rizzi/Waintal collaboration)	two-level parallelisation and GPU acceleration provide major practical gains in targeted regimes	retain local-operator-sum style execution when possible and minimise explicit large intermediates that create extra memory traffic
[WDADN22, WDN23] (GPU-distributed dense eigensolver lineage at JSC)	once local kernels are efficient, communication/orchestration limits dominate scaling	for one-node scope, prepare kernels and APIs so later overlap of local compute and communication is possible without redesign

2.5.5. Representative CUDA Implementation Patterns

The following snippets are representative implementation skeletons derived from the above literature patterns. They are intentionally simplified to expose the kernel-architecture idea.

Pattern A: Bucketed Small-GEMM Execution (DMRG/MPS Sweeps)

```
struct BatchBucket {
    int m, n, k, count;
    const float* A;   // strided batch base
    const float* B;
    float* C;
    long long strideA, strideB, strideC;
};

void run_bucket(const BatchBucket& b, cublasHandle_t h, cudaStream_t s) {
    cublasSetStream(h, s);
```

```
    const float alpha = 1.0f, beta = 0.0f;
    cublasGemmStridedBatchedEx(
        h, CUBLAS_OP_N, CUBLAS_OP_N,
        b.m, b.n, b.k,
        &alpha,
        b.A, CUDA_R_32F, b.m, b.strideA,
        b.B, CUDA_R_32F, b.k, b.strideB,
        &beta,
        b.C, CUDA_R_32F, b.m, b.strideC,
        b.count,
        CUBLAS_COMPUTE_32F_FAST_TF32, CUBLAS_GEMM_DEFAULT_TENSOR_OP);
}
```

Listing 2.10: Shape-bucketed small contractions using strided batched GEMM. Representative kernel-orchestration pattern inspired by massively parallel DMRG implementations.

Why it matters. Instead of launching one kernel per tiny contraction, this pattern executes many contractions per call and shifts the regime away from launch-dominated runtime.

Pattern B: Operator-Sum Path Without Explicit MPO Materialisation

```
// Pseudocode-level sketch: each term is mapped to a GEMM-like contraction.
for (int t = 0; t < num_terms; ++t) {
    // left[t], right[t], and local_op[t] encode one local contribution
    contract_term_kernel<<<grid, block, shmem, stream>>>(
        left[t], local_op[t], right[t], state_in, state_tmp);
}

// Optional: fuse accumulation to reduce global-memory traffic.
accumulate_terms_kernel<<<grid2, block2, 0, stream>>>(state_tmp, state_out);
```

Listing 2.11: Operator-sum contraction sketch. The core idea is to accumulate local terms directly instead of materialising a large explicit operator tensor first.

Why it matters. It follows the local-operator-sum idea discussed in recent TTN benchmarking. Intermediate tensor construction is reduced, and cache/memory-traffic behavior often improves [KRW25].

Pattern C: Overlap of Local Compute and Multi-GPU Reduction

```
cudaStream_t s_compute, s_comm;
cudaStreamCreate(&s_compute);
cudaStreamCreate(&s_comm);

// 1) Local block update on each GPU
local_update_kernel<<<grid, block, 0, s_compute>>>(local_A, local_B, local_C);

// 2) Asynchronous inter-GPU reduction / exchange
ncclAllReduce(local_C, global_C, count, ncclFloat, ncclSum, comm, s_comm);

// 3) Synchronize only when data dependency requires it
cudaEvent_t done_comm;
cudaEventCreate(&done_comm);
cudaEventRecord(done_comm, s_comm);
```

```
cudaStreamWaitEvent(s_compute, done_comm, 0);
postprocess_kernel<<<grid2, block2, 0, s_compute>>>(global_C);
```

Listing 2.12: Two-stream overlap pattern for distributed dense kernels. The communication stream runs asynchronous collectives while compute proceeds on the main stream.

Why it matters. It mirrors the communication/computation orchestration emphasis in Wu–Di Napoli style distributed GPU kernels and in massively parallel DMRG implementations [WDADN22, WDN23, ML25].

3. Design and Methodology

This chapter defines the optimisation target and methodology before the implementation details. It states the in-scope kernel patterns and baselines, then links data-layout decisions to testable performance hypotheses.

3.1. Target Kernels

The target workload is defined by contraction patterns that can be reduced to dense linear algebra kernels after index permutation and reshaping. We focus on kernel classes that are both frequent in tensor-network workflows and sensitive to GPU execution overhead:

1. **Small/medium GEMM-like contractions** (single and batched), including cases where dimensions are not multiples of warp tile sizes.
2. **Layout-transformation kernels** (permute/pack/unpack) required before and after contractions.
3. **Fused kernel sequences** that execute several contractions (and optional lightweight post-processing) inside one launch.

The baseline precision target is FP32 for compute and storage, with optional TF32 tensor-core execution where numerical tolerance permits it. FP64 remains a reference configuration for accuracy and performance comparison.

3.1.1. Selection Criteria

A kernel is included in the optimisation set if it satisfies at least two of the following conditions:

- high call frequency in representative traces,
- measurable contribution to end-to-end runtime,
- clear exposure to launch overhead or memory-traffic bottlenecks,
- portability to both single-GPU and one-node multi-GPU execution.

3.1.2. SOTA-Informed Initial Backlog (Working Notes)

Before full application traces are fixed, the optimisation backlog is organised as a set of generic kernel classes that repeatedly appear in the cited literature and in early profiling:

1. **Shape-bucketed small GEMM path.** Group contractions by (m, n, k) class and run batched/device-side execution to reduce launch count.

2. **Layout-heavy micro-pipeline path.** Fuse reshape/pack/contract segments when data dependencies allow it, and track whether reduced launch count offsets higher register/shared-memory pressure.

3. **Local-operator-sum contraction path.** Prefer execution plans that avoid explicit large intermediate operators when equivalent local-term accumulation is available.

For each class, the first profiling pass records the same minimum metric set: kernel runtime, launch count, achieved occupancy, DRAM throughput, plus dominant warp-stall reasons. This keeps early optimisation choices comparable even before the final production kernel list is fixed.

3.2. Case Study: Contraction Order in a Two-Site TN Update

A concrete tensor-network-style case shows how a reasonable first implementation can be much slower than a mathematically equivalent alternative.

3.2.1. Problem Definition

Consider a batched two-site update with fused physical dimension $d_p = d^2$:

$$Y_{b,p,r} = \sum_{q=0}^{d_p-1} \sum_{c=0}^{\chi-1} G_{p,q}\, X_{b,q,c}\, M_{c,r},$$

where:

- $b \in [0, B)$ is a batch index,
- $p, q \in [0, d_p)$ are fused physical indices,
- $c, r \in [0, \chi)$ are bond-space indices.

The tensor structure is representative of two-site gate application and local environment projection steps in MPS-style workflows.

3.2.2. HPC Formulation

The expression can be read as a matrix-chain computation repeated over a batch:

- for each fixed batch index b, interpret $X_{b,\cdot,\cdot}$ as a matrix $X_b \in \mathbb{R}^{d_p \times \chi}$,
- interpret G as $d_p \times d_p$ and M as $\chi \times \chi$,
- compute
$$Y_b = (G\, X_b)\, M, \qquad b = 0, \ldots, B-1.$$

From an implementation point of view, the core is a repeated dense linear-algebra pattern where contraction order decides both arithmetic work and memory traffic.

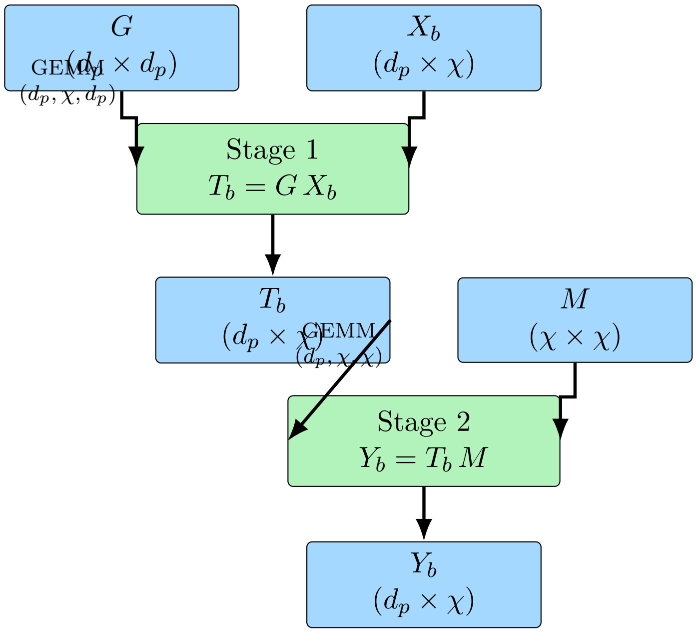


Figure 3.1.: Per-batch matrix view of the two-site update. The tensor expression is equivalent to a two-stage GEMM chain with a shared G and M across batches.

3.2.3. Direct Baseline

A straightforward GPU implementation computes one output element $Y_{b,p,r}$ per thread and evaluates both sums directly. This gives a direct cost

$$\mathcal{O}\Big(B\, d_p^2\, \chi^2\Big)\,,$$

with weak reuse of X and M across threads and heavy global-memory traffic. This baseline is implemented in `tn_two_site_bad_direct.cu`.

3.2.4. Ordered Contraction Formulation

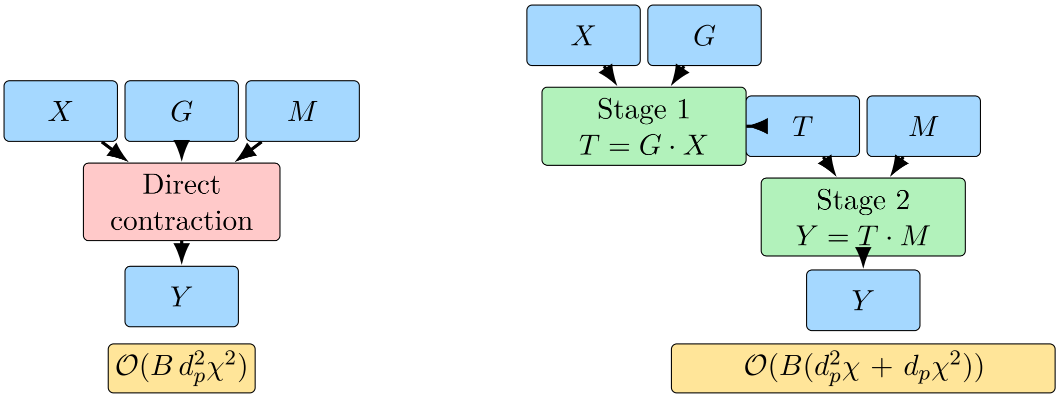
The change is the contraction order:

$$T_{b,p,c} = \sum_{q=0}^{d_p-1} G_{p,q} X_{b,q,c}, \qquad Y_{b,p,r} = \sum_{c=0}^{\chi-1} T_{b,p,c} M_{c,r}.$$

This reduces asymptotic work to

$$\mathcal{O}\Big(B\,(d_p^2\chi + d_p\chi^2)\Big)\,,$$

and maps directly to two strided-batched GEMMs. The implementation is `tn_two_site_good_batched_gemm.cu`.



(a) Direct baseline (single fused loop nest). (b) Ordered contraction (two GEMM stages).

Figure 3.2.: Algorithmic structure of the two-site case study: direct contraction (left) versus contraction-order decomposition into two GEMM-like stages (right).

Theoretical operation-ratio improvement (direct vs ordered) is

$$R = \frac{d_p^2 \chi^2}{d_p^2 \chi + d_p \chi^2} = \frac{d_p \chi}{d_p + \chi}.$$

For the benchmarked shape $d_p = 16$, $\chi = 256$, this gives $R \approx 15.1$, before any hardware-level optimisations.

3.2.5. Worked Numerical Example (Reference Shape)

Using the benchmarked shape $B = 1$, $d_p = 16$, $\chi = 256$:

- output size is $d_p \chi = 4096$ elements,
- direct variant evaluates $d_p \chi = 4096$ inner terms per output element,
- total direct term evaluations are

$$N_{\text{direct}} = d_p^2 \chi^2 = 16\,777\,216.$$

For ordered contraction:

$$N_{\text{stage1}} = d_p^2 \chi = 65\,536, \qquad N_{\text{stage2}} = d_p \chi^2 = 1\,048\,576,$$

so

$$N_{\text{ordered}} = N_{\text{stage1}} + N_{\text{stage2}} = 1\,114\,112.$$

This gives

$$\frac{N_{\text{direct}}}{N_{\text{ordered}}} \approx 15.06,$$

which matches the asymptotic ratio above.

Variant	Term evaluations	Relative to ordered
Direct contraction	16 777 216	15.06×
Ordered contraction (two-stage)	1 114 112	1.00×

Table 3.1.: Work comparison for the thesis reference shape ($B = 1$, $d_p = 16$, $\chi = 256$).

3.2.6. Design Decisions Behind the Better Variant

The implementation sequence follows the staged optimisation progression commonly used in incremental GEMM tuning studies [Boe22]: start with algorithmic structure, then improve memory behavior, then improve execution mapping.

Design decision	Why it helps
Change contraction order first	Removes redundant work ($\mathcal{O}(Bd_p^2\chi^2) \rightarrow \mathcal{O}(B(d_p^2\chi + d_p\chi^2))$).
Map to strided-batched GEMM	Uses highly tuned kernels instead of manual scalar loops.
Reuse shared operands with zero stride	G and M are reused across the whole batch without host-side relaunch loops.
Keep strict FP32 math mode	Preserves strict FP32 behavior while comparing algorithmic variants.
Profile with fixed KPI pack	Separates true algorithm gains from incidental launch/measurement noise.

Table 3.2.: Main design decisions for the two-site contraction case study.

3.2.7. Optimisation Stages

Beyond the single direct-vs-ordered comparison, this case follows a staged optimisation sequence: fix algorithmic work first, then remove memory inefficiencies, then improve scheduler feed and occupancy, and finally tune shape-dependent parameters.

1. **Stage 0 (correct baseline).** Start from a slow but clearly correct direct contraction to define a reproducible reference.

2. **Stage 1 (contraction order).** Remove redundant arithmetic in naive loop nests first; this is often the largest single gain for TN updates.

3. **Stage 2 (global-memory access).** Fix non-coalesced and heavily strided operand traversal so traffic scales with useful work.

4. **Stage 3 (on-chip reuse).** Avoid re-reading hot operands from HBM by increasing L2/shared-memory reuse where possible.

5. **Stage 4 (scheduler feed / ILP).** Increase independent work per warp to improve eligible-warps and issue efficiency.

6. **Stage 5 (resource balance).** Control register growth so occupancy does not collapse under aggressive unrolling/blocking.

7. **Stage 6 (launch cost).** Fuse tiny pre/post kernels around the contraction to amortise host launch overhead.

8. **Stage 7 (parameter search).** Tune tile/shape mappings per regime, since good choices are hardware- and size-dependent.

3.2.8. Profiling Reproducibility

The complete compile-and-profile job is provided as `ncu_tn_contraction_profile.slurm`. It collects the same KPI pack used in the other profiling sections and writes separate CSV files for both variants.

3.3. Algorithmic Approach

The optimisation strategy follows a profile-driven iterative loop:

1. establish a reproducible baseline with library implementations,
2. measure kernel-level behaviour with Nsight metrics,
3. construct a bottleneck hypothesis (launch-bound, bandwidth-bound, or compute-bound),
4. implement one targeted optimisation,
5. re-profile and accept/reject the change based on quantitative criteria.

This keeps low-level tuning tied to measured bottlenecks instead of guesswork.

3.3.1. Optimisation Axes

Four axes are prioritised throughout the implementation chapters:

- **Launch amortisation:** reduce host-side orchestration overhead through kernel fusion and device-side composition.
- **Memory-efficiency optimisation:** maximise coalesced access and data reuse via layout control and on-chip reuse (shared memory plus register blocking).
- **Occupancy-resource balance:** tune block sizes and launch-bounds against register and shared-memory budgets.
- **Precision-performance trade-offs:** evaluate FP64 and FP32 modes under fixed correctness checks.

3.3.2. Acceptance Criteria

An optimisation is considered successful only if:

- it improves runtime or throughput on the target workload set,
- the profiling counters match the intended mechanism (not incidental noise),
- numerical deviation remains within predefined tolerance,
- implementation complexity remains maintainable for further tuning.

3.3.3. Analytical Performance Models

Before implementation-level tuning, we use two compact analytical models to classify bottlenecks and to set realistic speedup expectations: roofline analysis for single-kernel limits and Amdahl's law for global parallel-scaling limits.

Roofline Reference Model

The canonical roofline definitions and bounds are given in Section 2.2. Here, the model is instantiated with hardware-specific FP32 ridge points for the platforms in scope:

$$I^\star_{\text{A100-40}} \approx 12.5, \quad I^\star_{\text{A100-80}} \approx 9.6, \quad I^\star_{\text{H100-80}} \approx 20.0 \text{ FLOPs/byte}.$$

Small and medium contractions with low reuse remain memory-bound unless layout/tiling increases operational intensity.

Roofline in Practice

In this project, roofline is used as an analysis tool, not just as a plot. The procedure is:

1. estimate FLOPs and bytes from kernel geometry (static model),
2. collect measured FLOPs, memory traffic, and runtime from Nsight Compute [NVI25e],
3. place each kernel on the roofline and compare against the bound,
4. choose the next optimisation based on the dominant gap.

With measured FLOPs F, measured runtime t, and measured DRAM traffic Q_{dram}, we use

$$I_{\text{meas}} = \frac{F}{Q_{\text{dram}}}, \qquad P_{\text{meas}} = \frac{F}{t}, \qquad \eta = \frac{P_{\text{meas}}}{\min(P_{\max}, B\, I_{\text{meas}})}.$$

Here η is a roofline efficiency ratio relative to the active bound.

Interpretation for optimisation decisions:

- if $I_{\text{meas}} < I^\star$ and η is low, prioritise memory coalescing and data reuse; adjust layouts where needed;
- if $I_{\text{meas}} \geq I^\star$ and η is low, prioritise instruction-level efficiency and occupancy/resource balance, then reduce launch overhead;
- if end-to-end speedup is still low while kernel η improves, serial orchestration is likely the limiting factor (Amdahl regime).

Arithmetic-Intensity Map for Rectangular Kernels

To isolate shape effects before full kernel implementation, we model a GEMM-like kernel

$$C_{M\times N} \leftarrow A_{M\times K} B_{K\times N} + C_{M\times N}$$

with element size s bytes (FP32: $s = 4$). Under an ideal one-pass traffic model (read A, read B, read+write C), the arithmetic intensity is

$$I_{\text{ideal}}(M, N, K) = \frac{2MNK}{s\,(MK + KN + 2MN)}.$$

If dimensions are padded to a tile size t for tensor-core-friendly execution, define

$$\hat{M} = \left\lceil \frac{M}{t} \right\rceil t, \quad \hat{N} = \left\lceil \frac{N}{t} \right\rceil t, \quad \hat{K} = \left\lceil \frac{K}{t} \right\rceil t.$$

Using useful FLOPs ($2MNK$) over padded traffic gives the effective intensity

$$I_{\text{eff}}(M, N, K; t) = \frac{2MNK}{s\,(\hat{M}\hat{K} + \hat{K}\hat{N} + 2\hat{M}\hat{N})}.$$

For the target shape regime, we use $K = 64$ and $t = 16$. Figure 3.3 maps I_{ideal} and I_{eff} across $M, N \in [16, 256]$. The trend is consistent with an area-to-perimeter argument: larger and more balanced rectangles increase intensity, while slender shapes and heavy padding reduce it. Under the same traffic model, moving from FP32 to FP64 doubles bytes per value and therefore halves arithmetic intensity at every point of the map.

Table 3.3.: Representative arithmetic-intensity values for fixed $K = 64$ in FP32 with $t = 16$ padding.

Shape (M, N)	I_{ideal}	I_{eff}	Drop (%)	Padded $(\hat{M}, \hat{N}, \hat{K})$
$(47, 84)$	7.76	6.85	11.7	$(48, 96, 64)$
$(63, 95)$	8.67	8.50	2.0	$(64, 96, 64)$
$(65, 130)$	9.20	7.23	21.4	$(80, 144, 64)$
$(111, 73)$	9.27	8.58	7.4	$(112, 80, 64)$
$(128, 128)$	10.67	10.67	0.0	$(128, 128, 64)$

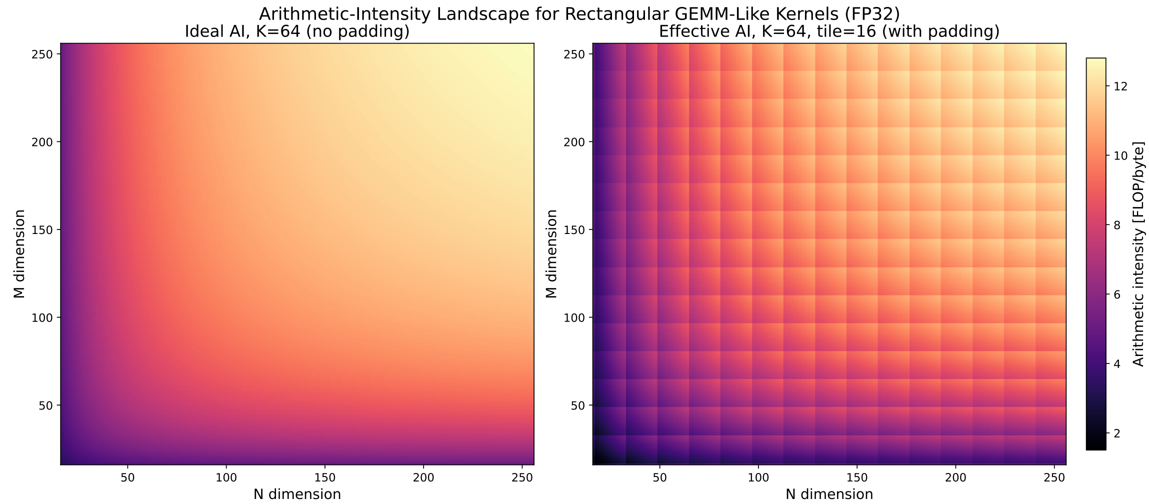


Figure 3.3.: Arithmetic-intensity map for rectangular GEMM-like kernels in FP32 at fixed $K = 64$. Left: ideal intensity without padding. Right: effective intensity with 16-element padding on M, N, K (useful FLOPs over padded traffic).

GEMM Storage Footprint vs A100 Capacity

For square GEMM dimensions (k, k, k), storing input and output matrices (A, B, C) requires

$$M(k; s) = 3k^2 s$$

bytes, where s is bytes per element ($s = 4$ for FP32, $s = 8$ for FP64). This is a storage model only (no temporary workspace, no extra staging buffers). The capacity-limited crossover dimension for a memory budget C is

$$k_{\max}(C; s) = \sqrt{\frac{C}{3s}}.$$

Table 3.4.: Capacity-limited square dimensions for the GEMM storage model $M(k; s) = 3k^2 s$.

Memory limit C	$k_{\max}$ (FP32)	$k_{\max}$ (FP64)
A100 HBM (40 GB)	57 735	40 825
A100 shared memory per SM (164 KiB)	118	84

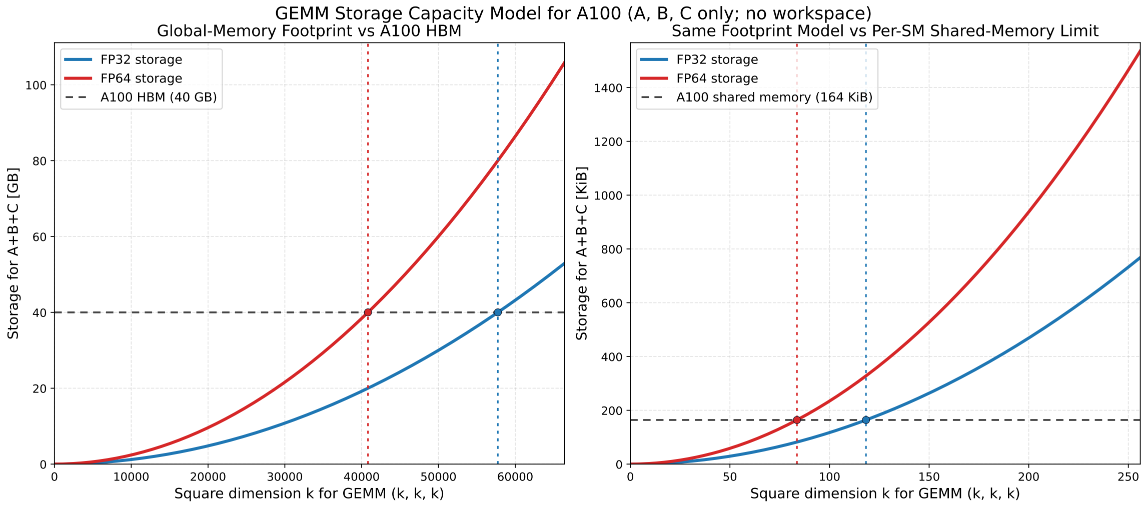


Figure 3.4.: Storage model $M(k; s) = 3k^2 s$ for square GEMM in FP32 and FP64. Left: intersection with A100 HBM capacity (40 GB). Right: same model against per-SM shared-memory capacity (164 KiB).

Precision choice amplifies all three advantages. The numbers in Table 3.4 make the compound effect of choosing FP32 over FP64 concrete for the A100. First, the maximum square tile that fits into a single SM's shared memory grows from 84×84 to 118×118—a ~40% increase in each dimension and roughly a $2\times$ increase in element count—which allows a thread block to cover a larger output region per launch and improves data reuse within the tile. Second, halving the element width from 8 to 4 bytes doubles the effective bandwidth of every level of the memory hierarchy (HBM, L2, shared memory), shifting the roofline ridge point to lower arithmetic intensities and widening the compute-bound regime where tensor cores are fully utilised. Third, the A100 provides $2\times$ the FP32 CUDA-core throughput and $8\times$ the effective FP32 tensor-core throughput (TF32) compared with FP64 (Section 2.3.3). These three effects—larger tiles, doubled bandwidth, and higher compute ceilings—compound multiplicatively, which is why the FP32 roofline envelope in Figure 3.5 sits dramatically above the FP64 envelope despite the same physical hardware.

As argued in Section 2.3.3, the physics modelling in tensor network computations does not require FP64: bond-dimension truncation errors (10^{-4} to 10^{-6}) dominate FP32 rounding ($\sim 10^{-7}$ per operation), iterative algorithms self-correct accumulated rounding, and the reduction lengths in typical contractions (K in the low hundreds) keep FP32 well within its reliable range (Section 2.2.3). Choosing FP32 is therefore not a precision compromise but an informed trade-off that unlocks substantially more of the hardware's capability.

Scaling-Bound Interpretation

The canonical Amdahl and Gustafson scaling relations are defined in Section 2.2. Here they are used only as interpretation bounds: if kernel-local efficiency improves while end-to-end scaling saturates, residual serial orchestration and communication terms dominate.

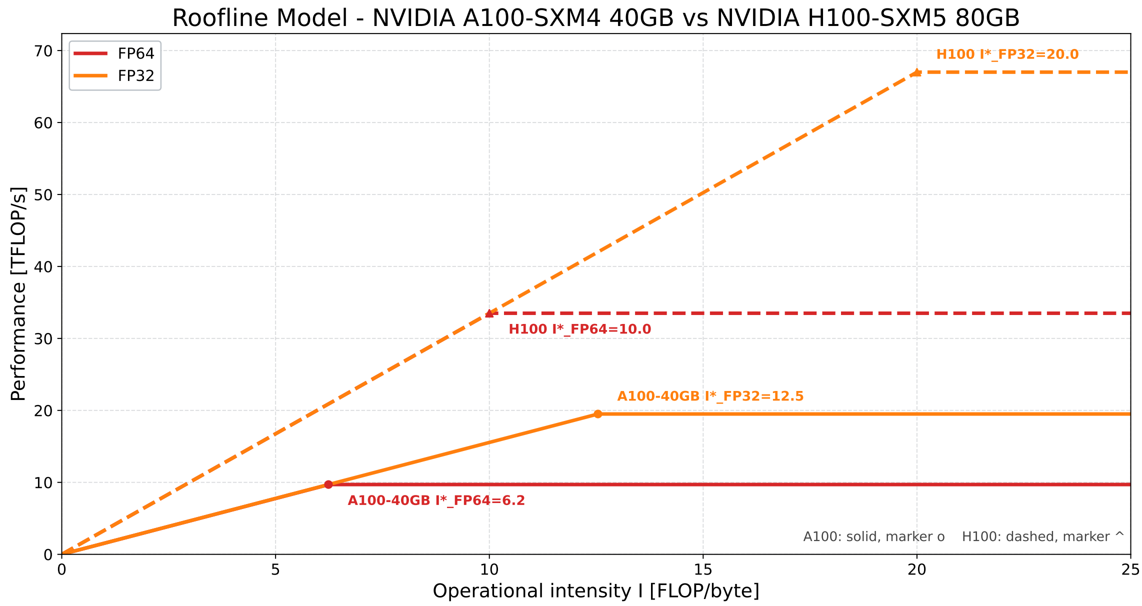


Figure 3.5.: Combined roofline envelopes for A100-SXM4-40GB and H100-SXM5-80GB in one shared axis, using FP64 and FP32 ceilings. Solid lines correspond to A100-SXM4-40GB, dashed lines to H100-SXM5-80GB; ridge markers indicate arithmetic-intensity crossover points.

3.4. Data Layout and Memory Strategy

Data layout is handled early in kernel design, not patched in afterwards. For contraction-heavy workloads, memory access patterns often dominate arithmetic cost, so layout and kernel structure are decided together.

3.4.1. Layout Principles

The implementation follows four layout rules:

1. keep the innermost thread-mapped dimension contiguous in memory to preserve coalesced global accesses,

2. minimise explicit transpose and permutation kernels by selecting a stable canonical in-memory layout,

3. perform reshape/pack operations in fused paths when possible,

4. use alignment-friendly leading dimensions to improve transaction efficiency and vectorised loads.

At warp level, a practical rule is that lane i should map to a nearby address, ideally

$$\mathrm{addr}(i) = \mathrm{base} + 4i.$$

This keeps a warp load/store close to fully coalesced and minimises the number of memory transactions. In contrast, strided mappings such as

$$\mathrm{addr}(i) = \mathrm{base} + 4si, \quad s \gg 1$$

fragment traffic into many split transactions and reduce effective bandwidth.

3.4.2. Memory-Level Strategy

The memory strategy mirrors the A100 hierarchy:

- **Registers:** accumulate partial results and keep loop-invariant scalars local.
- **Shared memory:** stage reused tiles and remove redundant global loads.
- **L2/HBM:** organise global accesses as contiguous bursts and avoid strided warp patterns unless unavoidable.

When tensors require non-trivial index reorderings, preprocessing overhead is quantified explicitly and accounted for in end-to-end performance claims.

3.5. Baseline Selection

To make performance claims meaningful, all custom kernels are compared against well-defined baselines under matched problem sizes and precision modes, with compatible memory-layout assumptions.

3.5.1. Primary Baselines

- **cuBLAS:** reference for GEMM-like contraction kernels.
- **cuTENSOR:** reference for direct tensor contraction and permutation-centric workflows.
- **cuBLASDx compositions:** reference for device-side small-GEMM execution where launch amortisation is central.

3.5.2. Fairness Criteria

Each comparison uses:

1. identical input tensors and dimensions,
2. identical precision modes and accumulation rules,
3. warm-up runs and repeated measurements for variance control,
4. consistent stream and synchronisation semantics.

Reported metrics include median runtime, throughput (GFLOP/s or TFLOP/s), effective memory bandwidth, and speedup relative to the strongest baseline per workload.

3.5.3. Profiling Evidence Gate

To keep optimisation claims defensible, each accepted speedup is paired with a matching profiler signal:

1. **Launch-amortisation claim:** lower launch count and lower host or launch-overhead fraction at similar numerical output.
2. **Memory-efficiency claim:** higher effective throughput and/or improved coalescing/reuse indicators with matching or lower runtime.
3. **Occupancy/resource claim:** improved runtime with a coherent change in achieved occupancy and stall breakdown.
4. **Precision-path claim (FP32/TF32):** runtime gain plus explicit numerical-deviation check against the FP64/FP32 reference path.

This gate is applied uniformly to custom kernels and library-based baselines, so comparison quality is maintained while the kernel set is still evolving.

3.6. Methodology and Reporting Standards

This section defines experiment execution, profiling order, and reporting requirements. The aim is to keep optimisation claims traceable and reproducible across kernels and optimisation stages.

3.6.1. Profiling Workflow Standard

Profiling is executed in two stages:

1. **System-level trace first (Nsight Systems).** Identify launch gaps and synchronisation bottlenecks before collecting kernel microarchitecture counters. In the same pass, check whether overlap is actually occurring. Traces are restricted to explicit regions of interest and short wall-clock windows to limit profiling perturbation and trace bloat [NVI25f, Pur24].

2. **Kernel-level trace second (Nsight Compute).** After the timeline bottleneck is localised, collect a fixed KPI pack for attribution. For routine iteration, use a minimal metric set and kernel filters; broad metric sweeps are reserved for focused deep dives because replay passes increase overhead and can bias short kernels [NVI25e].

The order is intentional: first find *where* time is spent, then explain *why* with counters.

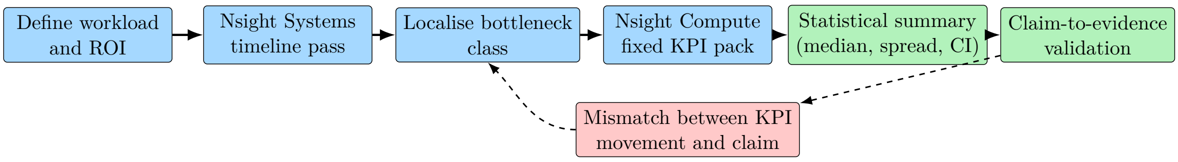


Figure 3.6.: Standard profiling-and-reporting pipeline. Timeline localisation precedes counter collection, and claim validation loops back when mechanism evidence is inconsistent.

3.6.2. Benchmark Design Standard

Each benchmark campaign follows these rules:

1. **Fixed conditions of observation:** hardware partition and software stack are fixed; compile flags and precision mode are fixed, and CPU/GPU affinity is fixed as well.

2. **Warm-up and steady-state timing:** warm-up iterations remove first-launch effects; measured iterations are reported separately.

3. **Replicated measurements:** repeated runs are required, with robust summary statistics (median and dispersion).

4. **Bias control:** run-order effects and hidden configuration dependencies are treated as validity risks and explicitly checked.

5. **Cross-workload aggregation:** when normalised speedups are aggregated across workloads, geometric means are used.

These choices follow established benchmarking guidance from systems literature [KJ13, MDHS09, FW86] and reporting standards in benchmark suites [SPE21].

3.6.3. Mandatory Reporting Items

Item	Minimum required disclosure
Platform	GPU model, interconnect/topology, CPU model, memory capacity.
Software stack	Driver, CUDA toolkit, compiler, library versions, profiler version.
Build configuration	Optimisation flags, debug/lineinfo flags, math mode (FP32/TF32/BF16/FP64).
Workloads	Exact tensor dimensions, batch sizes, data types, and input-generation policy.
Timing protocol	Warm-up count, measured iterations, repetitions, synchronisation boundaries.
Profiling protocol	Tool commands, kernel filters, metric set, launch-skip/count, replay-relevant options.
Statistics	Point estimate definition (e.g. median) and spread/uncertainty summary.
Correctness criteria	Numerical tolerance, reference implementation, validation frequency.
Artifacts	Script paths for run submission, CSV extraction, and figure generation.

Table 3.5.: Mandatory disclosure checklist for the benchmark campaign.

3.6.4. Claim-to-Evidence Rules

To avoid over-claiming, each optimisation claim must satisfy:

1. **Effect:** measurable runtime/throughput change on the target workload distribution.
2. **Mechanism match:** KPI movement is consistent with the proposed mechanism (e.g. memory, scheduler, occupancy, divergence).
3. **Numerical validity:** output deviation remains within tolerance.
4. **Reproducibility:** the result can be regenerated from documented scripts and environment settings.

If an effect is observed but mechanism counters do not match, the result is reported as empirical but non-attributed; no causal interpretation is claimed.

3.6.5. Threats-to-Validity Template

Each results subsection reports the following validity classes:

- **Internal validity:** profiler perturbation, run-to-run noise, and hidden synchronisation.
- **Construct validity:** mismatch between chosen KPIs and the intended bottleneck interpretation.
- **External validity:** portability limits across architectures, software versions, and tensor-shape distributions.

Mitigations are stated per experiment, using the mandatory reporting items in Table 3.5.

4. Implementation

This chapter translates the design choices into concrete CUDA implementation decisions, with emphasis on resource-aware kernel engineering and reproducible integration.

4.1. Integration and Build System

The implementation is integrated as a reproducible CUDA/C++ project with clear separation between benchmark harness and kernel library, with profiling scripts kept independent. This structure is necessary to compare optimisation variants consistently across multiple datasets and precision modes.

4.1.1. Build Configuration

All kernels target Ampere compute capability (`sm_80`) and are compiled in release mode with architecture-specific optimisation enabled. The build configuration records:

- compiler and CUDA toolkit versions,
- target architecture flags,
- enabled precision modes and feature toggles,
- linked baseline libraries (cuBLAS, cuTENSOR, cuBLASDx where used).

This metadata is logged with benchmark output so each result can be traced to a specific binary configuration.

4.1.2. Runtime Integration

Kernel entry points are exposed through a common interface that supports:

1. baseline execution,
2. custom unfused kernel execution,
3. fused-kernel execution paths.

Using a unified interface allows controlled A/B testing with minimal host-side differences between variants.

4.2. Kernel Design

Kernel design follows a bottom-up strategy: start from a minimal correct implementation, then introduce one optimisation mechanism at a time (tiling, fusion, vectorised loads, launch configuration tuning), validating each step by profiling.

4.2.1. Design Variants

For each target contraction pattern, three variants are maintained:

1. **Reference variant:** straightforward kernel for correctness and baseline profiling.
2. **Memory-optimised variant:** improved access locality and reduced redundant global loads.
3. **Launch-optimised variant:** fused execution that amortises host-side launch overhead.

Maintaining explicit variants makes it easier to attribute performance deltas and prevents "all-at-once" changes that are hard to analyse.

4.2.2. Correctness and Numerical Checks

Each variant is checked against a trusted baseline (library result or high precision reference) with fixed tolerance settings per precision mode. Any performance result is accepted only after numerical agreement is verified.

4.3. Occupancy and Launch Configuration

Launch configuration is tuned as a constrained resource-allocation problem on each SM. The central trade-off is that aggressive register/shared-memory usage can increase per-thread efficiency while reducing occupancy, which may hurt latency hiding.

4.3.1. Tuning Procedure

The tuning sequence is:

1. choose an initial block size (typically 128 or 256 threads),
2. inspect register use and shared-memory use per block,
3. estimate achievable occupancy,
4. benchmark and profile stall reasons,
5. retune block size and launch bounds based on measured bottlenecks.

If memory stalls dominate, higher occupancy is often preferred. If execution is compute-bound with low dependency stalls, lower occupancy with higher per-thread work can be superior.

4.3.2. Worked Occupancy Example on A100

The two occupancy schematics in Figure 4.1 use one SM with the following capacity model: 2048 threads, 65 536 registers, and 160 kB shared memory. This is the same setup used in the diagrams for visual clarity.

Initial kernel setup:

$$b = 256 \text{ threads per block,} \tag{4.1}$$
$$r_t = 64 \text{ registers per thread,} \tag{4.2}$$
$$s_b = 48\,\text{kB shared memory per block.} \tag{4.3}$$

Per-block resource use:

$$r_b = b \cdot r_t = 256 \cdot 64 = 16384 \text{ registers.} \tag{4.4}$$

This corresponds to:

- thread share per block: $256/2048 = 12.5\%$,
- register share per block: $16384/65536 = 25\%$,
- shared-memory share per block: $48/160 = 30\%$.

The block residency limits are (thread limit N_t, register limit N_r, shared-memory limit N_s):

$$N_t = \left\lfloor \frac{2048}{256} \right\rfloor = 8, \tag{4.5}$$
$$N_r = \left\lfloor \frac{65536}{16384} \right\rfloor = 4, \tag{4.6}$$
$$N_s = \left\lfloor \frac{160}{48} \right\rfloor = 3. \tag{4.7}$$

Therefore

$$N_{\text{res}} = \min(N_t, N_r, N_s) = \min(8, 4, 3) = 3, \tag{4.8}$$

so this case is **shared-memory occupancy limited** (Figure 4.1a).

After reducing shared memory to

$$s_b = 32\,\text{kB}, \tag{4.9}$$

the shared-memory limit becomes

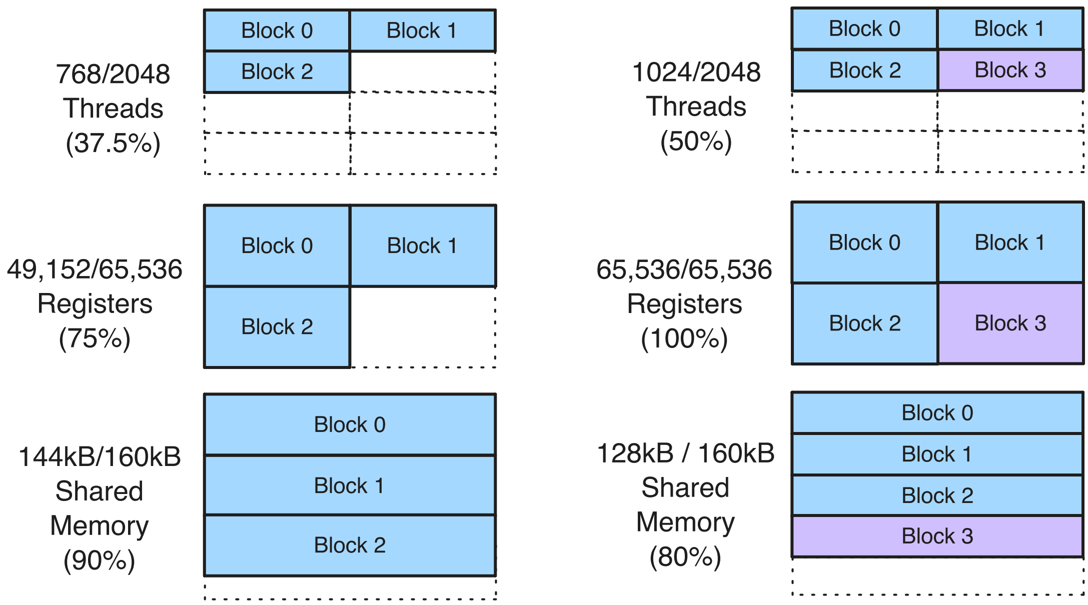
$$N_s = \left\lfloor \frac{160}{32} \right\rfloor = 5. \tag{4.10}$$

Now

$$N_{\text{res}} = \min(N_t, N_r, N_s) = \min(8, 4, 5) = 4, \tag{4.11}$$

so the limiting resource changes to **registers** (Figure 4.1b).

In this transition, resident threads increase from $3 \cdot 256 = 768$ to $4 \cdot 256 = 1024$, i.e. from 37.5 % to 50 % of the SM thread capacity.



(a) Initial setup with 48 kB shared memory per block. Occupancy is limited by shared memory.

(b) After reducing to 32 kB shared memory per block. Occupancy increases by one block, then registers become the occupancy limit.

Figure 4.1.: Occupancy transition example on one A100 SM: from a shared-memory-limited occupancy regime to a register-limited occupancy regime.

4.3.3. Launch Overhead Considerations

For small kernels, launch latency can dominate irrespective of occupancy. Therefore occupancy tuning is combined with launch reduction techniques (kernel fusion or persistent execution) rather than treated in isolation.

4.4. Shared Memory Tiling

Shared-memory tiling is used to raise arithmetic intensity by reusing data that would otherwise be fetched repeatedly from global memory. The design objective is to maximise reuse while staying within per-block shared-memory limits and avoiding bank conflicts.

4.4.1. Tile Design Choices

Tile sizes are selected based on:

- contraction dimensions and divisibility,
- register pressure from accumulator blocking,
- shared-memory footprint per block,
- expected occupancy after resource allocation.

Padding is introduced where needed to avoid systematic bank conflicts in column-wise shared-memory accesses.

4.4.2. Boundary Handling

Real workloads frequently use dimensions that are not multiples of tile sizes. Boundary tiles are handled with predicated loads/stores to preserve correctness without launching separate cleanup kernels.

5. Results

This chapter presents benchmark and profiling results for the optimisation process. It starts with the experimental protocol, then reports performance and scaling behaviour, followed by profiling evidence and baseline comparisons.

5.1. Experimental Setup

All measurements are performed on the A100-based JSC environment documented in Chapter A, with emphasis on reproducibility and fair comparison between custom kernels and vendor-library baselines.

5.1.1. Hardware/Software Baseline

Unless explicitly stated otherwise, experiments use a single node with 4× A100-SXM4-40GB GPUs. Single-GPU measurements are pinned to one device; multi-GPU measurements use NVLink-connected devices within the same node.

The software stack (CUDA toolkit, driver, compiler, profiling tools) is kept fixed across all experiments in a campaign.

5.1.2. Workload Definition

The workload suite includes:

- small to medium GEMM-like contractions representative of target traces,
- a two-site tensor-network contraction-order case study (Section 3.2),
- layout-transform kernels required by contraction workflows,
- fused-kernel variants for launch amortisation studies.

Dimensions are selected to cover both launch-dominated and compute-dominated regions. Problem sets include non-power-of-two sizes to reflect realistic tensor shapes.

5.1.3. Measurement Protocol

Each data point is measured with:

1. warm-up iterations to remove first-launch effects,
2. repeated timed iterations (CUDA events) for stable statistics,
3. Nsight Compute passes with a fixed ten-counter KPI pack for hardware-counter attribution (Table 5.2).

Reported values use median runtime by default; spread (e.g. min/max or percentiles) is included when variance is non-negligible.

5.2. Kernel Launch Overhead Benchmark

As discussed in Section 2.4.5, each CUDA kernel launch incurs a fixed overhead of several microseconds regardless of the kernel's computational workload. For the small GEMM operations typical of tensor network contractions, this overhead can dominate total runtime. The following benchmark quantifies the effect and validates the kernel fusion strategy adopted in this work.

5.2.1. Setup

We compare two strategies for executing 1000 identical $n \times n$ double-precision GEMM operations on a single A100-SXM4-40GB GPU:

1. **Separate launches:** each GEMM is dispatched as an individual cuBLASDx kernel, incurring the full launch overhead 1000 times.

2. **Fused kernel:** a single kernel is launched once and performs all 1000 GEMM operations sequentially using cuBLASDx's device-side API [NVI25a], amortising the launch cost to a single invocation.

Matrix sizes range from $n = 1$ to $n = 32$ (square matrices, $m = n = k$), covering the regime of small GEMMs representative of tensor contractions with low to moderate bond dimensions. All matrices use column-major layout and FP64 precision. Timings are recorded using CUDA events with device synchronisation.

5.2.2. Results

Table 5.1 reports the measured total execution time for both strategies, along with the derived speedup factor and the fraction of the separate-launch time attributable to overhead. Figure 5.1 visualises the same data.

Table 5.1.: Total time for 1000 FP64 GEMMs ($n \times n$) on A100. *Sep.* denotes 1000 kernel launches, *Fuse.* denotes one fused kernel, and *Overhd.* is the launch-overhead fraction of the separate-launch time.

n	Sep. (µs)	Fuse. (µs)	Speedup	Overhd. (%)
1	8022	172	46.7×	97.9
2	7941	185	43.0×	97.7
4	7835	180	43.5×	97.7
8	7895	180	43.8×	97.7
13	8477	355	23.9×	95.8
16	8104	347	23.4×	95.7
23	9217	717	12.9×	92.2
32	9752	1380	7.1×	85.9

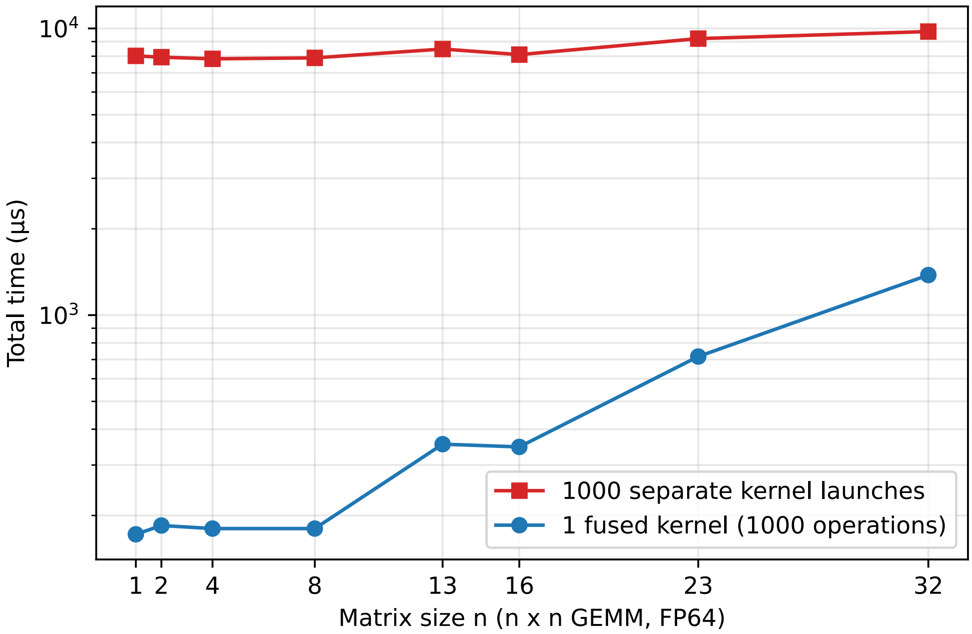


Figure 5.1.: Total time for 1000 GEMM operations as a function of matrix size, comparing 1000 separate kernel launches (red) against a single fused kernel (blue). The near-constant red curve below $n = 16$ confirms that launch overhead dominates over actual computation for small matrices.

5.2.3. Discussion

Two points are clear from the data. First, the total time for separate launches is nearly constant across all matrix sizes, varying only from 8022 μs at $n = 1$ to 9752 μs at $n = 32$. This confirms that the per-launch overhead of approximately 8 μs dominates the total runtime: the actual GEMM computation contributes less than 3% of the measured time for $n \leq 8$. Only at $n = 32$ does the computation begin to represent a meaningful fraction (14%) of the total.

Second, the fused kernel eliminates this overhead almost entirely. At $n = 1$, where the GEMM itself is trivial, fusion yields a 46.7× speedup. As the matrix size increases and the per-GEMM computation grows, the relative benefit of fusion decreases—but even at $n = 32$, the fused kernel remains 7.1× faster.

These results directly motivate the kernel fusion strategy used throughout this work. Tensor network algorithms typically require many contractions of modest size, which places them in the regime where launch overhead dominates. By fusing these contractions into a single kernel using cuBLASDx's device-side GEMM API, the launch cost is paid once rather than once per contraction. The design and implementation of the fused kernels are described in Section 4.2.

5.3. Single-GPU Performance

Single-device performance is reported for the target kernel set, with custom implementations compared against cuBLAS/cuTENSOR baselines.

5.3.1. Metrics

The primary metrics are:

- runtime per operation,
- achieved throughput (GFLOP/s or TFLOP/s),
- effective memory bandwidth,
- speedup relative to the strongest baseline.

5.3.2. Evaluation Dimensions

Results are stratified by:

1. problem size regime (small, medium),
2. precision mode (FP64, FP32, optional TF32),
3. kernel variant (baseline, memory-optimised, fused).

The split separates gains from arithmetic optimisation and gains from reduced launch overhead.

5.4. Scaling Behaviour

The analysis tracks how performance evolves when moving from one GPU to multiple GPUs within a single NVLink-connected node.

5.4.1. Strong and Weak Scaling Views

- **Strong scaling:** fixed total problem size, increasing GPU count.
- **Weak scaling:** per-GPU problem size fixed, total size increases with GPU count.

Both views are needed because kernel-level speedups may not translate linearly once communication and synchronisation costs are included.

5.4.2. Communication Effects

Scaling analysis explicitly separates:

1. pure compute time on each GPU,
2. peer-to-peer transfer time over NVLink,
3. host-side orchestration and synchronisation overhead.

The decomposition identifies when scaling is limited by communication topology rather than kernel efficiency.

ID	Primary counter name (Nsight Compute)
K1	`sm__throughput.avg.pct_of_peak_sustained_elapsed`
K2	`dram__throughput.avg.pct_of_peak_sustained_elapsed`
K3	`lts__throughput.avg.pct_of_peak_sustained_elapsed`
K4	`l1tex__t_sector_hit_rate.pct`
K5	`smsp__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active`
K6	`sm__warps_active.avg.pct_of_peak_sustained_active`
K7	`launch__occupancy_limit_registers`
K8	`smsp__warps_eligible.sum.per_cycle_active`
K9	`smsp__issue_active.avg.pct_of_peak_sustained_active`
K10	`smsp__thread_inst_executed_per_inst_executed.pct`

Table 5.2.: KPI identifier to primary metric mapping used for reporting. Scripts keep compatibility fallbacks for Nsight Compute version differences.

5.5. Profiling Analysis

Nsight Compute data is used to explain *why* performance changes after each optimisation, not just *how much* it changes.

5.5.1. KPI Pack Used for Attribution

To keep profiling interpretable and comparable, all Nsight Compute runs in this thesis use a fixed ten-counter KPI pack. The profiling scripts in the companion repository use exact metric names when available and otherwise fall back to equivalent names for the installed Nsight Compute version.

Interpretation by KPI.

K1 (SM throughput). K1 reports how much total SM work was completed relative to the sustained peak for the device. It is a top-level progress indicator across execution pipelines, so high values mean the SMs are busy doing useful work. Low K1 indicates unused compute capacity, but it does not by itself say whether the cause is memory stalls, low occupancy, or poor issue efficiency.

K2 (DRAM throughput). K2 measures bandwidth at the HBM interface as a percentage of sustained peak. High K2 means the kernel is pushing the DRAM subsystem hard, often indicating a bandwidth-sensitive region. If K2 is high while compute-side counters stay modest, the kernel is usually memory-bound.

K3 (L2 throughput). K3 measures traffic pressure through the L2 cache fabric relative to peak. It helps distinguish kernels limited by on-chip cache traffic from kernels limited in the core pipelines. High K3 with weak compute counters often means data movement inside the memory hierarchy dominates.

K4 (L1 hit rate). K4 is the fraction of L1/texture requests served by L1 instead of missing to lower levels. Higher values usually mean better locality and lower latency per memory access. Low K4 is not automatically bad for streaming kernels with little temporal reuse, so it must be read together with K2/K3 and runtime.

K5 (FP32 FMA pipe active). K5 reports how active the FP32 FMA pipelines are relative to their sustained peak activity. It is the most direct signal for whether FP32 arithmetic hardware is actually being used. High K1 but low K5 usually means time is spent in non-FMA instructions or waiting, not in dense FP32 math.

K6 (warps active). K6 is runtime occupancy: active warps as a percentage of the architectural peak during kernel execution. High K6 means many warps are resident and participating, which improves latency hiding potential. However, high K6 alone does not guarantee speed if those warps are mostly stalled.

K7 (register occupancy limit). K7 captures how strongly register allocation constrains theoretical occupancy. A stronger register limit means fewer blocks/warps can reside on an SM at once, reducing latency-hiding headroom. It should be interpreted with K6 and K8 to separate static resource limits from dynamic scheduler starvation.

K8 (warps eligible per cycle). K8 measures how many warps are ready to issue in each active cycle of an SM sub-partition. High K8 means the scheduler has choices and can keep issue slots fed. Low K8 is a direct starvation signal, typically from dependencies or long-latency memory waits.

K9 (issue active). K9 reports how often issue slots actually dispatch instructions relative to peak sustained activity. High K9 means the front end is productive and instruction delivery is healthy. Low K9 together with low K8 indicates not enough ready warps, while low K9 with high K8 can indicate other front-end bottlenecks.

K10 (thread-inst per inst). K10 approximates average participating threads per issued instruction as a percentage of full-warp execution. Values near 100% indicate mostly uniform control flow, while values around 50% indicate substantial lane under-utilisation from divergence or predication. It is therefore the primary SIMT efficiency counter for branch-heavy kernels.

5.5.2. Interpretation Rules

Attribution follows these rules under identical problem sizes and launch configurations:

1. **Compute-limited pattern:** K1 and K5 are high, while K2/K3 are not dominant.
2. **Memory-limited pattern:** K2 and/or K3 are high, K8/K9 are reduced due to memory waiting, and throughput scales weakly with more FMAs.
3. **Occupancy/resource-limited pattern:** K7 indicates a register limit and K6 remains below expected active-warp levels.
4. **Scheduler starvation pattern:** K8 and K9 drop together, indicating insufficient eligible warps or long-latency dependencies.
5. **Divergence/predication loss pattern:** K10 decreases while kernel time increases under otherwise similar arithmetic work.

Variant	Avg. step time (ms)	TFLOP/s
Bad direct (serial launches)	2.843314	0.020681
Good batched (one launch/step)	0.039640	1.631807

Table 5.3.: Timing summary for the irregular contraction training pair.

5.5.3. Application to Benchmark Pairs

The same KPI pack is applied to the three benchmark pairs used in this chapter:

- **FP32 GEMM pair (cuBLAS vs naive):** expected separation is high K1/K5/K9 for cuBLAS and lower compute-issue efficiency for the naive kernel, with different memory-pressure signatures in K2–K4.

- **Warp-divergence pair (uniform vs divergent):** expected separation is primarily in K10, with secondary reductions in K8/K9 and effective throughput for the divergent variant.

- **TN two-site contraction pair (ordered vs direct):** expected separation is higher K1/K5/K9 for the ordered strided-batched GEMM mapping and lower scheduler efficiency plus heavier memory pressure for the direct contraction-order baseline.

Measured example values. For the FP32 GEMM benchmark pair, the measured counters show: K1 98.45% (cuBLAS) vs 54.32% (naive), K5 98.67% vs 28.05%, K9 56.15% vs 31.50%, with higher memory pressure in the naive kernel (K2 16.54% vs 3.62%). The numbers indicate that the naive kernel is not occupancy-limited (K6 is high) but issue/compute-efficiency-limited.

For the warp-divergence pair, the key discriminator is K10: 100% (warp-uniform) vs 50.35% (forced divergent split), matching the expected half-warp execution efficiency for a lane-based 16/16 branch split.

Using a fixed KPI pack reduces trial-and-error profiler exploration and keeps profile evidence comparable across optimisation steps.

5.5.4. Irregular Contraction Training Run

The first end-to-end irregular pipeline run used `batch` $= 128$, $m = 47$, $n = 84$, and $k = 64$ on one A100. Two variants were compared: `tn_irregular_direct_bad_seq` (serial launch regime) and `tn_irregular_batched_good` (batched launch regime).

Normalising by step time, the batched variant is 71.7× faster, and its throughput is 78.9× higher.

Three points are directly reusable for subsequent kernel work. First, K10 is almost unchanged between direct and batched kernels (86.14% vs 86.01%), so divergence is not the main limiter in this case. Second, K6/K8/K9 move sharply upward in the batched version, which identifies scheduler starvation and underfilled execution as the central issue in the serial-launch regime. Third, the repack kernel itself is not the pathological kernel in this run: its K1/K5/K9 values are much higher than the direct contraction kernel.

Metric	Bad direct kernel	Good batched kernel	Bad repack kernel
K1: SM throughput (%)	1.20	37.37	46.57
K2: DRAM throughput (%)	0.19	3.26	17.55
K5: FP32 FMA pipe active (%)	3.62	23.79	41.72
K6: Warps active (%)	12.34	88.80	77.76
K8: Warps eligible (warp/cycle active)	26.83	329.80	1167.58
K9: Issue active (%)	5.75	32.89	62.02
K10: Thread-inst/inst (%)	86.14	86.01	100.00

Table 5.4.: Selected Nsight Compute counters for the irregular training pair. Direct and batched values are averages over two sampled launches.

Stride	Avg. kernel time (ms)	Effective BW (GB/s)	BW/BW$_{s=1}$
1	0.1078	1245.34	1.000
2	0.1526	879.68	0.706
4	0.2429	552.58	0.444
8	0.4329	310.01	0.249
16	0.8229	163.11	0.131
32	0.8412	159.55	0.128
64	0.6003	223.58	0.180
128	0.2378	564.36	0.453

Table 5.5.: Coalescing sweep results from the synthetic stride microbenchmark on A100 (FP32 loads/stores).

The Nsight Systems summary from the same job is consistent with this view. In the bad path, device-to-host copies total about 10.1 MB across five copy events, while the good path shows only one negligible device-to-host copy. Combined with the timing gap in Table 5.3, this supports a simple rule for this workload class: keep irregular contractions batched on device and avoid host-side synchronisation/copy steps inside the hot loop.

5.5.5. Global-Memory Coalescing Sweep

From stride 1 to stride 32, average kernel time increases from 0.108 ms to 0.841 ms (7.8×), while the effective bandwidth proxy drops from 1245 to 160 GB/s (0.13×). This is the expected signature of reduced coalescing: lanes in a warp map to more memory sectors, increasing memory transactions per useful word. For tensor network kernels this result shows that operand layouts in critical kernels should preserve unit-stride accesses for warp-contiguous dimensions whenever possible.

The partial recovery at stride 64 and 128 is treated as a benchmark artifact from the masked synthetic index mapping (`idx = (tid * stride) & (N-1)`), which introduces aliasing/reuse at large power-of-two strides. Therefore, interpretation is based primarily on the monotonic degradation regime up to stride 32.

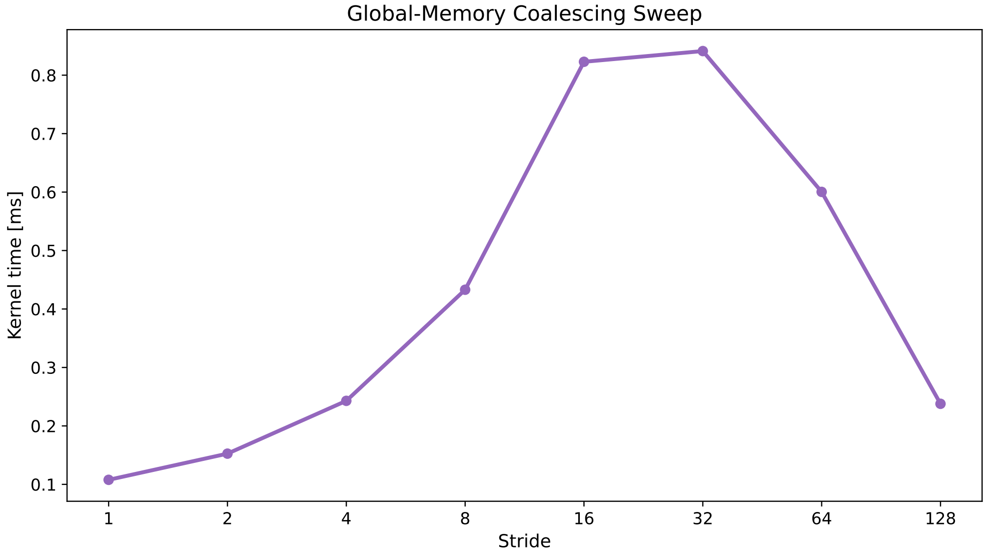


Figure 5.2.: Global-memory coalescing sweep over stride.

Variant	Avg. kernel time (ms)	Core effective TFLOP/s	Theoretical occupancy (%)
Low register pressure	0.5942	14.46	100.0
High register pressure	0.6710	12.80	62.5

Table 5.6.: Register-pressure microbenchmark results on A100 (FP32 arithmetic core work).

5.5.6. Register Pressure and Occupancy

Increasing live register state reduces theoretical occupancy from 100.0% to 62.5%. Runtime rises by about 12.9%, and core effective throughput drops from 14.46 to 12.80 TFLOP/s (about 11.5%). This indicates that the high-register variant exposes less latency hiding due to fewer concurrently resident warps per SM. For contraction kernels, this supports limiting accumulator blocking and aggressive unrolling when they push register usage past the occupancy-efficient range.

5.6. Comparison with cuBLAS and cuTENSOR

Optimised kernels are compared with vendor-library baselines under matched precision and workload definitions, with aligned layout assumptions.

5.6.1. Comparison Protocol

For each workload class:

1. run cuBLAS/cuTENSOR baseline,
2. run custom unfused variant,
3. run custom fused/layout-aware variant,

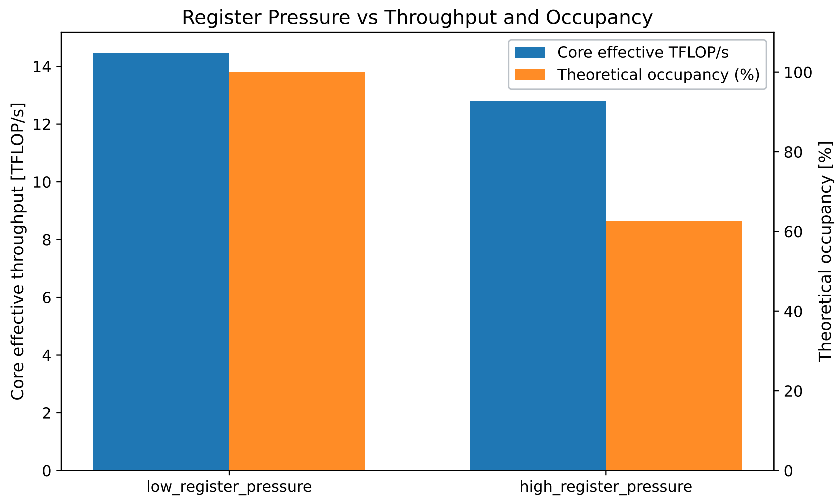


Figure 5.3.: Register pressure versus throughput and occupancy.

4. report speedups and counter-based explanation.

The strongest baseline per workload is used as denominator for reported speedup to avoid optimistic bias.

5.6.2. Interpretation Strategy

Outperformance is interpreted by regime:

- if custom kernels win in small sizes, launch reduction is the likely mechanism,
- if wins appear in medium sizes with high bandwidth utilisation, memory access improvements are the likely mechanism,
- if libraries remain dominant at larger sizes, this indicates the practical limit of workload-specific optimisation.

5.7. Energy Efficiency Considerations

Performance is the primary optimisation target, but energy-to-solution is also an operational objective in HPC: cluster power budgets are finite, and lower joules per job improve both operating cost and system throughput under queue load [KPB+23, TOP25]. For this reason, the same optimisation decisions used for speed (contraction order, coalescing, launch reduction, occupancy control) are evaluated here from an energy perspective as well.

5.7.1. Why Energy Matters

The target workload class (tensor-network-style contractions) executes many short and medium kernels in iterative loops. In this regime, wasted launches, redundant arithmetic, and avoidable memory traffic increase both runtime and board energy. On A100-class GPUs, where arithmetic throughput is high and kernel orchestration is often the bottleneck for moderate sizes, runtime improvements frequently translate directly into energy improvements at fixed clock/power settings [NVI20a, NVI25b].

Scope note: the measurements and models in this section focus on GPU board energy for relative comparison between algorithmic variants on the same node. Whole-facility effects (cooling plant, PUE, rack-level sharing) are out of scope.

5.7.2. Metrics

For one benchmark run of duration T, instantaneous board power $P(t)$, and total floating-point work F, the core quantities are

$$E = \int_0^T P(t)\,\mathrm{d}t \approx \bar{P}\,T, \qquad \eta_{\mathrm{F}} = \frac{F}{E}, \qquad \eta_{\mathrm{T}} = \frac{\text{time-to-solution}^{-1}}{\bar{P}},$$

where E is energy-to-solution (J), η_{F} is computational energy efficiency (FLOP/J), and η_{T} is throughput per watt for a fixed workload.

In addition, a standard trade-off metric is energy-delay product (EDP):

$$\mathrm{EDP} = E \cdot T,$$

and, when latency is weighted more strongly, energy-delay-squared product:

$$\mathrm{ED^2P} = E \cdot T^2.$$

These metrics are useful when comparing alternatives that exchange small runtime gains for larger power increases (or vice versa) [HTHW16].

For memory-intensive kernels with transferred bytes Q, a useful auxiliary metric is

$$\eta_{\mathrm{B}} = \frac{Q}{E} \quad [\text{bytes/J}],$$

which helps identify memory-system inefficiency.

5.7.3. First-Order Model for Contraction Kernels

A practical starting point is

$$E = \int_0^T P(t)\,\mathrm{d}t = P_{\mathrm{base}}\,T + \int_0^T P_{\mathrm{dyn}}(t)\,\mathrm{d}t.$$

Approximating the dynamic term by compute- and data-movement work yields

$$E \approx P_{\mathrm{base}}\,T + e_{\mathrm{F}}\,F + e_{\mathrm{B}}\,Q,$$

with:

- P_{base}: baseline board power while the GPU is active,
- e_{F}: effective energy per floating-point operation,
- e_{B}: effective energy per byte moved through memory.

The model is intentionally coarse, but it is sufficient for optimisation decisions and consistent with the runtime decomposition used in Section 2.2 [WWP09, HJ88]. It captures the central trade-off: optimisations that reduce runtime T, redundant work F, or memory traffic Q usually improve energy efficiency simultaneously.

5.7.4. Implications for the Current Kernel Set

The profiling results already support concrete energy-efficiency expectations:

1. **Contraction-order improvement** (TN case study) reduces arithmetic work from $\mathcal{O}(Bd_p^2\chi^2)$ to $\mathcal{O}(B(d_p^2\chi + d_p\chi^2))$, so both time-to-solution and expected energy-to-solution decrease.
2. **Coalescing improvements** reduce excess memory transactions, improving both runtime and bytes/J in memory-sensitive regimes.
3. **Launch overhead reduction** is energy-relevant: many tiny launches spend time at non-trivial board power without proportional progress.
4. **Register-pressure control** avoids occupancy collapse that prolongs runtime and therefore increases $P_{\text{base}}T$.

The same mechanisms that improve throughput in Section 5.5 and Tables 5.5 and 5.6 also dominate energy-to-solution.

5.7.5. Measurement Protocol on A100 Nodes

On the JSC platform used here, practical energy measurement can be added with minimal workflow change:

1. sample board power during each run via `nvidia-smi` (`power.draw`) telemetry,
2. align measurement start/stop with explicit CUDA synchronisation points so the integration window matches kernel execution,
3. integrate sampled power over wall-clock time (trapezoidal rule) to estimate E,
4. repeat runs and report robust summaries (median and spread) together with runtime and Nsight KPIs.

With samples $(t_i, P_i)_{i=1}^m$, the numerical estimate is

$$E \approx \sum_{i=1}^{m-1} \frac{P_i + P_{i+1}}{2}\,(t_{i+1} - t_i).$$

Representative telemetry fields and command syntax are documented in the `nvidia-smi` documentation [NVI25g]. However, recent analysis of NVIDIA's built-in power sensor shows important caveats for short kernel windows: sampling is asynchronous to kernel boundaries, short transients can be under-resolved, and naive polling can bias integrated energy if the window is too short [YAA24b, YAA24a].

Accordingly, the measurement protocol uses the following safeguards:

1. **Windowing for observability.** Integrate energy over repeated benchmark windows (many kernel iterations) rather than single tiny kernels, so each measurement covers multiple telemetry updates.
2. **Strict boundary control.** Place `cudaDeviceSynchronize` at measurement boundaries and timestamp only after synchronisation to avoid host/device skew.

Table 5.7.: Current measured subset from the SGEMM value-pattern sweep on A100 (active GPU, normalized to 120 s for comparability across runs).

Metric	all-zero	zero/2	zero/3	zero/4
Active window (s)	109.3	122.8	123.2	123.0
Mean power (W)	223.9	308.4	365.0	337.6
Norm. energy at 120 s (kJ)	26.86	37.00	43.80	40.51
Relative energy	1.00	1.38	1.63	1.51

3. **Warm-up and stationarity.** Exclude warm-up iterations and run only after clocks/thermals stabilise to reduce drift.

4. **Uncertainty reporting.** Report median and spread over repeated runs with a fixed protocol, consistent with rigorous benchmarking practice [KJ13].

5. **Cross-check subset.** When possible, validate a subset of runs against an independent measurement path (for example node-level meter or alternative telemetry stack) before drawing absolute-power conclusions.

With these constraints, `nvidia-smi`-based integration is appropriate for *relative* algorithmic comparisons in this work. Absolute board energy values should still be interpreted conservatively, and whole-node/facility energy remains out of scope.

5.7.6. TODO: Fill in — GEMM Data-Pattern Switching Experiment

To test whether data values alone can measurably change GPU board power at fixed kernel structure, a controlled SGEMM sweep is added in `phenomenon_gemm_value_switching.cu`. The kernel shape (M, N, K), launch configuration, and software stack are kept constant; only the matrix initialization changes:

1. all zeros,

2. periodic zeros (every 2nd/3rd/4th/5th element),

3. normal-distributed values,

4. uniform-distributed values.

The associated SLURM runner `09_gemm_value_switching_energy.slurm` captures per-mode runtime CSV files and per-mode `nvidia-smi` power traces, so energy can be integrated mode-by-mode with the protocol above. Industry talks regularly motivate this with switching-activity arguments ($\alpha CV^2 f$ intuition), but this experiment is structured to check the effect empirically for the specific kernels and node setup used in this thesis [Jon22].

Mode labels `zero/`n denote periodic-zero initialization where every n-th element is set to zero.

With the currently imported modes, throughput remains near FP32 peak while board power separates strongly by input pattern. Relative to `all_zero`, `zero_every_2` increases normalized energy by about 38%, `zero_every_4` by about 51%, and `zero_every_3` by about 63%. This indicates a clear value-pattern energy effect without a comparable throughput shift in the same runs.

Expected outcome from prior work:

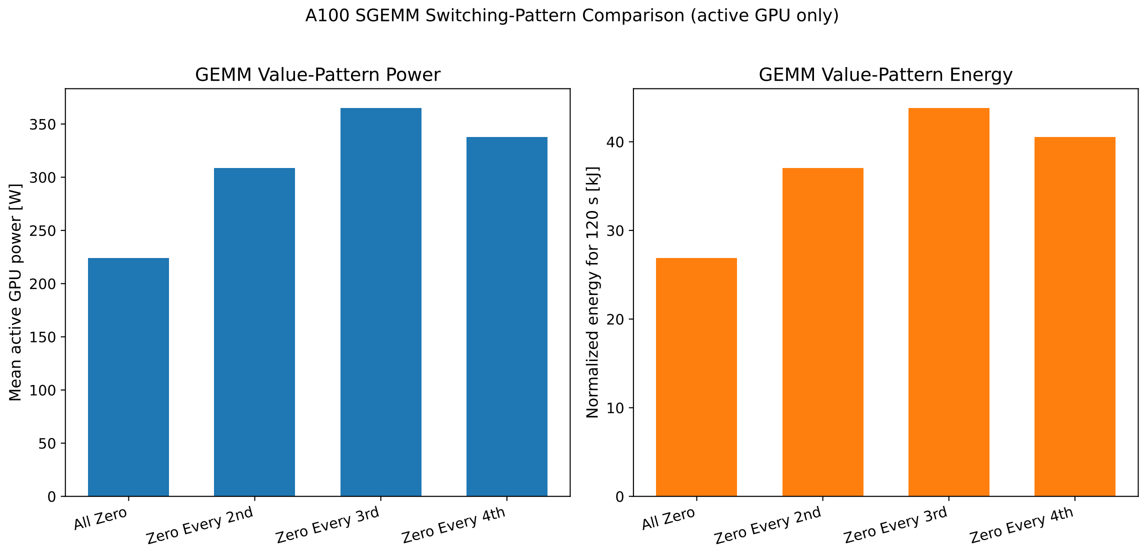


Figure 5.4.: Measured SGEMM value-pattern sweep on A100 (active GPU only, currently imported subset of modes). Left: mean active board power. Right: energy normalized to a 120 s active window.

- Data-dependent board-power variation has already been reported on recent NVIDIA GPUs; measured spreads depend on kernel type and intensity, with larger effects for low-intensity and memory-sensitive kernels (reported average variation around 6% with worst cases near 38%) [HACA24].
- GPU-memory power also depends on value patterns due to I/O signaling effects (including bus-inversion behavior), so identical access counts can still yield different memory energy [SKL19].
- In this SGEMM setup, the arithmetic path is dense and regular, so the conservative expectation is a smaller but still measurable separation between highly sparse-value patterns and fully random-value patterns; runtime should stay close while energy shifts primarily through average board power.

The intent of this experiment is comparative, not absolute: if the measured spread is small, it still defines a practical upper bound on value-pattern effects for the contraction kernels studied here.

5.8. Discussion

This section combines kernel-level and system-level findings into practical guidance for the next optimisation cycle.

5.8.1. What Improved and Why

Beyond raw speedup, the key question is which mechanism caused the improvement (launch reduction, memory locality, occupancy tuning, or precision-path choice). Profiling evidence is used to support each claim, using the fixed KPI pack defined in Table 5.2 so that attributions remain comparable across kernels and optimisation stages.

5.8.2. How Well Results Transfer

Improvements are classified as:

- **likely to carry over** (expected to transfer across Ampere-like systems),
- **topology dependent** (depends on node NUMA/NVLink structure),
- **workload dependent** (effective only for the studied contraction distribution).

5.8.3. Microbenchmark Implications for Tensor Networks

The two most useful profiling microbenchmarks are the coalescing sweep (Table 5.5 and Figure 5.2) and the register pressure study (Table 5.6 and Figure 5.3).

For coalescing, the stride-1 to stride-32 degradation quantifies how quickly memory efficiency collapses when warp lanes lose contiguous access patterns. The result supports prioritising layout transforms and contraction ordering that keep the hottest contracted index unit-stride in memory.

For register pressure, the occupancy drop from 100% to 62.5% with an accompanying throughput loss shows that "more in-register work" is not monotonically beneficial. In practice, this motivates tuning unroll depth and accumulator count jointly with occupancy targets rather than maximising instruction-level reuse in isolation.

The irregular training pair in Tables 5.3 and 5.4 adds one immediately usable rule for upcoming kernels with skewed shapes (e.g. 47×84 blocks): prioritise launch structure and host/device orchestration before low-level arithmetic tuning. In that run, moving from serial per-batch launches to one batched launch per step gave a $71.7\times$ step-time reduction while K10 stayed nearly unchanged, which indicates that scheduler feed and launch/copy structure, not divergence, dominated performance.

6. Conclusion

6.1. Summary

This work examines tensor-network workloads from an HPC perspective. It identifies the dominant bottlenecks on A100-class GPUs, applies profile-guided optimisations, and measures performance against vendor-library baselines. The main control variables are launch overhead, memory-hierarchy efficiency, and occupancy-resource trade-offs.

6.2. Limitations

The current scope has three deliberate limitations:

- physics-specific algorithm derivations and notation are out of scope,
- optimisation focus is on Ampere-era kernels and one-node execution,
- coverage of workload diversity is bounded by available benchmark traces.

6.3. Future Work

Natural continuation points are:

1. extending the same methodology to Hopper/H100-class hardware while preserving comparability with A100 results,
2. deeper multi-GPU overlap strategies (communication/computation pipelining),
3. automated kernel-configuration search constrained by profiler-derived bottleneck models.

Bibliography

[A$^+$23] Thomas Ayral et al. Density matrix renormalization group algorithm for simulating quantum circuits with a finite fidelity. *PRX Quantum*, 4(3):030307, 2023. `https://arxiv.org/abs/2302.01941`.

[Adv20] Advanced Micro Devices (AMD). *RDBMS Tuning Guide for AMD EPYC 7002 Series Processors*, 2020. `https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/tuning-guides/amd-epyc-7002-database-on-linux-tuning-guide.pdf`.

[Amd67] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, pages 483–485. ACM, 1967.

[Boe22] Simon Boehm. How to optimize a cuda matmul kernel for cublas-like performance: a worklog, 2022. `https://siboehm.com/articles/22/CUDA-MMM`.

[DR22] Wolfgang Dahmen and Arnold Reusken. *Numerik für Ingenieure und Naturwissenschaftler*. Springer Spektrum, 3 edition, 2022.

[EHH22] Stijn Eyerman, Wim Heirman, and Ibrahim Hur. DRAM bandwidth and latency stacks: Visualizing DRAM bottlenecks. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 322–331, 2022.

[ET94] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

[FHL$^+$23] Massimiliano Fasi, Nicholas J. Higham, Florent Lopez, Theo Mary, and Mantas Mikaitis. Matrix multiplication in multiword arithmetic: Error analysis and application to GPU tensor cores. *SIAM Journal on Scientific Computing*, 45(1):C1–C19, 2023.

[Fru20] Andrei Frumusanu. TSMC details its N5 process technology: Performance, density, and defect improvements, 2020. `https://www.anandtech.com/show/16028/tsmc-details-n5`.

[FW86] Philip J. Fleming and John J. Wallace. How not to lie with statistics: The correct way to summarize benchmark results. *Communications of the ACM*, 29(3):218–221, 1986.

[Gus88] John L. Gustafson. Reevaluating Amdahl's law. *Communications of the ACM*, 31(5):532–533, 1988.

[GWMW20] Elijah Gilman, Samuel Walls, Alexander Merritt, and Bryan C. Ward. Demystifying the placement policies of the NVIDIA GPU thread block scheduler. In *Proceedings of the International Conference on Real-Time Networks and Systems (RTNS)*. ACM, 2020. https://cake.wpi.edu/assets/papers/gilman20_performance.pdf.

[HACA24] Nour Hammouda, Karel Adamek, Nikolay I. Chankov, and Wesley Armour. Input-dependent power usage in gpus, 2024. Workshop paper. https://s3hpc.github.io/files/workshop/2024/papers/s3hpc24-paper11.pdf.

[Hig02] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2 edition, 2002.

[HJ88] R. W. Hockney and C. R. Jesshope. *Parallel Computers 2: Architecture, Programming and Algorithms*. Adam Hilger, 1988.

[HM22] Nicholas J. Higham and Theo Mary. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*, 31:347–414, 2022.

[HR09] Peter J. Huber and Elvezio M. Ronchetti. *Robust Statistics*. Wiley, 2 edition, 2009.

[HTDH18] Azzam Haidar, Stanimire Tomov, Jack Dongarra, and Nicholas J. Higham. Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 603–613. IEEE, 2018.

[HTHW16] Georg Hager, Jan Treibig, Johannes Habich, and Gerhard Wellein. Exploring performance and power properties of modern multicore chips via simple machine models. *Concurrency and Computation: Practice and Experience*, 28(2):189–210, 2016.

[IEE19] IEEE. IEEE standard for floating-point arithmetic, 2019.

[Jai91] Raj Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.

[JCL+17] Hongshin Jun, Jinhee Cho, Kangseol Lee, Ho-Young Son, Kwiwook Kim, Hanho Jin, and Keith Kim. HBM (High Bandwidth Memory) DRAM technology and architecture. In *2017 IEEE International Memory Workshop (IMW)*, pages 1–4, 2017.

[JED13] JEDEC Solid State Technology Association. High bandwidth memory (HBM) DRAM, JESD235. Technical report, 2013.

[JED16] JEDEC Solid State Technology Association. High bandwidth memory (HBM) DRAM, JESD235A. Technical report, 2016. Defines HBM2 specification.

[JED20] JEDEC Solid State Technology Association. High bandwidth memory (HBM) DRAM, JESD235C. Technical report, 2020. Defines HBM, HBM2, and HBM2E specifications.

[JLM17] Claude-Pierre Jeannerod, Nicolas Louvet, and Jean-Michel Muller. Further analysis of kahan's algorithm for the accurate computation of 2×2 determinants. *Mathematics of Computation*, 86(307):2245–2264, 2017.

[Jon22] Stephen Jones. GTC 2022: How CUDA programming works, 2022. NVIDIA GTC session video. `https://www.youtube.com/watch?v=QQceTDjA4f4`.

[Jue26] Juelich Supercomputing Centre (JSC). JURECA Data Centric (DC): Configuration, 2026. `https://apps.fz-juelich.de/jsc/hps/jureca/configuration.html` (accessed 2026-02-22).

[KJ13] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management (ISMM)*, pages 63–74. ACM, 2013.

[KMM+19] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. A study of BFLOAT16 for deep learning training. In *arXiv preprint arXiv:1905.12322*, 2019.

[KPB+23] Jan Koćot, Dmitrii Pogodaev, Damian Bajer, Miroslaw Lisowski, Marek Gorgon, and Jacek Leszczynski. Energy-aware computing and practical measures for energy efficiency: A comprehensive survey. *Energies*, 16(2):890, 2023.

[KRW25] Aleksei V. Krinitsin, Matteo Rizzi, and Xavier Waintal. Time evolution of the quantum ising model in two dimensions using tree tensor networks. *arXiv preprint arXiv:2505.07612*, 2025. `https://arxiv.org/abs/2505.07612`.

[LRJ18] Shang Li, Dhiraj Reddy, and Bruce Jacob. A performance & power comparison of modern high-speed DRAM architectures. In *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, 2018. `https://doi.org/10.1145/3240302.3240315`.

[MCL+18] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. NVIDIA tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 522–531. IEEE, 2018.

[MDHS09] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276. ACM, 2009.

[Mel24] Chris Mellor. Intel says TSMC 2nm defect density better than 3nm, 5nm, and 7nm at same stage, 2024. https://www.tomshardware.com/tech-industry/intel-says-tsmc-2nm-defect-density-better-than-3nm-5nm-and-7nm-at-same-stage.

[Mic21] Micron Technology. Integrating and operating HBM2E memory, 2021. Technical Brief. https://media-www.micron.com/-/media/client/global/documents/products/technical-marketing-brief/micron-hbm2e-memory-wp.pdf.

[ML+23a] Tamas Menczer, Ors Legeza, et al. Boosting the effective performance of massively parallel tensor network state algorithms on hybrid CPU-GPU based architectures via non-Abelian symmetries. *arXiv preprint arXiv:2309.16724*, 2023. https://arxiv.org/abs/2309.16724.

[ML+23b] Tamas Menczer, Ors Legeza, et al. Massively parallel tensor network state algorithms on hybrid CPU-GPU based architectures. *arXiv preprint arXiv:2305.05581*, 2023. https://arxiv.org/abs/2305.05581.

[ML24a] Andor Menczer and Örs Legeza. Tensor network state algorithms on AI accelerators. *Journal of Chemical Theory and Computation*, 20(20):8897–8910, 2024. https://doi.org/10.1021/acs.jctc.4c00800.

[ML+24b] Tamas Menczer, Ors Legeza, et al. Parallel implementation of the density matrix renormalization group method achieving a quarter petaflops performance on a single DGX-H100 GPU node. *arXiv preprint arXiv:2407.07411*, 2024. https://arxiv.org/abs/2407.07411.

[ML25] Andor Menczer and Örs Legeza. Massively parallel tensor network state algorithms on hybrid CPU-GPU based architectures. *Journal of Chemical Theory and Computation*, 21(4):1572–1587, 2025. https://doi.org/10.1021/acs.jctc.4c00661.

[NVI17] NVIDIA Corporation. *NVIDIA Tesla V100 GPU Architecture*, 2017. Whitepaper, WP-08608-001_v1.1.

[NVI20a] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*, 2020. Whitepaper v1.0.

[NVI20b] NVIDIA Developer Blog. NVIDIA ampere architecture in-depth, 2020. https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/.

[NVI22] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture*, 2022. Whitepaper.

[NVI24a] NVIDIA Corporation. cuBLAS library, 2024. https://developer.nvidia.com/cublas.

[NVI24b] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2024. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[NVI24c] NVIDIA Corporation. cuTENSOR: A high-performance tensor primitives library, 2024. https://developer.nvidia.com/cutensor.

[NVI25a] NVIDIA Corporation. cuBLASDx: Device side BLAS extensions, 2025. https://docs.nvidia.com/cuda/cublasdx/.

[NVI25b] NVIDIA Corporation. *CUDA C++ Best Practices Guide*, 2025. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/.

[NVI25c] NVIDIA Corporation. NVIDIA a100 tensor core GPU: Product specifications, 2025. https://www.nvidia.com/en-us/data-center/a100/.

[NVI25d] NVIDIA Corporation. *NVIDIA Ampere GPU Architecture Tuning Guide*, 2025. https://docs.nvidia.com/cuda/ampere-tuning-guide/index.html.

[NVI25e] NVIDIA Corporation. *NVIDIA Nsight Compute Profiling Guide*, 2025. https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html.

[NVI25f] NVIDIA Corporation. *NVIDIA Nsight Systems User Guide*, 2025. https://docs.nvidia.com/nsight-systems/UserGuide/index.html.

[NVI25g] NVIDIA Corporation. *NVIDIA System Management Interface (nvidia-smi)*, 2025. https://docs.nvidia.com/deploy/nvidia-smi/index.html.

[Oru14] Roman Orus. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, 2014.

[Pur24] Budirijanto Purnomo. Understanding the visualization of overhead and latency in NVIDIA Nsight systems. NVIDIA Technical Blog, 2024. https://developer.nvidia.com/blog/understanding-the-visualization-of-overhead-and-latency-in-nsight-systems/.

[RH20] Ronny Ramirez and William Hsu. Optimizing applications for NVIDIA Ampere GPU architecture. GTC 2020, Session S21819, 2020. https://developer.download.nvidia.com/video/gputechconf/gtc/2020/presentations/s21819-optimizing-applications-for-nvidia-ampere-gpu-architecture.pdf.

[Sch11] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.

[SKL19] Seonghwan Song, Minsoo Kang, and Sukhan Lee. MEMPower: A data-aware GPU memory power model. *IEEE Access*, 7:152462–152475, 2019. https://doi.org/10.1109/ACCESS.2019.2945732.

[SPE21] SPEC Open Systems Group. *SPEC CPU2017 Run and Reporting Rules*, 2021. https://www.spec.org/cpu2017/Docs/runrules.html.

[SSK17] Paul Springer, Tong Su, and Tamara G. Kolda. Tensor contractions with extended BLAS kernels on CPU and GPU. In *IEEE 24th International Conference on High Performance Computing (HiPC)*. IEEE, 2017. https://research.nvidia.com/sites/default/files/pubs/2017-10_Tensor-Contractions-with/tensors_hipc.pdf.

[TOP25] TOP500 Project. The Green500 List, 2025. https://www.top500.org/lists/green500/.

[WDADN22] Xinzhe Wu, Davor Davidović, Sebastian Achilles, and Edoardo Di Napoli. ChASE: a distributed hybrid CPU-GPU eigensolver for large-scale hermitian eigenvalue problems. In *Proceedings of the Platform for Advanced Scientific Computing Conference (PASC '22)*, pages 1–12. ACM, 2022.

[WDN23] Xinzhe Wu and Edoardo Di Napoli. Advancing the distributed multi-GPU ChASE library through algorithm optimization and NCCL library. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W '23)*, pages 1688–1696. ACM, 2023.

[Whi92] Steven R. White. Density matrix formulation for quantum renormalization groups. *Physical Review Letters*, 69(19):2863–2866, 1992.

[Whi93] Steven R. White. Density-matrix algorithms for quantum renormalization groups. *Physical Review B*, 48(14):10345–10356, 1993.

[WM95] Wm. A. Wulf and Sally A. McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24, 1995.

[WSDN19] Jan Winkelmann, Paul Springer, and Edoardo Di Napoli. ChASE: Chebyshev accelerated subspace iteration eigensolver for sequences of Hermitian eigenvalue problems. *ACM Transactions on Mathematical Software*, 45(2):1–34, 2019.

[Wu19] Xinzhe Wu. *Contribution to the Emergence of New Intelligent Parallel and Distributed Methods Using a Multi-level Programming Paradigm for Extreme Computing*. PhD thesis, University of Lille, 2019.

[WWP09] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[YAA24a] Zeyu Yang, Karel Adamek, and Wesley Armour. Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU's built-in power sensor. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–17. IEEE, 2024. https://doi.org/10.1109/SC41406.2024.00028.

[YAA24b] Zeyu Yang, Karel Adamek, and Wesley Armour. Part-time power measurements: nvidia-smi's lack of attention, 2024. https://arxiv.org/abs/2312.02741.

[ZSW20] Shangnan Zhou, E. Miles Stoudenmire, and Xavier Waintal. What limits the simulation of quantum computers? *Physical Review X*, 10(4):041038, 2020. `https://arxiv.org/abs/2002.07730`.

A. Experimental Environment and Reproducibility

All reported experiments were conducted on a single compute node of the JURECA-DC cluster at the Jülich Supercomputing Centre (JSC). The hardware and software configuration is listed below for reproducibility.

A.1. Hardware Configuration

Table A.1.: Compute node hardware specification.

Component	Specification
GPUs	4× NVIDIA A100-SXM4-40GB
GPU memory per device	40 GB HBM2
GPU interconnect	NVLink 3.0, 4 links per GPU pair
NVLink bandwidth per link	25 GB/s (uni-directional)
PCIe	Gen 4 ×16
CPUs	2× AMD EPYC 7742, 64 cores per socket
CPU threads (total)	256 (SMT-2)
NUMA domains	8
Host memory	503 GiB DDR4
Network	2× Mellanox ConnectX (mlx5) HDR InfiniBand (200 Gbit/s)

The four GPUs are fully connected via NVLink 3.0 with four links between every pair (NV4 topology), providing 100 GB/s of uni-directional bandwidth between any two devices. Each GPU is associated with a distinct NUMA domain; GPU-affine memory allocation was used where applicable.

A.2. Software Environment

Table A.2.: Software versions used for all experiments. Components marked with † are loaded via the JSC module system.

Software	Version
Operating system	Rocky Linux 9.7 (Blue Onyx)
Linux kernel	5.14.0-611.16.1.el9_7.x86_64
NVIDIA driver	590.48.01
CUDA Toolkit†	12.6.20
GCC†	13.3.0
CMake†	3.29.3
C++ standard	C++20
Python	3.9.25
NVIDIA Libraries	
MATHDx†	25.06.0
cuBLASDx	0.4.0 (bundled with MATHDx)
CUTLASS	3.9.0 (bundled with MATHDx)
cuBLAS	Bundled with CUDA Toolkit 12.6
Profiling Tools	
Nsight Compute (ncu)†	2024.3.0.0 (build 34567288)
Nsight Systems (nsys)†	TBD

The NVIDIA driver (590.48.01) reports forward-compatibility with CUDA 13.1; however, all code was compiled against the CUDA 12.6 toolkit. CUTLASS and cuBLASDx require compute capability 8.0 (Ampere) or higher; all kernels were compiled with `-DCMAKE_CUDA_ARCHITECTURES=80`.

A.3. GPU Topology

Table A.3 summarises the interconnect topology as reported by `nvidia-smi topo -m`. All GPU pairs are connected via NV4 (four bonded NVLinks). The two network interfaces (mlx5_0, mlx5_1) are each PCIe-local to one GPU (GPU 1 and GPU 2 respectively), which is relevant for multi-node communication patterns.

Table A.3.: GPU interconnect topology. NV4 = four bonded NVLinks; PIX = single PCIe bridge; SYS = traverses PCIe and inter-NUMA interconnect.

	GPU 0	GPU 1	GPU 2	GPU 3	CPU affinity	NUMA
GPU 0	—	NV4	NV4	NV4	48–63, 176–191	3
GPU 1	NV4	—	NV4	NV4	16–31, 144–159	1
GPU 2	NV4	NV4	—	NV4	112–127, 240–255	7
GPU 3	NV4	NV4	NV4	—	80–95, 208–223	5

A.4. Build and Run Procedure

The project uses CMake as its build system. A minimal build on JURECA-DC proceeds as follows:

```
# Load required modules (JURECA-DC, Stages/2025)
module load Stages/2025
module load CUDA/12.6 GCC/13.3.0 CMake/3.29.3 MATHDx/25.06.0

git clone https://github.com/<your-repo>.git && cd <your-repo>
mkdir build && cd build
cmake .. -DCMAKE_CUDA_ARCHITECTURES=80 -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
```

Listing A.1: Build commands for JURECA-DC.

A.5. Measurement Methodology

A.6. Data Availability

B. Supplementary Benchmarks

TODO

C. Mathematical and Numerical Details

This appendix collects derivations, worked examples, and statistical definitions that support the main text but are not required for the core optimisation narrative. They are gathered here so that the background chapter can focus on concepts and key results.

C.1. Parallel Scaling Formulas

Given runtime T_p on p processing units, speedup and parallel efficiency are

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p}.$$

For strong scaling with serial fraction f_s, Amdahl's bound is

$$S_p \leq \frac{1}{f_s + \frac{1-f_s}{p}}.$$

For weak scaling with scaled problem size, Gustafson's relation is

$$S_p \approx p - f_s(p-1).$$

These bounds are used to interpret node-level scaling results in Section 5.4 [Amd67, Gus88].

C.2. Timing Statistics

For repeated runtime samples $\{t_i\}_{i=1}^{n}$, the sample median is the primary point estimate

$$\tilde{t} = \mathrm{median}(t_i)$$

because it is robust to occasional outliers from scheduler and system noise. This is preferable to the arithmetic mean when runtime distributions are skewed or contain sporadic long-tail events from OS, driver, or queue interference [Jai91, KJ13].

Dispersion is tracked with the median absolute deviation (MAD):

$$\mathrm{MAD} = \mathrm{median}\Big(|t_i - \tilde{t}|\Big),$$

and a robust spread estimate

$$\hat{\sigma}_{\mathrm{rob}} \approx 1.4826 \cdot \mathrm{MAD}.$$

The scaling factor makes $\hat{\sigma}_{\mathrm{rob}}$ comparable to a standard deviation under a normal reference model, while retaining robustness under heavy-tailed noise [HR09].

Uncertainty of the median is reported with a nonparametric bootstrap confidence interval [ET94]. The procedure is:

1. Draw B resamples (typically $B = 1000$) from the original n timing samples, each resample obtained by sampling n values uniformly at random *with replacement*.
2. Compute the median of each resample, producing B bootstrap medians $\tilde{t}_1^*, \tilde{t}_2^*, \ldots, \tilde{t}_B^*$.
3. Sort the bootstrap medians and take the 2.5th and 97.5th percentiles as the lower and upper bounds of the 95% confidence interval.

This avoids any normality assumption on the runtime distribution, which is important because GPU timing samples are frequently skewed by OS jitter and scheduling noise.

For stability reporting, we also track the robust relative spread

$$\mathrm{RRS} = \frac{\hat{\sigma}_{\mathrm{rob}}}{\tilde{t}},$$

which expresses dispersion as a fraction of the central estimate. When RRS exceeds a threshold (typically 1–2%), we increase the repetition count before drawing conclusions. This keeps timing results stable even in short benchmark campaigns.

C.3. Floating-Point Worked Examples

The following step-by-step examples illustrate the encoding rules, absorption, and truncation effects described in Section 2.3.3. They are included for reference and can be skipped by readers familiar with IEEE 754 arithmetic.

C.3.1. Encoding a Decimal Number in FP16

The decimal number 0.15625_{10} has the binary representation 0.00101_2 because

$$0.15625 = \frac{1}{8} + \frac{1}{32} = 0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 0 \times 2^{-4} + 1 \times 2^{-5}$$

holds. To express this as an FP16 number we write it in normalised form,

$$0.00101_2 = 1.01_2 \times 2^{-3},$$

and match it against Equation (2.1): the sign is $s = 0$ (positive), the true exponent is -3, and the stored exponent is $e = -3 + 15 = 12 = 01100_2$. The fraction field stores the bits after the implicit leading 1, i.e. 01 padded with zeros to 10 bits. The complete bit representation is therefore

$$\underbrace{0}_{\text{sign}} \mid \underbrace{01100}_{\text{exp}} \mid \underbrace{0100000000}_{\text{mantissa}}.$$

C.3.2. Absorption in FP16 Addition

Consider adding 1024.0 and 0.5 in half precision. The value $1024.0 = 1.0 \times 2^{10}$ is stored as

$$\underbrace{0}_{\text{sign}} \mid \underbrace{11001}_{\text{exp=25}} \mid \underbrace{0000000000}_{\text{mantissa}},$$

while $0.5 = 1.0 \times 2^{-1}$ is stored as

$$\underbrace{0}_{\text{sign}} \mid \underbrace{01110}_{\text{exp=14}} \mid \underbrace{0000000000}_{\text{mantissa}}.$$

To perform the addition, the hardware aligns the exponents by shifting the smaller operand's significand to the right by $10 - (-1) = 11$ positions:

$$0.5 = \underbrace{0.00000000001}_{\text{11 places after the radix point}} \times 2^{10}.$$

Since FP16 retains only 10 fraction bits after the implicit leading 1, the shifted significand falls entirely outside the representable precision and is rounded to zero. The result is 1024.0—the smaller operand has been *absorbed*.

Had the same computation been carried out in FP32 (23 fraction bits), the shift of 11 places is well within the available precision and the result is correctly obtained as 1024.5. This example illustrates why mixed-precision strategies—performing arithmetic at higher precision than the storage format—can recover accuracy at modest cost.

C.3.3. TF32 Truncation in a Matrix Multiply

The A100 tensor cores accept FP32 inputs but internally truncate the significand from 23 to 10 fraction bits before performing the multiply, while the accumulation remains in full FP32. To see the effect, consider two 2×2 matrices with entries drawn from familiar constants:

$$A = \begin{pmatrix} 3.1415927 & 1.2345679 \\ 2.7182817 & 0.9876543 \end{pmatrix}, \qquad B = \begin{pmatrix} 0.9876543 & 2.7182817 \\ 1.2345679 & 3.1415927 \end{pmatrix}.$$

After TF32 truncation, the entries lose their lower 13 significand bits:

$$\tilde{A} = \begin{pmatrix} 3.1406250 & 1.2343750 \\ 2.7167969 & 0.9873047 \end{pmatrix}, \qquad \tilde{B} = \begin{pmatrix} 0.9873047 & 2.7167969 \\ 1.2343750 & 3.1406250 \end{pmatrix}.$$

The tensor core computes $C_{\mathrm{TF32}} = \tilde{A}\,\tilde{B}$ with the multiplies at TF32 precision and the accumulation in FP32, giving

$$C_{\mathrm{TF32}} = \begin{pmatrix} 4.6244 & 12.4091 \\ 3.9010 & 10.4817 \end{pmatrix},$$

whereas exact FP32 arithmetic yields

$$C_{\mathrm{FP32}} = A\,B = \begin{pmatrix} 4.6270 & 12.4182 \\ 3.9040 & 10.4919 \end{pmatrix}.$$

The relative error in each entry is on the order of 5×10^{-4} to 10^{-3}, consistent with TF32's machine epsilon of $\varepsilon \approx 9.8 \times 10^{-4}$.

C.3.4. Fused Multiply-Add: Single vs. Double Rounding

Modern GPU floating-point units implement the *fused multiply-add* (FMA) operation

$$\mathrm{fma}(a, b, c) = \mathrm{round}(a \times b + c),$$

where the product $a \times b$ is computed to *full* (unrounded) precision before the addition and only a single rounding step is applied to the final result. By contrast, the non-fused sequence

$$t = \mathrm{round}(a \times b), \qquad r = \mathrm{round}(t + c)$$

incurs two rounding errors. The difference is most visible when $a \times b$ and c are of similar magnitude but opposite sign, so that their sum is much smaller than either operand.

As a concrete example, let $a = b = 1 + 2^{-12}$ and $c = -(1 + 2^{-11})$, all exactly representable in FP32. The exact product is

$$a \times b = (1 + 2^{-12})^2 = 1 + 2^{-11} + 2^{-24},$$

so the exact result is $a \times b + c = 2^{-24} \approx 5.96 \times 10^{-8}$. In the non-fused path, the intermediate rounding of $a \times b$ to FP32 discards the 2^{-24} term (which is below one unit in the last place at magnitude ≈ 1), yielding $t = 1 + 2^{-11}$, and consequently $r = t + c = 0$—a complete loss of the true result. The FMA, retaining the full product before the addition, returns 2^{-24} correctly.

D. Supplementary Hardware Context

This appendix collects hardware/economic background and extended node-path modeling details that support interpretation but are not required for reading the main optimisation narrative.

D.1. GA100 Full Die Floorplan

Figure D.1 reproduces the GA100 floorplan from the main text (Figure 2.2) at full page width for legibility. All GPC, memory-controller, L2, and NVLink/PCIe labels are readable at this scale.

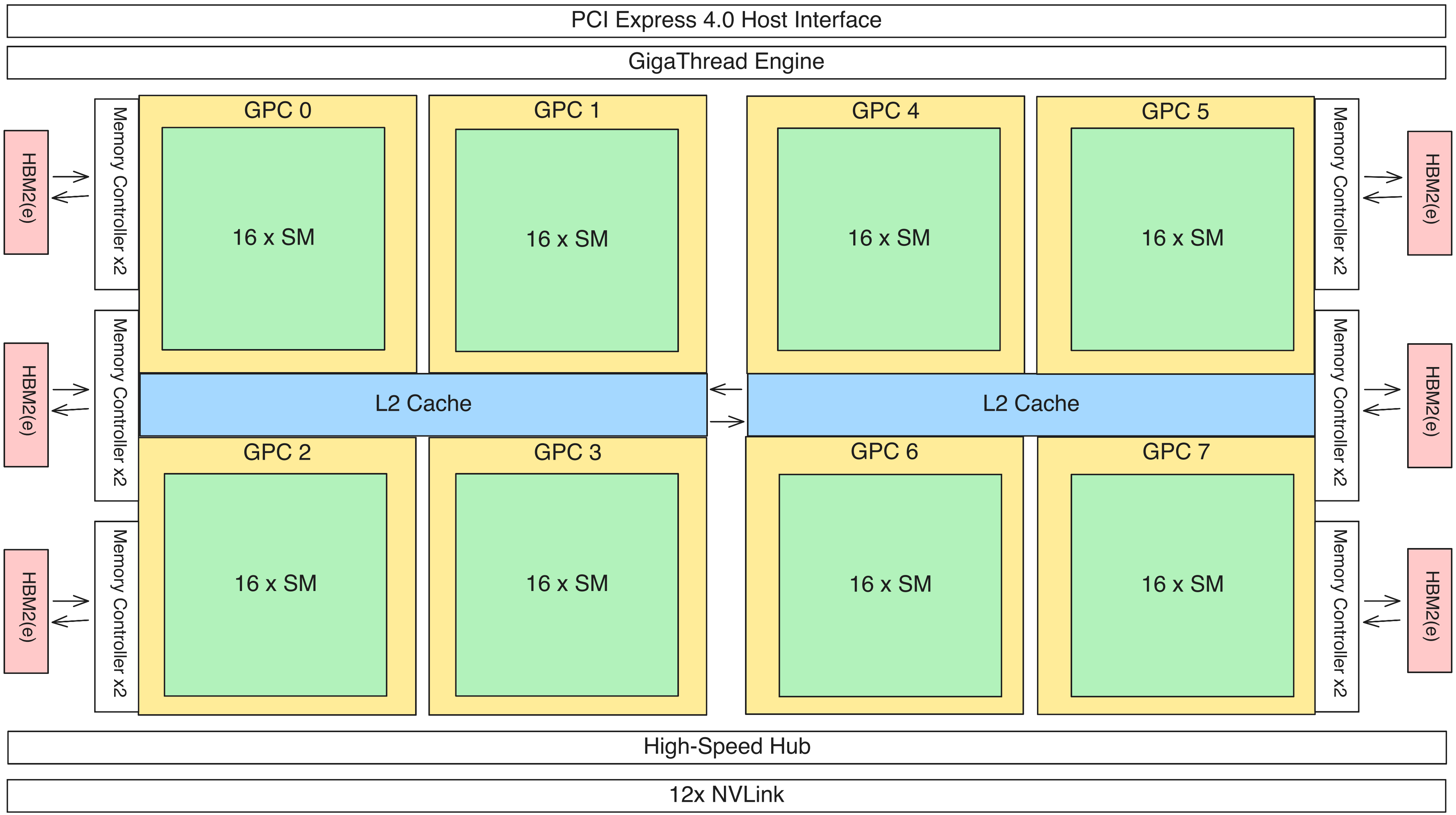


Figure D.1.: Full GA100 floorplan schematic. Graphics processing clusters (GPCs), L2 cache fabric, memory controllers, and NVLink/PCIe interfaces are shown. Compare with Figure 2.2 in the main text and the per-SM detail in Figure 2.3.

D.2. GA100 Yield Sensitivity (TSMC N7, Approximate)

GA100 is a very large monolithic die (about 826 mm^2) with 128 physical SM partitions in the full design. Shipping A100 SXM4-40GB parts expose 108 active SMs (20 disabled) [NVI20a, NVI20b]. This section gives a first-order yield sensitivity estimate to provide economic context for why large dies are commonly harvested/binned.

Using the standard first-order Poisson defect model for full-die yield,

$$Y_{\text{full}}(A, D_0) \approx e^{-AD_0},$$

where A is die area in cm^2 and D_0 is defect density in defects/cm^2. For GA100, $A = 8.26$.

To approximate harvesting allowances, we also model the 128 physical SM partitions as equal critical regions with area $a_{\text{SM}} = A/128$. Under the same Poisson assumption, one partition survives with probability

$$q(D_0) = e^{-a_{\text{SM}} D_0},$$

so the number of broken partitions K is approximated as $K \sim \text{Binomial}(128,\, 1 - q)$. The corresponding harvested-yield curves are

$$Y_{\leq k}(D_0) = \Pr[K \leq k] = \sum_{i=0}^{k} \binom{128}{i} (1-q)^i\, q^{128-i},$$

for selected allowances $k \in \{0, 1, 2, 3, 4, 20\}$. The case $k = 0$ recovers Y_{full}. The $k = 20$ curve corresponds to the common A100 shipping configuration with 108 active SM partitions out of 128.

Absolute TSMC N7 defect-density time series are not publicly released in full. However, industry-reported anchor values from public disclosures place $D_0 \approx 0.33$ defects/cm^2 in a pre-MP reference regime and $D_0 \approx 0.09$ defects/cm^2 for N7 around three quarters after high-volume manufacturing ramp [Mel24, Fru20]. These two regimes are retained as the marker points in Figure D.2.

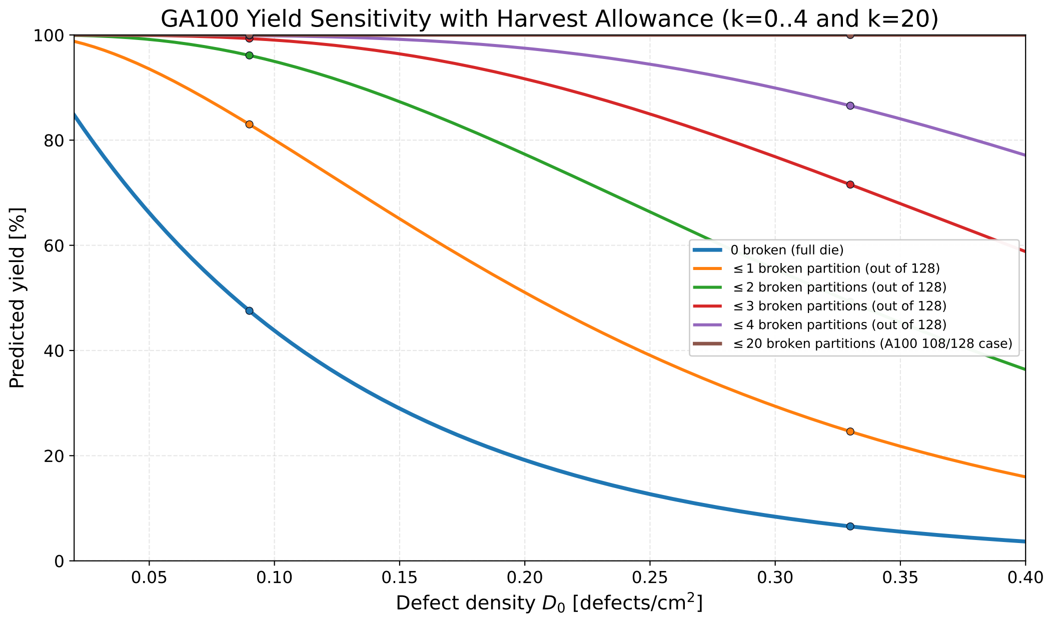


Figure D.2.: First-order defect-limited yield sensitivity for GA100 area ($826\,\mathrm{mm}^2$) using harvest allowances $k \in \{0, 1, 2, 3, 4, 20\}$ broken partitions out of 128. Marker points use industry-reported N7 anchor values ($D_0 \approx 0.33$ and $D_0 \approx 0.09$).

At those two anchor points, the full-die model gives

$$Y_{\text{full}}(D_0 = 0.33) \approx 6.6\%, \qquad Y_{\text{full}}(D_0 = 0.09) \approx 47.5\%.$$

Under the same simplified partition model, allowing at most four broken partitions raises the predicted values to about 86.5% and 99.9%, respectively. Allowing at most twenty broken partitions (the 108/128-SM case) saturates the model near 100% at both anchor points. This should not be interpreted as NVIDIA's product yield: shipping yield is higher due to redundancy, harvesting (disabled units), and performance binning. The model is used here only to quantify the sensitivity of large-die economics to defect density.

Area increment sanity check. For a die-area increase ΔA, Poisson yield scales as

$$\frac{Y(A + \Delta A, D_0)}{Y(A, D_0)} = e^{-D_0 \Delta A}.$$

With $\Delta A = 0.1\,\mathrm{cm}^2$, this multiplier is about 0.968 at $D_0 = 0.33$ and 0.991 at $D_0 = 0.09$, i.e. a 3.2 % and 0.9 % relative yield drop, respectively.

D.3. Extended Host–Device Path Cost Model

For a transfer of size V bytes, we use a first-order latency+bandwidth model in the standard Hockney form [HJ88]:

$$T_{\text{H2D}}(\pi, V) \approx T_0(\pi) + \frac{V}{B_{\text{eff}}(\pi)},$$

where π is the path class (local, same-socket remote, or cross-socket remote), T_0 is the fixed setup/latency term, and B_{eff} is the effective end-to-end bandwidth.

To make the topology penalty explicit, we write

$$T_0(\pi) = T_0^{\text{local}} + n_{\text{IF}}(\pi)\,\tau_{\text{IF}},$$

with n_{IF} the number of additional Infinity Fabric segments and τ_{IF} an average per-segment latency penalty. This gives the expected ordering

$$T_{\text{local}} < T_{\text{same socket remote}} < T_{\text{cross socket}}.$$

Concrete mapping for this node. Using the measured affinity in Table 2.16, the path classes can be made explicit. For example, for GPU 1 (local NUMA 1):

- **Local path** ($\pi = \pi_{\text{local}}$): thread and pinned host buffer on NUMA 1 (cores 16–31), modeled with $n_{\text{IF}} = 0$.
- **Same-socket remote path** ($\pi = \pi_{\text{ss}}$): thread/buffer on NUMA 3 (same socket, cores 48–63), modeled with one additional fabric segment ($n_{\text{IF}} \approx 1$).
- **Cross-socket path** ($\pi = \pi_{\text{xs}}$): thread/buffer on NUMA 5 or NUMA 7 (other socket), modeled with inter-socket traversal plus local fabric ($n_{\text{IF}} \approx 2$ in this simplified model).

The same local/same-socket/cross-socket classification applies to GPU 0, GPU 2, and GPU 3 by replacing NUMA IDs with their entries from Table 2.16. The practical expectation is

$$B_{\text{eff}}(\pi_{\text{local}}) > B_{\text{eff}}(\pi_{\text{ss}}) > B_{\text{eff}}(\pi_{\text{xs}}),$$

and correspondingly higher transfer time for the same V.

Cost instantiation for this architecture. For topology-aware staging costs, we model a sequence of N host-device transfers with total volume $V_{\text{tot}} = \sum_{i=1}^{N} V_i$ as

$$T_{\text{stage}}(\pi) = \sum_{i=1}^{N} T_{\text{H2D}}(\pi, V_i) \approx N\,T_0(\pi) + \frac{V_{\text{tot}}}{B_{\text{eff}}(\pi)}.$$

With the path classes used in this node model ($n_{\text{IF}}(\pi_{\text{local}}) = 0$, $n_{\text{IF}}(\pi_{\text{ss}}) = 1$, $n_{\text{IF}}(\pi_{\text{xs}}) = 2$), this becomes

$$\begin{aligned}
T_{\text{stage}}(\pi_{\text{local}}) &\approx N\,T_0^{\text{local}} + \frac{V_{\text{tot}}}{B_{\text{eff}}(\pi_{\text{local}})},\\
T_{\text{stage}}(\pi_{\text{ss}}) &\approx N\Big(T_0^{\text{local}} + \tau_{\text{IF}}\Big) + \frac{V_{\text{tot}}}{B_{\text{eff}}(\pi_{\text{ss}})},\\
T_{\text{stage}}(\pi_{\text{xs}}) &\approx N\Big(T_0^{\text{local}} + 2\tau_{\text{IF}}\Big) + \frac{V_{\text{tot}}}{B_{\text{eff}}(\pi_{\text{xs}})}.
\end{aligned}$$

Relative to local placement, the predicted penalties are therefore

$$\begin{aligned}
\Delta T_{\text{ss}} &= T_{\text{stage}}(\pi_{\text{ss}}) - T_{\text{stage}}(\pi_{\text{local}}) \approx N\,\tau_{\text{IF}} + V_{\text{tot}}\left(\frac{1}{B_{\text{eff}}(\pi_{\text{ss}})} - \frac{1}{B_{\text{eff}}(\pi_{\text{local}})}\right),\\
\Delta T_{\text{xs}} &= T_{\text{stage}}(\pi_{\text{xs}}) - T_{\text{stage}}(\pi_{\text{local}}) \approx 2N\,\tau_{\text{IF}} + V_{\text{tot}}\left(\frac{1}{B_{\text{eff}}(\pi_{\text{xs}})} - \frac{1}{B_{\text{eff}}(\pi_{\text{local}})}\right).
\end{aligned}$$

How parameters are obtained in practice. For each path class π, measure transfer times for multiple sizes $V \in [V_{\min}, V_{\max}]$ and fit

$$T(V) = a_\pi + b_\pi V, \qquad T_0(\pi) = a_\pi, \quad B_{\text{eff}}(\pi) = \frac{1}{b_\pi}.$$

This linear fit is the standard way to instantiate latency+bandwidth models in system-performance analysis [HJ88, Jai91]. Once fitted, the above expressions directly provide predicted staging overhead for any (N, V_{tot}) in the workload.

Per-GPU path classes. The local and same-socket "remote" NUMA domains used for orchestration are:

GPU	π_{local}	π_{ss} (used)	π_{xs} (used)
GPU 0	NUMA 3	NUMA 1	NUMA 5 or NUMA 7
GPU 1	NUMA 1	NUMA 3	NUMA 5 or NUMA 7
GPU 2	NUMA 7	NUMA 5	NUMA 1 or NUMA 3
GPU 3	NUMA 5	NUMA 7	NUMA 1 or NUMA 3

Table D.1.: Path-class mapping used for topology-aware cost evaluation.

For each row in Table D.1, the same $n_{\text{IF}} = \{0, 1, 2\}$ model applies, so the transfer-cost expressions above are directly reusable across all four GPUs.

Declaration of Authorship

I hereby declare that I have created this work completely on my own and used no other sources or tools than the ones listed, and that I have marked any quotes accordingly.

Aachen, March 6, 2026 Daniel Sinkin